



INSTITUTE OF AERONAUTICAL ENGINEERING
(Autonomous)
Dundigal, Hyderabad - 500 043

Lab Manual

OBJECT ORIENTED PROGRAMMING WITH JAVA LABORATORY (ACSD02)

Prepared by

DR.C V RAMA PADMAJA (IARE10879)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
INSTITUTE OF AERONAUTICAL ENGINEERING

2023-24

Contents

Content	vi
1 INTRODUCTION	1
1.1 Introduction	1
1.1.1 Student Responsibilities	1
1.1.2 Responsibilities of Faculty Teaching the Lab Course	1
1.1.3 Laboratory In-charge Responsibilities	2
1.1.4 Course Coordinator Responsibilities	2
1.2 Lab Policy and Grading	2
Grading Policy:	2
Pre-Requisites and Co-Requisites:	2
1.3 Course Goals and Objectives	3
1.4 Use of Hardware and Software Tools	3
1.4.1 Computer System	3
1.4.2 Text Editor or IDE	4
1.4.3 Java Development Kit (JDK)	4
1.4.4 Operating System	4
1.4.5 Console/Command Prompt	4
1.4.6 Internet Connectivity	5
1.4.7 Web Browser	5
1.4.8 Safety Precautions	5
1.5 Data Recording and Reports	5
1.5.1 The Laboratory Worksheets	5
1.5.2 The Laboratory Files/Reports	5
2 Getting Started Exercises	6
2.1 Introduction	6
2.2 Pre-Lab Preparation	6
2.2.1 Objective	6
2.2.2 Background	6
Syntax and Structure:	6
Variables and Numerical Operations:	6
Printing Output:	7
Conditional Statements:	7
2.2.3 Hardware/Software Requirements	7
2.2.4 Pre-Lab Questions	7
2.3 Experiment Procedure	7
2.4 Experiemnts, Results & Discussion	8
2.4.1 HelloWorld	8
2.4.2 Writing Good Programs(Coding Conventions & Programming Practices) .	8
2.4.3 CheckPassFail	12
2.4.4 CheckOddEven	12

2.4.5	PrintNumberInWord	13
2.4.6	PrintDayInWord	14
2.5	Post-Lab Viva voce	14
2.6	Further Probing Experiments	15
3	EXERCISES ON DECISION AND LOOP	16
3.1	Introduction	16
3.2	Pre-Lab Preparation	16
3.2.1	Objectives	16
3.2.2	Background	16
	while loop	17
	do-while loop	17
	for loop	17
3.2.3	Hardware/Software Requirements	18
3.2.4	Pre-Lab Questions	18
3.3	Experiment Procedure	18
3.4	Experiments, Results & Discussion	18
3.4.1	SumAverageRunningInt	18
3.4.2	Product1ToN	20
3.4.3	HarmonicSum	21
3.4.4	ComputePI	22
3.4.5	CozaLozaWoza	23
3.4.6	Fibonacci	25
3.4.7	ExtractDigits	26
3.5	Post-Lab Viva voce	26
3.6	Further Probing Experiments	27
4	EXERCISES ON INPUT, DECISION AND LOOP	28
4.1	Introduction	28
4.2	Prelab Preparation	28
4.2.1	Objective	28
4.2.2	Background	28
4.2.3	Hardware/Software Requirements	29
4.2.4	Pre-Lab Questions	29
4.3	Experiment Procedure	29
4.4	Experiemnts	30
4.4.1	Add2Integer	30
4.4.2	SumProductMinMax3	31
4.4.3	CircleComputation	32
4.4.4	Swap2Integers	33
4.4.5	IncomeTaxCalculator	33
4.4.6	IncomeTaxCalculatorWithSentinel	35
4.4.7	PensionContributionCalculatorWithSentinel	36
4.4.8	SalesTaxCalculator	38
4.4.9	ReverseInt	39
4.4.10	SumOfDigitsInt	40
4.4.11	InputValidation	41
4.4.12	AverageWithInputValidation	42
4.5	Post-Lab Viva Voce Questions	43
4.6	Further Probing Experiments	43
5	EXERCISES ON NESTED-LOOPS	44

5.1	Introduction	44
5.2	Prelab Preparation	44
5.2.1	Objective	44
5.2.2	Background	44
5.2.3	Hardware/Software Requirements	45
5.2.4	Pre-Lab Questions	45
5.3	Experiment Procedure	45
5.4	Experiemnts	46
5.4.1	SquarePattern (nested-loop)	46
5.4.2	SquarePattern (nested-loop)	47
5.4.3	MultiplicationTable (nested-loop)	47
5.4.4	TriangularPattern (nested-loop)	48
5.4.5	BoxPattern (nested-loop)	51
5.4.6	HillPattern (nested-loop)	53
5.4.7	NumberPattern (nested-loop)	55
5.5	Sample Viva Voce Questions	58
5.6	Further Probing Experiments	58
6	MAGIC(SPECIAL) NUMBERS	60
6.1	Introduction	60
6.2	Prelab Preparation	60
6.2.1	Objective	60
6.2.2	Background	60
6.2.3	Hardware/Software Requirements	61
6.2.4	Pre-Lab Questions	62
6.3	Experiment Procedure	62
6.4	Experiemnts	62
6.4.1	AmicableNumbers	62
6.4.2	ArmstrongNumber	63
6.4.3	CapricornNumber	64
6.4.4	CircularPrime	65
6.4.5	HappyNumber	67
6.4.6	AutomorphicNumber	68
6.4.7	DisariumNumber	69
6.4.8	Magic Number	70
6.4.9	NeonNumber	71
6.4.10	PalindromicNumber	71
6.4.11	PerfectNumber	72
6.4.12	SpecialNumber	73
6.4.13	SpyNumber	74
6.4.14	UglyNumber	75
6.5	Sample Viva Voce Questions	76
6.6	Further Probing Experiments	76
7	EXERCISES ON STRING AND CHAR OPERATIONS	77
7.1	Introduction	77
7.2	Prelab Preparation	77
7.2.1	Objective	77
7.2.2	Background	77
7.2.3	Hardware/Software Requirements	79
7.2.4	Pre-Lab Questions	79
7.3	Experiment Procedure	79

7.4	Experiments, Results & Discussion	80
7.4.1	ReverseString (String & char)	80
7.4.2	CountVowelsDigits (String & char)	80
7.4.3	PhoneKeyPad (String & char)	81
7.4.4	Caesar's Code (String & char)	83
7.4.5	Decipher Caesar's Code (String & char)	83
7.4.6	Exchange Cipher (String & char)	84
7.4.7	TestPalindromicWord (String & char)	85
7.4.8	CheckBinStr (String & char)	86
7.4.9	CheckHexStr (String & char)	87
7.4.10	Bin2Dec (String & char)	88
7.4.11	Hex2Dec (String & char)	89
7.4.12	Oct2Dec (String & char)	90
7.5	Sample Viva Voce Questions	91
7.6	Further Probing Experiments	91
8	EXERCISES ON ARRAYS	92
8.1	Introduction	92
8.2	Prelab Preparation	92
8.2.1	Objective	92
8.2.2	Background	92
8.2.3	Hardware/Software Requirements	93
8.2.4	Pre-Lab Questions	93
8.3	Experiment Procedure	93
8.4	Experiments, Results & Discussion	94
8.4.1	PrintArray (Array)	94
8.4.2	PrintArrayInStars (Array)	95
8.4.3	GradesStatistics (Array)	96
8.4.4	Hex2Bin (Array for Table Lookup)	97
8.4.5	Dec2Hex (Array for Table Lookup)	98
8.5	Sample Viva Voce Questions	99
8.6	Further Probing Experiments	99
9	EXERCISES ON METHODS	100
9.1	Introduction	100
9.2	Prelab Preparation	100
9.2.1	Objective	100
9.2.2	Background	100
9.2.3	Hardware/Software Requirements	102
9.2.4	Pre-Lab Questions	102
9.3	Experiment Procedure	103
9.4	Experiments, Results & Discussion	103
9.4.1	exponent() (method)	103
9.4.2	isOdd() (method)	104
9.4.3	hasEight() (method)	105
9.4.4	print() (Array & Method)	106
9.4.5	arrayToString() (Array & Method)	108
9.4.6	contains() (Array & Method)	109
9.4.7	search() (Array & Method)	110
9.4.8	equals() (Array & Method)	110
9.4.9	copyOf() (Array & Method)	111
9.4.10	swap() (Array & Method)	113

9.4.11	reverse() (Array & Method)	114
9.4.12	GradesStatistics (Array & Method)	115
9.4.13	GradesHistogram (Array & Method)	117
9.5	Sample Viva Voce Questions	119
9.6	Further Probing Experiments	120
10	EXERCISES ON COMMAND-LINE ARGUMENTS, RECURSION	121
10.1	Introduction	121
10.2	Prelab Preparation	121
10.2.1	Objective	121
10.2.2	Background	121
10.2.3	Hardware/Software Requirements	123
10.2.4	Pre-Lab Questions	123
10.3	Experiment Procedure	123
10.4	Experiemnts	124
10.4.1	Arithmetic (Command-Line Arguments)	124
10.4.2	FactorialRecursive	125
10.4.3	Fibonacci (Recursive)	126
10.4.4	Length of a Running Number Sequence (Recursive)	127
10.4.5	GCD (Recursive)	129
10.5	Sample Viva Voce Questions	130
10.6	Further Probing Experiments	130
11	MORE (DIFFICULT) EXERCISES	131
11.1	Introduction	131
11.2	Prelab Preparation	131
11.2.1	Objective	131
11.2.2	Background	131
11.2.3	Hardware/Software Requirements	132
11.2.4	Pre-Lab Questions	132
11.3	Experiment Procedure	132
11.4	Experiemnts	133
11.4.1	JDK Source Code	133
11.4.2	Matrices (2D Arrays)	135
11.4.3	PrintAnimalPattern (Special Characters and Escape Sequences)	140
11.4.4	Print Patterns (nested-loop)	141
11.4.5	Print Triangles (nested-loop)	146
11.4.6	Trigonometric Series	148
11.4.7	Exponential Series	150
11.4.8	Special Series	151
11.4.9	FactorialInt (Handling Overflow)	152
11.4.10	FibonacciInt (Handling Overflow)	153
11.4.11	Number System Conversion	155
11.4.12	NumberGuess	157
11.4.13	WordGuess	158
11.4.14	DateUtil	160
11.5	Sample Viva Voce Questions	162
11.6	Further Probing Experiments	163
12	EXERCISES ON CLASSES AND OBJECTS	164
12.1	Introduction	164
12.2	Prelab Preparation	164

12.2.1	Objective	164
12.2.2	Background	164
12.2.3	Hardware/Software Requirements	166
12.2.4	Pre-Lab Questions	167
12.3	Experiment Procedure	167
12.4	Experiemnts	167
12.4.1	The Rectangle Class	167
12.4.2	The Employee Class	169
12.4.3	The InvoiceItem Class	172
12.4.4	The Account Class	174
12.4.5	The Date Class	177
12.5	Sample Viva Voce Questions	179
12.6	Further Probing Experiments	179
13	EXERCISES ON INHERITANCE	181
13.1	Introduction	181
13.2	Prelab Preparation	181
13.2.1	Objective	181
13.2.2	Background	181
13.2.3	Hardware/Software Requirements	182
13.2.4	Pre-Lab Questions	182
13.3	Experiment Procedure	182
13.4	Experiments	183
13.4.1	The Circle and Cylinder Classes	183
13.4.2	Superclass Person and its subclasses	187
13.4.3	Point2D and Point3D	191
13.4.4	Point and MovablePoint	194
13.4.5	Superclass Shape and its subclasses Circle, Rectangle and Square	198
13.5	Sample Viva Voce Questions	204
13.6	Further Probing Experiments	204
14	EXERCISES ON POLYMORPHISM, ABSTRACT CLASSES AND INTERFACES	205
14.1	Introduction	205
14.2	Prelab Preparation	205
14.2.1	Objective	205
14.2.2	Background	205
14.2.3	Hardware/Software Requirements	206
14.2.4	Pre-Lab Questions	206
14.3	Experiment Procedure	206
14.4	Experiemnts	207
14.5	Experiments	207
14.5.1	Abstract Superclass Shape and Its Concrete Subclasses	207
14.5.2	GeometricObject Interface and its Implementation Classes Circle and Rectangle	217
14.5.3	Movable Interface and its Implementation MovablePoint Class	220
14.5.4	Movable Interface and Classes MovablePoint and MovableCircle	223
14.5.5	Interfaces Resizable and GeometricObject	227
14.5.6	Abstract Superclass Animal and its Implementation Subclasses	231
14.6	Sample Viva Voce Questions	234
14.7	Further Probing Experiments	234

INTRODUCTION

1.1 Introduction

This laboratory course is designed to complement and enrich the academic journey of students in the field of Object-Oriented Programming through Java, a subject closely intertwined with the broader context of Computer Science Engineering. Building upon the foundational knowledge acquired in the classroom, this practical laboratory is tailored to equip students with essential hands-on skills vital for modern software development. Throughout this course, students will engage with a diverse range of experiments and exercises, to explore the working of Java programming. The primary objective is to bridge the gap between theory and practice, allowing students to not only apply the concepts introduced in the classroom but also gain a deeper understanding through direct experimentation. Success in this laboratory course hinges on students' active participation and diligent preparation, as they navigate through a series of experiments that encompass the principles of Object-Oriented Programming. Moreover, it is a collaborative effort, with responsibilities shared among the students, faculty, laboratory in-charge, and faculty coordinator, all working together to ensure the attainment of the lab's educational objectives. As students progress through this course, they will not only enhance their practical skills but also cultivate a profound appreciation for the vital role of experimental learning in Computer Science Engineering.

1.1.1 Student Responsibilities

The student is expected to be prepared for each lab. Lab preparation includes reading the lab experiment and related textbook material. If you have questions or problems with the preparation, contact your Laboratory Teaching faculty, but in a timely manner. Do not wait until an hour or two before the lab and then expect the lab faculty to be immediately available.

Active participation by each student in lab activities is expected. The student is expected to ask the laboratory faculty any questions they may have. A large portion of the student's grade is determined in the comprehensive final exam, resulting in a requirement of understanding the concepts and procedure of each lab experiment for the successful completion of the lab class.

The student should remain alert and use common sense while performing a lab experiment. They are also responsible for keeping a professional and accurate record of the lab experiments in the lab manual wherever tables are provided. Students should report any errors in the lab manual to the teaching assistant.

1.1.2 Responsibilities of Faculty Teaching the Lab Course

The laboratory faculty shall be completely familiar with each lab prior to class. The laboratory faculty shall provide the students with a syllabus and safety review during the first class. The syllabus shall include the laboratory faculty office hours, telephone number, and the name of the faculty coordinator. The laboratory faculty is responsible for ensuring that all the necessary equipment and/or preparations for the lab are available and in working condition. Lab experiments should be checked in advance to make sure everything is in working order. The

laboratory faculty should fully answer any questions posed by the students and supervise the students performing the lab experiments. The laboratory faculty is expected to grade the lab notebooks and reports in a fair and timely manner. The reports should be returned to the students in the next lab period following submission. The laboratory faculty should report any errors in the lab manual to the faculty coordinator.

1.1.3 Laboratory In-charge Responsibilities

The Laboratory In-charge should ensure that the laboratory is properly equipped, i.e., the Faculty teaching the lab receive any equipment/components necessary to perform the experiments. He/She is responsible for ensuring that all the necessary equipment for the lab is available and in working condition. The Laboratory In-charge is responsible for resolving any problems that are identified by the teaching Faculty or the students.

1.1.4 Course Coordinator Responsibilities

The faculty coordinator should ensure that the Object Oriented Programming with Java laboratory is properly arranged. The coordinator is responsible for supervising the teaching assistants and resolving any questions or problems that are identified by the teaching assistants or the students. The coordinator may supervise the format of the final exam for the lab. They are also responsible for making any necessary corrections to this manual and ensuring that it is continually updated and available.

1.2 Lab Policy and Grading

The student should understand the following policy:

ATTENDANCE: Attendance is mandatory as per the academic regulations.

LAB RECORD's: The student must:

1. Write the work sheets for the allotted experiment and keep them ready before the beginning of each lab.
2. Keep all work in preparation of and obtained during lab.
3. Perform the experiment and record the observations in the worksheets.
4. Analyze the results and get the work sheets evaluated by the Faculty.
5. Upload the evaluated reports online from CMS LOGIN within the stipulated time.

Grading Policy:

The final grade of this course is awarded using the criterion detailed in the academic regulations. A large portion of the student's grade is determined in the comprehensive final exam of the Laboratory course (SEE PRACTICALS), resulting in a requirement of understanding the concepts and procedure of each lab experiment for successful completion of the lab course.

Pre-Requisites and Co-Requisites:

The lab course is to be taken during the same semester as ACSD02, but receives a separate grade. Students are required to have completed both ACSD01 and ACSD02 with minimum passing grade or better grade in each.

1.3 Course Goals and Objectives

The Object Oriented Programming with Java Laboratory course is designed as a foundation course to provide the student with the knowledge to understand the basic concepts in object oriented programming which have lot of applications in the field of computer science engineering.

The experiments are designed to complement the concepts introduced in ACSD01. In addition, the student should learn how to record experimental results effectively and present these results in a written report.

More explicitly, the class objectives are:

1. To gain proficiency in the use of Java development environment
2. To enhance understanding of theoretical concepts including:
 - Java syntax and program structure
 - Number systems
 - Variables and Naming conventions
 - Data types
 - Control Structures
 - Arrays and Strings
 - Functions, Recursion
 - Classes and Objects
 - Constructors and Destructors
 - Inheritance
 - Polymorphism
3. To develop communication skills through:
 - Verbal interchanges with the Faculty and other students.
 - Preparation of succinct, but complete laboratory reports.
 - Maintenance of laboratory worksheets as permanent, written descriptions of procedures, analysis and results.
4. To compare theoretical predictions with experimental results and to determine the source of any apparent errors.

1.4 Use of Hardware and Software Tools

laboratory instruments typically refer to the computer hardware and software tools used by students to conduct programming exercises and execute Java programs. These instruments play a crucial role in facilitating the learning experience and practical application of programming concepts. Here, we will explore the use of laboratory instruments specific to this lab:

1.4.1 Computer System

The primary laboratory instrument is the computer system itself. Students use computers to write, compile, and execute Java programs. The computer's processing power, memory, and storage capacity are essential for running the Integrated Development Environment (IDE), text editors, and the Java Development Kit (JDK) smoothly.

1. Ensure that your computer is in good working condition and free from malware or viruses.
2. Keep your computer charged or connected to a power source during lab sessions to avoid interruptions due to battery depletion.

1.4.2 Text Editor or IDE

A text editor or Integrated Development Environment (IDE) is an indispensable instrument for writing and editing Java code. Students can choose from various popular IDEs like Eclipse, IntelliJ IDEA, or Visual Studio Code. These tools provide features such as code highlighting, auto-completion, and debugging capabilities, making the programming process more efficient.

1. Choose an IDE or text editor that you are comfortable with and that supports Java development.
2. Familiarize yourself with the features of the chosen IDE or text editor, such as code auto-completion, debugging tools, and shortcuts.

1.4.3 Java Development Kit (JDK)

The JDK is a critical laboratory instrument as it provides the necessary tools, compilers, and libraries for Java programming. Students rely on the JDK to compile their Java source code into executable files and ensure compatibility with the Java Virtual Machine (JVM).

1. Install the latest version of the JDK compatible with your operating system.
2. Verify that the JDK is correctly configured, and the system's PATH variable includes the JDK's bin directory.

1.4.4 Operating System

The choice of the operating system (e.g., Windows, macOS, Linux) is a significant consideration as it affects the compatibility and behavior of Java programs. Different operating systems may require specific configurations for Java development.

1. Ensure your operating system is up-to-date with the latest security patches and updates.
2. Be aware of any specific instructions or configurations required for Java development on your chosen operating system.

1.4.5 Console/Command Prompt

The command prompt or terminal is used to run Java programs and compile source code. Students use this instrument to execute compiled Java programs and view the program's output on the console.

1. Practice navigating directories and executing Java programs from the command prompt or terminal.
2. Understand how to compile Java source code using the `javac` command and execute programs using the `java` command.

1.4.6 Internet Connectivity

A reliable internet connection can be a valuable instrument for students seeking additional resources, documentation, or online tutorials related to Java programming. It enables access to a wealth of information and support when facing programming challenges.

1. Maintain a reliable internet connection to access online resources, documentation, and tutorials related to Java programming.
2. Avoid downloading or installing software from untrusted sources to protect your computer from potential security risks.

1.4.7 Web Browser

Students may use web browsers to search for Java documentation, reference materials, and online forums for troubleshooting and learning additional programming concepts. The web browser serves as a research tool to enhance their understanding.

1. Use web browsers for accessing online documentation, and seeking help from programming forums and communities.
2. Be cautious when downloading code or software from the internet; ensure its source is reputable.

1.4.8 Safety Precautions

1. Ensure that you are working in a secure computing environment.
2. Save your work regularly to prevent data loss.

1.5 Data Recording and Reports

1.5.1 The Laboratory Worksheets

Students must write their experimental outputs in the provided tables in this laboratory manual and reproduce them in the lab reports. Reports are integral to recording the methodology and results of an experiment. In engineering practice, the laboratory notebook serves as an invaluable reference to the technique used in the lab and is essential when trying to duplicate a result or write a report. Therefore, it is important to learn to keep accurate data. Make plots of data and sketches when these are appropriate in the recording and analysis of observations. Note that the data collected will be an accurate and permanent record of the data obtained during the experiment and the analysis of the results. You will need this record when you are ready to prepare a lab report.

1.5.2 The Laboratory Files/Reports

Reports are the primary means of communicating your experience and conclusions to other professionals. In this course you will use the lab report to inform your laboratory faculty about what you did and what you have learned from the experience. Engineering results are meaningless unless they can be communicated to others. You will be directed by your laboratory faculty to prepare a lab report on lab experiments during the semester. Your laboratory report should be clear and concise. Your report will be the result of your individual effort in order to provide you with practice in technical communication.

Getting Started Exercises

2.1 Introduction

The purpose of this comprehensive laboratory experiment is to provide students with a foundational understanding of Java programming by guiding them through the creation and execution of various simple Java programs. This multifaceted lab serves as an essential starting point for beginners in the world of programming, introducing them to fundamental programming concepts and control structures in Java. Through a series of hands-on exercises, students will gain practical experience and develop their problem-solving skills, setting the stage for more advanced programming tasks.

2.2 Pre-Lab Preparation

2.2.1 Objective

By the end of this lab, students should be able to:

1. Understand the fundamental syntax and structure of Java programs.
2. Write, compile, and execute basic Java programs.
3. Learn how to work with variables and user input in Java.
4. Comprehend the logic and use of conditional statements for decision-making.
5. Gain exposure to programming tasks based on displaying messages and decision making.

2.2.2 Background

This laboratory experiment delves into fundamental concepts in Java programming, focusing on key topics essential for building a strong foundation in the language. As students progress through this lab, they will explore and gain hands-on experience in several core programming concepts:

Syntax and Structure:

Java's syntax and program structure form the backbone of any Java application. Understanding how to write and organize code in a Java program is crucial for effective programming.

Variables and Numerical Operations:

The concept of variables is central to programming. Students will work with variables to store and manipulate data. Numerical operations, such as checking for even or odd numbers, are common in programming. These operations teach students how to perform basic calculations and make decisions based on numeric values.

Printing Output:

Printing output to the console is a fundamental aspect of programming. Students will explore how to display information to the user effectively.

Conditional Statements:

Conditional statements, such as if and else, are powerful tools for decision-making within programs. Students will learn how to use these constructs to control the flow of their programs based on specified conditions.

Each of these concepts is vital for aspiring programmers to grasp, as they provide a solid foundation upon which more complex programs and applications can be built. By completing the exercises in this lab, students will gain practical knowledge that can be applied to a wide range of programming challenges in the future.

2.2.3 Hardware/Software Requirements

To successfully complete this lab, students will need access to the following:

1. A computer with a suitable text editor or Integrated Development Environment (IDE) installed, such as Eclipse, IntelliJ IDEA, or Visual Studio Code.
2. Java Development Kit (JDK) installed on the computer. Ensure that the JDK is correctly configured, and the system's PATH variable is set to include the JDK's bin directory.

2.2.4 Pre-Lab Questions

1. Have you installed a Java Development Kit (JDK) on your computer?
2. Do you know how to set the system's PATH variable to include the JDK's bin directory?
3. Are you familiar with how to launch a text editor or Integrated Development Environment (IDE) for Java programming?
4. Can you explain what a programming language is and why it's used?

2.3 Experiment Procedure

1. **Write the Source Code:** Enter the source codes(provided in Experiments section), using a programming text editor.

Save the source file as "xxxxxxx.java" (for example HelloWorld.java). A Java source file should be saved with a file extension of ".java". The filename shall be the same as the classname. Filename and classname are case-sensitive.

2. **Compile the Source Code:** Compile the source code "HelloWorld.java" into Java bytecode (or machine code) "HelloWorld.class" using JDK's Java Compiler "javac".

Start a CMD Shell (Windows) or Terminal (UNIX/Linux/macOS) and issue these commands:

```
// Change directory (cd) to the directory (folder)
// containing the source file "HelloWorld.java"
javac HelloWorld.java
```

3. **Run the Program:** Run the machine code using JDK's Java Runtime "java", by issuing this command:

```
java HelloWorld
```

It will display the output as **Hello World!**

2.4 Experiemnts, Results & Discussion

2.4.1 HelloWorld

Aim

Write a Java program to display "Hello World"

Program

```
public class HelloWorld{  
    public static void main(String args[]) {  
        System.out.println("Hello World!");  
    }  
}
```

Result

Hello World!

2.4.2 Writing Good Programs(Coding Conventions & Programming Practices)

Why Code Conventions

Code conventions are important to programmers for a number of reasons:

- 80% of the lifetime cost of a piece of software goes to maintenance.
- Hardly any software is maintained for its whole life by the original author.
- Code conventions improve the readability of the software, allowing engineers to understand new code more quickly and thoroughly.
- If you ship your source code as a product, you need to make sure it is as well packaged and clean as any other product you create.

Indentation Four spaces should be used as the unit of indentation. The exact construction of the indentation (spaces vs. tabs) is unspecified. Tabs must be set exactly every 8 spaces (not 4).

Line Length: Avoid lines longer than 80 characters, since they're not handled well by many terminals and tools. **Note:** Examples for use in documentation should have a shorter line length—generally no more than 70 characters.

Wrapping Lines: When an expression will not fit on a single line, break it according to these general principles:

- Break after a comma.
- Break before an operator.
- Prefer higher-level breaks to lower-level breaks.

- Align the new line with the beginning of the expression at the same level on the previous line.
- If the above rules lead to confusing code or to code that’s squished up against the right margin, just indent 8 spaces instead.

Comments Java programs can have two kinds of comments: implementation comments and documentation comments. Implementation comments are those found in C++, which are delimited by `/*...*/`, and `//`. Documentation comments (known as “doc comments”) are Java-only, and are delimited by `/**...*/`. Doc comments can be extracted to HTML files using the javadoc tool.

Implementation comments are mean for commenting out code or for comments about the particular implementation. Doc comments are meant to describe the specification of the code, from an implementation-free perspective. to be read by developers who might not necessarily have the source code at hand.

Comments should be used to give overviews of code and provide additional information that is not readily available in the code itself. Comments should contain only information that is relevant to reading and understanding the program. For example, information about how the corresponding package is built or in what directory it resides should not be included as a comment.

Block Comments: Block comments are used to provide descriptions of files, methods, data structures and algorithms. Block comments should be used at the beginning of each file and before each method. They can also be used in other places, such as within methods. Block comments inside a function or method should be indented to the same level as the code they describe. A block comment should be preceded by a blank line to set it apart from the rest of the code. Block comments have an asterisk “*” at the beginning of each line except the first.

```
/*
 * Here is a block comment.
 */
```

Single-Line Comments:Short comments can appear on a single line indented to the level of the code that follows. If a comment can’t be written in a single line, it should follow the block comment format (see section 5.1.1). A single-line comment should be preceded by a blank line. Here’s an example of a single-line comment in Java code:

```
if (condition) {
    /* Handle the condition. */
    ...
}
```

Trailing Comments:Very short comments can appear on the same line as the code they describe, but should be shifted far enough to separate them from the statements. If more than one short comment appears in a chunk of code, they should all be indented to the same tab setting. Avoid the assembly language style of commenting every line of executable code with a trailing comment. Here’s an example of a trailing comment in Java code

```
if (a == 2) {
    return TRUE; /* special case */
} else {
    return isprime(a); /* works only for odd a */
}
```

End-Of-Line Comments:The `//` comment delimiter begins a comment that continues to the newline. It can comment out a complete line or only a partial line. It shouldn’t be used on

consecutive multiple lines for text comments; however, it can be used in consecutive multiple lines for commenting out sections of code. Examples of all three styles follow:

```
if (foo > 1) {
    // Do a double-flip.
    ...
}
else
return false; // Explain why here.
//if (bar > 1) {
    //
    // // Do a triple-flip.
    // ...
    //}
//else
// return false;
```

Documentation Comments: Doc comments describe Java classes, interfaces, constructors, methods, and fields. Each doc comment is set inside the comment delimiters `/**...*/`, with one comment per API. This comment should appear just before the declaration:

```
/**
 * The Example class provides ...
 */
class Example { ...
```

Naming Conventions Naming conventions make programs more understandable by making them easier to read. They can also give information about the function of the identifier—for example, whether it’s a constant, package, or class—which can be helpful in understanding the code.

Identifier Type	Rules for Naming	Examples
Classes	Class names should be nouns, in mixed case with the first letter of each internal word capitalized. Try to keep your class names simple and descriptive. Use whole words—avoid acronyms and abbreviations (unless the abbreviation is much more widely used than the long form, such as URL or HTML).	class Raster; class ImageSprite;
Interfaces	Interface names should be capitalized like class names.	interface RasterDelegate; interface Storing;
Methods	Methods should be verbs, in mixed case with the first letter lowercase, with the first letter of each internal word capitalized.	run(); runFast(); getBackground();
Variables	Except for variables, all instance, class, and class constants are in mixed case with a lowercase first letter. Internal words start with capital letters. Variable names should be short yet meaningful. The choice of a variable name should be mnemonic— that is, designed to indicate to the casual observer the intent of its use. One-character variable names should be avoided except for temporary “throwaway” variables. Common names for temporary variables are i, j, k, m, and n for integers; c, d, and e for characters.	int i; char *cp; float myWidth;
Constants	The names of variables declared class constants and of ANSI constants should be all uppercase with words separated by underscores (“_”). (ANSI constants should be avoided, for ease of debugging.)	int MIN_WIDTH = 4; int MAX_WIDTH = 999; int GET_THE_CPU = 1;

TRY

Write a Java program to find the sum of two numbers. Provide enough comments while generating code.

2.4.3 CheckPassFail**Aim**

Write a program called CheckPassFail which prints "PASS" if the int variable "mark" is more than or equal to 50; or prints "FAIL" otherwise. The program shall always print "DONE" before exiting.

Program

```
public class CheckPassFail{ // Save as "CheckPassFail.java"
    public static void main(String[] args) {
        int marks = 49;
        System.out.println("The mark is " + marks);
        if ( marks>=50 ) {
            System.out.println( "Pass" );
        } else {
            System.out.println( "Fail" );
        }
        System.out.println( "DONE");
    }
}
```

Result

The mark is 49

Fail

DONE

TRY

marks = 0, 49, 50, 51, 100 and verify your results.

2.4.4 CheckOddEven**Aim**

Write a program called CheckOddEven which prints "Odd Number" if the int variable "number" is odd, or "Even Number" otherwise. The program shall always print "bye!" before exiting.

Program

```
public class CheckOddEven {

    //Trying if-else statement and modulus (%) operator.

    public static void main(String[] args) { // Program entry point
        int number = 49; // Set the value of "number" here!
        System.out.println("The number is " + number);
        if (number%2 == 0) {
```

```

        System.out.println(number + "is Even");    // even number
    } else {
        System.out.println( number + "is Odd" );    // odd number
    }
    System.out.println( "Bye");
}
}

```

Result

The number is 49

49is Odd

Bye

TRY

number = 0, 1, 88, 99, -1, -2 and verify your results.

2.4.5 PrintNumberInWord

Aim

Write a program called PrintNumberInWord which prints "ONE", "TWO",... , "NINE", "OTHER" if the int variable "number" is 1, 2,... , 9, or other, respectively. Use (a) a "else-if" statement; (b) a "switch-case-default" statement.

Program

```

public class PrintNumberInWord {
    public static void main(String[] args) {
        int number = 5; // Set the value of "number" here!
        // Using else-if
        if(number == 1){
            System.out.println("ONE");
        } else if(number == 2) {
            System.out.println("TWO");
        } else if (number == 3) {
            System.out.println("THREE");
        } else if (number == 4) {
            System.out.println("FOUR");
        } else if (number == 5) {
            System.out.println("FIVE");
        } else {
            System.out.println("Bad Number. Please Try Again");
        }
    }
}

```

Result

FIVE

TRY

1. Give diiferent values to number like 0, 1, 2, 3, ..., 9, 10 and verify your results.
2. Modify the program with *switch case*

2.4.6 PrintDayInWord

Aim

Write a program called PrintDayInWord which prints “Sunday”, “Monday”, ... “Saturday” if the int variable ”dayNumber” is 0, 1, ..., 6, respectively. Otherwise, it shall print ”Not a valid day”. Use (a) a ”nested-if” statement; (b) a ”switch-case-default” statement.

Program

```
public class PrintDayInWord {
    public static void main(String[] args) {
        int dayNumber = 5; // Set the value of "number" here!
        switch(dayNumber) {
            case 0:
                System.out.println("SUNDAY");
                break;
            case 1:
                System.out.println("MONDAY");
                break;
            case 2:
                System.out.println("TUESDAY");
                break;
            case 3:
                System.out.println("WEDNESDAY");
                break;
            case 4:
                System.out.println("THURSDAY");
                break;
            case 5:
                System.out.println("FRIDAY");
                break;
            case 6:
                System.out.println("SATURDAY");
                break;
            default: System.out.println("Try again");
        }
    }
}
```

Result

FRIDAY

TRY

1. Give different values for dayNumber like 0, 1, 2, 3, 4, 5, 6, 7 and verify your results.
2. Modify the program with *else if ladder*

2.5 Post-Lab Viva voce

1. What is the primary purpose of the ”Hello World” program in programming languages like Java?
2. Explain the difference between even and odd numbers in mathematics. How can you determine if a number is even or odd in Java?

3. What is a variable in programming, and why are they essential? Can you provide an example of declaring and initializing a variable in Java?
4. How do conditional statements, such as if and else, work in Java? Provide an example of when you might use an if-else statement in a program.
5. In Java, what is the purpose of the `System.out.println()` statement, and how do you use it to display output to the console?
6. What are numerical operations in programming, and how do they differ from mathematical operations? Provide an example of a numerical operation you might perform in Java.

2.6 Further Probing Experiments

1. **Convert miles into kilometers:** Miles and kilometers are two different units of distance measurement. To convert miles into kilometers, we can use a simple mathematical formula:
$$\text{kilometers} = \text{miles} * 1.60934$$

Write a Java program to convert the miles into kilometers.

2. **Simple interest** is a common financial calculation used to determine the interest earned or paid on a principal amount of money over a certain period of time. The formula for simple interest is:

$$\text{Simple Interest (SI)} = (\text{Principal} * \text{Rate} * \text{Time}) / 100$$
 Where:

Principal is the initial amount of money.

Rate is the rate of interest per year.

Time is the time period in years.

Write a Java program to calculate the simple interest earned or paid.

3. Print the roots of the quadratic equation. If there are two real roots, print both. If there is only one real root (a repeated root), print it once. If there are no real roots (complex roots), print a message indicating so. Write a Java Program to find and display the roots of a quadratic equation

EXERCISES ON DECISION AND LOOP

3.1 Introduction

In this session of the Java Lab, we delve deeper into Java programming, focusing on mathematical and computational concepts. This week's experiments will expand your programming skills as you explore topics such as summing and averaging integers, calculating factorials, understanding the harmonic series, and computing the mathematical constant Pi. These practical exercises are designed to enhance your Java proficiency while introducing key mathematical principles and computational techniques. Let's embark on this week's journey of professional and precise Java programming.

3.2 Pre-Lab Preparation

3.2.1 Objectives

By the end of this lab, students should be able to:

1. Understand of various looping structures in Java, such as for, while, and do-while loops, and be capable of selecting the most appropriate loop for a given task.
2. Apply their knowledge of conditional statements to solve real-world programming problems, including scenarios where multiple conditions need to be evaluated.
3. Calculate the harmonic sum of series and estimate the mathematical constant Pi using iterative methods, gaining insights into precision and approximation in numerical calculations.
4. Experiment manipulating numeric data, particularly in extracting and processing individual digits from numerical values.

3.2.2 Background

Loops are fundamental control structures in programming that enable the repetition of a set of instructions until a specified condition is met. They are crucial for automating repetitive tasks and processing data efficiently. In Java, loops are implemented using various constructs, including for, while, and do-while loops, each with its specific use cases and advantages.

The "Loops in Java" experiments provide an opportunity for students to deepen their understanding of looping structures in Java and gain hands-on experience in applying them to solve real-world programming problems.

while loop

A while loop in Java is a fundamental control structure that allows a block of code to be executed repeatedly as long as a specified condition remains true. It's used when you want to repeat a task an unknown or variable number of times, and the loop's continuation depends on a certain condition.

Here's the syntax of a while loop in Java:

```
while (condition) {  
    // Code to be executed while the condition is true  
}
```

condition: A Boolean expression that determines whether the loop should continue executing or terminate. As long as this condition remains true, the loop will keep running.

do-while loop

A do-while loop in Java is similar to a while loop, but with a crucial difference: the code block inside the do-while loop is executed at least once before checking the loop condition. After the code block executes, the loop condition is evaluated, and if it's true, the loop continues to run. If the condition is false, the loop terminates.

Here's the syntax of a do-while loop in Java:

```
do {  
    // Code to be executed at least once  
} while (condition);
```

condition: A Boolean expression that determines whether the loop should continue executing or terminate. After the code block is executed, the condition is checked. If it's true, the loop continues; if it's false, the loop terminates.

for loop

A for loop in Java is a control structure used for repeatedly executing a block of code a specific number of times. It's ideal when you know in advance how many times you want to loop.

Here's the syntax of a for loop in Java:

```
for (initialization; condition; iteration) {  
    // Code to be executed in each iteration  
}
```

initialization: This part is executed only once when the loop starts. It's often used to initialize loop control variables.

condition: The loop continues to execute as long as this condition is true. If the condition becomes false, the loop terminates.

iteration: This part is executed after each iteration of the loop. It's often used to update loop control variables.

3.2.3 Hardware/Software Requirements

To successfully complete this lab, students will need access to the following:

1. A computer with a suitable text editor or Integrated Development Environment (IDE) installed, such as Eclipse, IntelliJ IDEA, or Visual Studio Code.
2. Java Development Kit (JDK) installed on the computer. Ensure that the JDK is correctly configured, and the system's PATH variable is set to include the JDK's bin directory.

3.2.4 Pre-Lab Questions

1. What is the primary purpose of using loops in Java?
2. What are the three common types of loops in Java?
3. In a for loop, what is the role of the initialization statement?
4. How does a while loop differ from a do-while loop?

3.3 Experiment Procedure

1. Open Your Java Development Environment
2. Create a New Java Project
3. Create a Java Class
4. Write the Java Code
5. Compile Your Java Program
6. Execute Your Java Program
7. View the Output
8. Analyze the Output
9. Record your observations and any insights gained from this experiment.

3.4 Experiments, Results & Discussion

3.4.1 SumAverageRunningInt

Aim

Write a program called SumAverageRunningInt to produce the sum of 1, 2, 3, ..., to 100. Store 1 and 100 in variables lowerbound and upperbound, so that we can change their values easily. Also compute and display the average.

Program

```
/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */
public class SumAverageRunningInt {
```

```

public static void main (String[] args) {
    // Define variables
    int sum = 0;           // The accumulated sum, init to 0
    double average;       // average in double
    final int LOWERBOUND = 1;
    final int UPPERBOUND = 100;

    // Use a for-loop to sum from lowerbound to upperbound
    for (int number = LOWERBOUND; number <= UPPERBOUND; ++number) {
        // The loop index variable
        //number = 1, 2, 3, ..., 99, 100
        sum += number;    // same as "sum = sum + number"
    }
    // Compute average in double.
    // Beware that int / int produces int!

    average = sum/(UPPERBOUND-LOWERBOUND+1);
    // Print sum and average
    System.out.println("The sum of "+LOWERBOUND+" to "+UPPERBOUND+" is
        "+sum);
    System.out.println("The average is "+average);
}
}

```

Result

The sum of 1 to 100 is 5050

The average is 50.0

TRY

1. Modify the program to use a "while-do" loop instead of "for" loop.

HINTS:

```

int sum = 0;
int number = LOWERBOUND;           // declare and init loop index variable
while (number <= UPPERBOUND) {    // test
    sum += number;
    ++number;                      // update
}

```

2. Modify the program using do-while loop

HINTS:

```

int sum = 0;
int number = LOWERBOUND;           // declare and init loop index
    variable
do {
    sum += number;
    ++number;                      // update
} while (number <= UPPERBOUND);    // test

```

3. What is the difference between "for" and "while-do" loops? What is the difference between "while-do" and "do-while" loops?
4. Modify the program to sum from 111 to 8899, and compute the average. Introduce an int variable called count to count the numbers in the specified range (to be used in computing the average).

5. Modify the program to find the "sum of the squares" of all the numbers from 1 to 100, i.e. $1*1 + 2*2 + 3*3 + \dots + 100*100$.
6. Modify the program to produce two sums: sum of odd numbers and sum of even numbers from 1 to 100. Also computer their absolute difference.

3.4.2 Product1ToN

Aim

Write a program called Product1ToN to compute the product of integers from 1 to 10 (i.e., $1 \times 2 \times 3 \times \dots \times 10$), as an int. Take note that It is the same as factorial of N.

Program

```
/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */
public class Product1ToN {
    public static void main (String[] args) {
        // Define variables
        int fact = 1;
        final int LOWERBOUND = 1;
        final int UPPERBOUND = 10;

        for (int number = LOWERBOUND; number <= UPPERBOUND; ++number) {
            fact = fact*number;
        }
        System.out.println("The factorial of "+UPPERBOUND+" is "+fact);
    }
}
```

Result

The factorial of 10 is 3628800

TRY

1. Compute the product from 1 to 11, 1 to 12, 1 to 13 and 1 to 14. Write down the product obtained and decide if the results are correct.

HINTS:

Factorial of 13 (=6227020800) is outside the range of int [-2147483648, 2147483647]. Take note that computer programs may not produce the correct result even though the code seems correct!

2. Repeat the above, but use long to store the product. Compare the products obtained with int for N=13 and N=14.

HINTS:

With long, you can store factorial of up to 20.

3.4.3 HarmonicSum

Aim

Write a program called HarmonicSum to compute the sum of a harmonic series, as shown below, where $n=50000$. The program shall compute the sum from left-to-right as well as from the right-to-left. Are the two sums the same? Obtain the absolute difference between these two sums and explain the difference. Which sum is more accurate?

$$\text{Harmonic}(n) = 1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \dots + \frac{1}{n}$$

Program

```
/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

/**
 * Compute the sum of harmonics series from left-to-right and right-to-left.
 */
public class HarmonicSum {    // Save as "HarmonicSum.java"
    public static void main (String[] args) {

        // Define variables
        final int MAX_DENOMINATOR = 50000;
        double sumL2R = 0.0;
        double sumR2L = 0.0;
        double absDiff;

        // for-loop for summing from left-to-right
        for (int denominator = 1; denominator <= MAX_DENOMINATOR; ++
            denominator) {
            sumL2R += 1.0 / denominator;
        }
        System.out.println("The sum from left-to-right is: " + sumL2R);

        // for-loop for summing from right-to-left
        for (int denominator = MAX_DENOMINATOR; denominator >= 1 ; --
            denominator) {
            sumR2L += 1.0 / denominator;
        }

        System.out.println("The sum from right-to-left is: " + sumR2L);
        // Find the absolute difference and display
        if (sumL2R > sumR2L)
            absDiff = sumL2R-sumR2L;
        else
            absDiff = sumR2L-sumL2R;
        System.out.println("The absolute Difference = " + absDiff);

    }
}
```

Result

The sum from left-to-right is: 11.397003949278504
The sum from right-to-left is: 11.397003949278519
The absolute Difference = 1.4210854715202004E-14

Note:

- In this case, the sum from left to right and from right to left are not the same.
- The absolute difference represents the discrepancy between these two methods of summation.
- The sum from left to right is typically more accurate for large values of n because it starts with the smallest values and accumulates larger values later.

3.4.4 ComputePI

Aim

Write a program called ComputePI to compute the value of π , using the following series expansion. Use the maximum denominator (MAX_DENOMINATOR) as the terminating condition. Try MAX_DENOMINATOR of 1000, 10000, 100000, 1000000 and compare the PI obtained. Is this series suitable for computing PI? Why?

Program

```
/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */
public class ComputePI {
    public static void main (String[] args) {
        double sum = 0.0;
        int MAX_DENOMINATOR = 10000;    // Try 1000, 100000, 1000000
        int term = 1;
        for (int denominator = 1; denominator <= MAX_DENOMINATOR;
            denominator += 2) {
            // denominator = 1, 3, 5, 7, ..., MAX_DENOMINATOR
            sum = sum + (1.0/denominator * term);
            term = term*(-1);
        }
        sum = 4.0*sum;
        System.out.println("The PI value = " + sum);
    }
}
```

Result

The PI value = 3.141392653591791

TRY

1. Instead of using maximum denominator as the terminating condition, rewrite your program to use the maximum number of terms (MAX_TERM) as the terminating condition.

HINTS:

```
int MAX_TERM = 10000; // number of terms used in computation
int sum = 0.0;
for (int term = 1; term <= MAX_TERM; term++) {
    // term = 1, 2, 3, 4, ..., MAX_TERM
    if (term % 2 == 1) { // odd term number: add
        sum += 1.0 / (term * 2 - 1);
    } else { // even term number: subtract
        .....
    }
}
```

2. JDK maintains the value of π in a built-in double constant called Math.PI (=3.141592653589793). Add a statement to compare the values obtained and the Math.PI, in percents of Math.PI, i.e., (piComputed / Math.PI) * 100.

3.4.5 CozaLozaWoza

Aim

Write a program called CozaLozaWoza which prints the numbers 1 to 110, 11 numbers per line. The program shall print "Coza" in place of the numbers which are multiples of 3, "Loza" for multiples of 5, "Woza" for multiples of 7, "CozaLoza" for multiples of 3 and 5, and so on.

Program

```
/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

public class CozaLozaWoza { // Save as "CozaLozaWoza.java"
    public static void main(String[] args) {

        final int LOWERBOUND = 1, UPPERBOUND = 110;

        for (int number = LOWERBOUND; number <= UPPERBOUND; ++number) {

            // number = LOWERBOUND+1, LOWERBOUND+2, ..., UPPERBOUND
            // Print "Coza" if number is divisible by 3
            if ( number%3 == 0 ) {
                System.out.print("Coza");
            }

            // Print "Loza" if number is divisible by 5
            if ( number % 5 == 0 ) {
                System.out.print("Loza");
            }
        }
    }
}
```

```

        // Print "Woza" if number is divisible by 7
        if(number % 7 == 0){
            System.out.print("Woza");
        }

        // Print the number if it is not divisible by 3, 5 and 7
        // (i.e., it has not been processed above)
        if(number%3 != 0 && number%5 != 0 && number %7!=0){
            System.out.print(number);
        }

        // After processing the number, print a newline if number is
        // divisible by 11;
        // else print a space
        if ( number % 11 == 0){
            System.out.println(); // print newline
        } else {
            System.out.print(' '); // print a space
        }
    }
}

```

Result

```

1 2 Coza 4 Loza Coza Woza 8 Coza Loza 11
Coza 13 Woza CozaLoza 16 17 Coza 19 Loza CozaWoza 22
23 Coza Loza 26 Coza Woza 29 CozaLoza 31 32 Coza
34 LozaWoza Coza 37 38 Coza Loza 41 CozaWoza 43 44
CozaLoza 46 47 Coza Woza Loza Coza 52 53 Coza Loza
Woza Coza 58 59 CozaLoza 61 62 CozaWoza 64 Loza Coza
67 68 Coza LozaWoza 71 Coza 73 74 CozaLoza 76 Woza
Coza 79 Loza Coza 82 83 CozaWoza Loza 86 Coza 88
89 CozaLoza Woza 92 Coza 94 Loza Coza 97 Woza Coza
Loza 101 Coza 103 104 CozaLozaWoza 106 107 Coza 109 Loza

```

Notes

1. You cannot use nested-if (use if ... else if ... else if ... else) for this problem. It is because the tests are not mutually exclusive. For example, 15 is divisible by both 3 and 5. Nested-if is only applicable if the tests are mutually exclusive.
2. The tests above look messy. A better solution is to use a boolean flag to keep track of whether the number has been processed, as follows:

```

final int LOWERBOUND = 1, UPPERBOUND = 110;
boolean printed;
for (int number = LOWERBOUND; number <= UPPERBOUND; ++number) {
    printed = false; // init before processing each number
    // Print "Coza" if number is divisible by 3
    if ( ..... ) {
        System.out.print( ..... );
        printed = true; // processed!
    }
    // Print "Loza" if number is divisible by 5
    if ( ..... ) {

```

```

        System.out.print( ..... );
        printed = true;    // processed!
    }
    // Print "Woza" if number is divisible by 7
    .....
    // Print the number if it has not been processed
    if (!printed) {
        .....
    }
    // After processing the number, print a newline if it is divisible
    // by 11;
    // else, print a space
    .....
}

```

3.4.6 Fibonacci

Aim

Write a program called Fibonacci to print the first 20 Fibonacci numbers $F(n)$, where $F(n)=F(n-1)+F(n-2)$ and $F(1)=F(2)=1$. Also compute their average.

Program

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 *
 * Print first 20 Fibonacci numbers and their average
 */
public class Fibonacci {
    public static void main (String[] args) {
        int n = 3;           // The index n for F(n), starting from n=3, as
                             // n=1 and n=2 are pre-defined
        int fn;              // F(n) to be computed
        int fnMinus1 = 1;    // F(n-1), init to F(2)
        int fnMinus2 = 1;    // F(n-2), init to F(1)
        int nMax = 20;       // maximum n, inclusive
        int sum = fnMinus1 + fnMinus2; // Need sum to compute average
        double average;

        System.out.println("The first " + nMax + " Fibonacci numbers are:");
        ;
        System.out.print(fnMinus1 + " " + fnMinus2);

        while (n <= nMax) { // n starts from 3
            // n = 3, 4, 5, ..., nMax
            // Compute F(n), print it and add to sum
            fn = fnMinus1 + fnMinus2;
            sum = sum+fn;
            System.out.print(fn + " ");

            // Increment the index n and shift the numbers for the next
            // iteration
            ++n;
            fnMinus2 = fnMinus1;
            fnMinus1 = fn;
        }
    }
}

```



```

        // Compute and display the average (=sum/nMax).
        // Beware that int/int gives int.
        average = sum/nMax;
        System.out.println("\n"+ average);
    }
}

```

Result

The first 20 Fibonacci numbers are:

1 12 3 5 8 13 21 34 55 89 144 233 377 610 987 1597 2584 4181 6765
885.0

TRY

Tribonacci numbers are a sequence of numbers $T(n)$ similar to Fibonacci numbers, except that a number is formed by adding the three previous numbers, i.e., $T(n)=T(n-1)+T(n-2)+T(n-3)$, $T(1)=T(2)=1$, and $T(3)=2$. Write a program called Tribonacci to produce the first twenty Tribonacci numbers.

3.4.7 ExtractDigits

Aim

Write a program called ExtractDigits to extract each digit from an int, in the reverse order. For example, if the int is 15423, the output shall be "3 2 4 5 1", with a space separating the digits.

Program

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */
public class ExtractDigits {
    public static void main (String[] args) {
        int n = 15423;
        while (n > 0) {
            int digit = n % 10; // Extract the least-significant digit
            // Print this digit
            System.out.print(digit+" ");
            // Drop the least-significant digit and repeat the loop
            n = n / 10;
        }
    }
}

```

Result

3 2 4 5 1

3.5 Post-Lab Viva voce

1. What does the loop control variable do in a loop?
2. In a for loop, can you have multiple statements in the initialization, condition, and iteration sections?

3. How do you ensure a loop terminates when using a while or do-while loop?
4. When is a do-while loop useful in programming?

3.6 Further Probing Experiments

1. A palindrome is a word, phrase, number, or other sequence of characters that reads the same forwards and backwards, ignoring spaces, punctuation, and capitalization. Given a number, write a Java program to determine whether it is a palindrome according to the definition provided above.
2. A prime number is a positive integer greater than 1 that has no positive integer divisors other than 1 and itself. In other words, a prime number is only divisible by 1 and itself. Write a Java program to determine whether a given positive integer is a prime number or not.

EXERCISES ON INPUT, DECISION AND LOOP

4.1 Introduction

In this Java lab, students will explore Input, Decisions, and Loops. These are important concepts for making software. Through practical exercises, students will learn how to get information from users, make choices in programs, and repeat tasks efficiently. This lab is like a stepping stone, helping students build a strong foundation for writing interactive and useful computer programs. It's a great start for students' journey into software development.

4.2 Prelab Preparation

4.2.1 Objective

By the end of this lab, students should be able to:

1. Understand the basics of taking input from users and displaying output in Java programs.
2. Develop problem-solving skills through hands-on programming exercises that involve user interaction, decision making, and looping.

4.2.2 Background

In this lab on input data in Java, students will get on a foundational journey into the core concepts of acquiring and utilizing input within their programs. This lab lays the groundwork for understanding how Java programs can communicate with users. Key concepts to be explored include:

User Interaction: Students will learn how to facilitate communication between their Java programs and users. They'll discover methods for collecting input from users during program execution, enabling interactive applications.

Input Streams: The lab will introduce the concept of input streams, emphasizing the role they play in reading data sequentially from various sources, such as the keyboard. Students will grasp the significance of input streams in Java's input mechanisms.

Scanner Class: Students will become acquainted with the Scanner class, an essential tool for reading input data. They will learn how to utilize this class to efficiently gather input of various data types, such as integers, strings, and floating-point numbers.

System.in: The lab will explore System.in as the standard input stream, enabling students to read data directly from the keyboard. This foundational skill is crucial for building interactive programs.

Data Validation: Students will understand the importance of validating and handling input data. They will learn basic techniques to ensure that user-provided input meets the expected format and constraints.

Syntax for Inputting Data using Scanner: To input data in Java using the Scanner class, you need to follow these basic steps:

1. **Import the Scanner class:**

```
import java.util.Scanner;
```

2. **Create a Scanner object to read input:**

```
Scanner scanner = new Scanner(System.in);
```

3. **Use the Scanner methods to read data. Common methods include:**

next(): Reads a single word (a string without spaces).

nextInt(): Reads an integer.

nextDouble(): Reads a double-precision floating-point number.

nextLine(): Reads a whole line of text.

Don't forget to close the Scanner object when you're done to release system resources.

4.2.3 Hardware/Software Requirements

To successfully complete this lab, students will need access to the following:

1. A computer with a suitable text editor or Integrated Development Environment (IDE) installed, such as Eclipse, IntelliJ IDEA, or Visual Studio Code.
2. Java Development Kit (JDK) installed on the computer. Ensure that the JDK is correctly configured, and the system's PATH variable is set to include the JDK's bin directory.

4.2.4 Pre-Lab Questions

1. What is the primary purpose of input data handling in Java programming?
2. How do you create a Scanner object for reading user input?
3. Explain the significance of the System.in stream in Java.
4. What are the common data types that the Scanner class can read from user input?

4.3 Experiment Procedure

1. Ensure you have access to a Java development environment on your computer. Open your Java IDE or text editor for coding.
2. Create a new Java class with a suitable name in your IDE or text editor.
3. Write Java code to solve the problem, incorporating the following elements:
 - Input: Use the Scanner class to read input from the user or define input data within the program.

- Decision: Implement conditional statements (e.g., if, else if, switch) to make decisions based on the input or other conditions.
 - Loops: Utilize looping structures (e.g., for, while, do-while) to perform repetitive tasks or iterate through data.
4. Test your code with different input values to ensure it behaves as expected. Debug and fix any issues that arise during testing.
 5. Document the problem statement, the approach used to solve it, and the code you've written in your lab notebook or report.
 6. Experiment with variations of the problem, explore different scenarios, and modify your code to handle various cases.

4.4 Experiemnts

4.4.1 Add2Integer

Aim:

Write a program called Add2Integers that prompts user to enter two integers. The program shall read the two integers as int; compute their sum; and print the result.

Program

```
/**
 *author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 * @author cvrpa
 */

import java.util.Scanner;

public class Add2Integers {
    public static void main (String[] args) {
        // Declare variables
        int number1, number2, sum;

        // Put up prompting messages and read inputs as "int"
        Scanner in = new Scanner(System.in); // Scan the keyboard for
        input
        System.out.print("Enter first integer: "); // No newline for
        prompting message
        number1 = in.nextInt(); // Read next input as "
        int"

        System.out.print("Enter second integer: ");
        number2 = in.nextInt();
        in.close(); // Close Scanner

        // Compute sum
        sum = number1 + number2;

        // Display result
        System.out.println("The sum is: " + sum); // Print with newline
    }
}
```

Result

Enter first integer: 34
Enter second integer: 25
The sum is: 59

4.4.2 SumProductMinMax3

Aim: Write a program called SumProductMinMax3 that prompts user for three integers. The program shall read the inputs as int; compute the sum, product, minimum and maximum of the three integers; and print the results.

Program

```
/**
 *author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 * @author cvrpa
 */
import java.util.Scanner;

public class SumProductMinMax3 {
    public static void main(String args[]){
        // Declare variables
        int number1, number2, number3;
        int sum, product, min, max;

        Scanner in = new Scanner(System.in);

        System.out.print("Enter first integer: ");
        number1 = in.nextInt();

        System.out.print("Enter second integer: ");
        number2 = in.nextInt();

        System.out.print("Enter third integer: ");
        number3 = in.nextInt();
        in.close();

        // Compute sum and product
        sum = number1+number2+number3;
        product = number1*number2*number3;

        min = number1;
        if (number2 < min) {
            min = number2;
        }
        if (number3 < min) {
            min = number3;
        }

        // Compute max - similar to min
        max = number1;
        if(number2 > max){
            max = number2;
        }
    }
}
```

```

        if(number3 > max){
            max = number3;
        }

        // Print results
        System.out.println("The sum is: " + sum);
        System.out.println("The product is: " + product);
        System.out.println("The min is: " + min);
        System.out.println("The max is: " + max);
    }
}

```

Result

```

Enter first integer: 56
Enter second integer: 45
Enter third integer: 89
The sum is: 190
The product is: 224280
The min is: 45
The max is: 89

```

4.4.3 CircleComputation

Aim: Write a program called CircleComputation that prompts user for the radius of a circle in floating point number. The program shall read the input as double; compute the diameter, circumference, and area of the circle in double; and print the values rounded to 2 decimal places. Use System-provided constant Math.PI for pi.

Program

```

/**
 *author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 * @author cvrpa
 */
import java.util.Scanner;
public class CircleComputation {
    public static void main(String args[]){
        // Declare variables
        double radius, diameter, circumference, area;

        Scanner in = new Scanner(System.in);
        System.out.print("Enter the radius: ");
        radius = in.nextDouble(); // read input as double

        // Compute in "double"
        diameter = 2.0 * radius;
        area = Math.PI * radius * radius;
        circumference = 2.0 * Math.PI * radius;

        System.out.printf("Diameter is: %.2f\n", diameter);
        System.out.printf("Area is: %.2f\n", area);
        System.out.printf("Circumference is: %.2f\n", circumference);
    }
}

```

Result

Enter the radius: 5
Diameter is: 10.00
Area is: 78.54
Circumference is: 31.42

4.4.4 Swap2Integers

Aim: Write a program called Swap2Integers that prompts user for two integers. The program shall read the inputs as int, save in two variables called number1 and number2; swap the contents of the two variables; and print the results.

Program

```
/**
 *author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 * @author cvrpa
 */
import java.util.Scanner;

public class Swap2Integers {
    public static void main(String args[]){
        int number1, number2;
        Scanner in = new Scanner(System.in);  // Scan the keyboard

        System.out.print("Enter first integer: ");
        number1 = in.nextInt();

        System.out.print("Enter second integer: ");
        number2 = in.nextInt();

        int temp = number1;
        number1 = number2;
        number2 = temp;
        System.out.println("After the swap, first integer is: "+number1+ ",
            second integer is: "+number2);
    }
}
```

Result

Enter first integer: 56
Enter second integer: 34
After the swap, first integer is: 34, second integer is: 56

4.4.5 IncomeTaxCalculator

Aim: Write a program called IncomeTaxCalculator that reads the taxable income (in int).

For example, suppose that the taxable income is \$85000, the income tax payable is \$20000*0% + \$20000*10% + \$20000*20% + \$25000*30%.

The program shall calculate the income tax payable (in double); and print the result rounded to 2 decimal places.

Program

```
/**
 *author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 * @author cvrpa
 */
import java.util.Scanner;

public class IncomeTaxCalculator {
    public static void main(String args[]){
        // Declare constants first (variables may use these constants)
        // The keyword "final" marked these as constant (i.e., cannot be
        // changed).
        // Use uppercase words joined with underscore to name constants
        final double TAX_RATE_ABOVE_20K = 0.1;
        final double TAX_RATE_ABOVE_40K = 0.2;
        final double TAX_RATE_ABOVE_60K = 0.3;

        // Declare variables
        int taxableIncome;
        double taxPayable;

        Scanner in = new Scanner(System.in); // Scan the keyboard

        System.out.print("Enter the taxable income: $");
        taxableIncome = in.nextInt();

        // Compute tax payable in "double" using a nested-if to handle 4
        // cases
        if (taxableIncome <= 20000) { // [0, 20000]
            taxPayable = 0;
            System.out.println(taxPayable);
        } else if (taxableIncome <= 40000) { // [20001, 40000]
            taxPayable = (taxableIncome-20000) * TAX_RATE_ABOVE_20K;
        } else if (taxableIncome <= 60000) { // [40001, 60000]
            taxPayable = 20000*TAX_RATE_ABOVE_20K + (taxableIncome-40000)
                * TAX_RATE_ABOVE_40K;
        } else { // [60001, ]
            taxPayable = (20000*TAX_RATE_ABOVE_20K) + (20000 *
                TAX_RATE_ABOVE_40K) + (taxableIncome-60000) *
                TAX_RATE_ABOVE_60K;
        }
        // Print results rounded to 2 decimal places
        System.out.printf("The income tax payable is: $%.2f\n", taxPayable)
            ;
    }
}
```

Result

Enter the taxable income: \$41234
The income tax payable is: \$2246.80

Enter the taxable income: \$67891
The income tax payable is: \$8367.

Enter the taxable income: \$85432
The income tax payable is:\$13629.60

Enter the taxable income: \$12345
The income tax payable is: \$0.00

4.4.6 IncomeTaxCalculatorWithSentinel

Aim: Based on the previous exercise, write a program called IncomeTaxCalculatorWithSentinel which shall repeat the calculation until user enter -1.

Program

```
/**
 *author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 * @author cvrpa
 */
import java.util.Scanner;

public class IncomeTaxCalculatorWithSentinel {
    public static void main(String args[]){
        // Declare constants first (variables may use these constants)
        // The keyword "final" marked these as constant (i.e., cannot be
        // changed).
        // Use uppercase words joined with underscore to name constants
        final double TAX_RATE_ABOVE_20K = 0.1;
        final double TAX_RATE_ABOVE_40K = 0.2;
        final double TAX_RATE_ABOVE_60K = 0.3;
        final int SENTINEL = -1;    // Terminating value for input

        // Declare variables
        int taxableIncome;
        double taxPayable;

        Scanner in = new Scanner(System.in);    // Scan the keyboard

        System.out.print("Enter the taxable income (or -1 to end): $");
        taxableIncome = in.nextInt();

        while (taxableIncome != SENTINEL){
            // Compute tax payable in "double" using a nested-if to handle
            // 4 cases
            if (taxableIncome <= 20000) {          // [0, 20000]
                taxPayable = 0;
                System.out.println(taxPayable);
            } else if (taxableIncome <= 40000) {    // [20001, 40000]
                taxPayable = (taxableIncome-20000) * TAX_RATE_ABOVE_20K;
            } else if (taxableIncome <= 60000) {    // [40001, 60000]
                taxPayable = 20000*TAX_RATE_ABOVE_20K + (taxableIncome
                    -40000) * TAX_RATE_ABOVE_40K;
            }
        }
    }
}
```

```

    } else {
        // [60001, ]
        taxPayable = (20000*TAX_RATE_ABOVE_20K) + (20000 *
            TAX_RATE_ABOVE_40K) + (taxableIncome-60000) *
            TAX_RATE_ABOVE_60K;
    }
    // Print results rounded to 2 decimal places
    System.out.printf("The income tax payable is: $%.2f\n",
        taxPayable);

    System.out.print("Enter the taxable income (or -1 to end): $");
    taxableIncome = in.nextInt();
}
System.out.print("Bye. Have a good day!");
}
}

```

Result

Enter the taxable income (or -1 to end): \$41000
 The income tax payable is: \$2200.00

Enter the taxable income (or -1 to end): \$62000
 The income tax payable is: \$6600.00

Enter the taxable income (or -1 to end): \$73123
 The income tax payable is: \$9936.90

Enter the taxable income (or -1 to end): \$84328
 The income tax payable is: \$13298.40

Enter the taxable income: \$-1
 Bye. Have a good day!

4.4.7 PensionContributionCalculatorWithSentinel

Aim: Both the employer and the employee are mandated to contribute a certain percentage of the employees salary towards the employees pension fund. The rate is tabulated as follows:

Employees Age	Employee Rate (%)	Employer Rate (%)
55 and below	20	17
above 55 to 60	13	13
above 60 to 65	7.5	9
above 65	5	7.5

Program

```

/**
 *author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 * @author cvrpa
 */
import java.util.Scanner;

```

```

public class IncomeTaxCalculatorWithSentinel {
    public static void main(String args[]){
        // Declare constants first (variables may use these constants)
        // The keyword "final" marked these as constant (i.e., cannot be
        // changed).
        // Use uppercase words joined with underscore to name constants
        final double TAX_RATE_ABOVE_20K = 0.1;
        final double TAX_RATE_ABOVE_40K = 0.2;
        final double TAX_RATE_ABOVE_60K = 0.3;
        final int SENTINEL = -1;    // Terminating value for input

        // Declare variables
        int taxableIncome;
        double taxPayable;

        Scanner in = new Scanner(System.in);    // Scan the keyboard

        System.out.print("Enter the taxable income (or -1 to end): $");
        taxableIncome = in.nextInt();

        while (taxableIncome != SENTINEL){
            // Compute tax payable in "double" using a nested-if to handle
            // 4 cases
            if (taxableIncome <= 20000) {          // [0, 20000]
                taxPayable = 0;
                System.out.println(taxPayable);
            } else if (taxableIncome <= 40000) {    // [20001, 40000]
                taxPayable = (taxableIncome-20000) * TAX_RATE_ABOVE_20K;
            } else if (taxableIncome <= 60000) {    // [40001, 60000]
                taxPayable = 20000*TAX_RATE_ABOVE_20K + (taxableIncome
                    -40000) * TAX_RATE_ABOVE_40K;
            } else {                                // [60001, ]
                taxPayable = (20000*TAX_RATE_ABOVE_20K) + (20000 *
                    TAX_RATE_ABOVE_40K) + (taxableIncome-60000) *
                    TAX_RATE_ABOVE_60K;
            }
            // Print results rounded to 2 decimal places
            System.out.printf("The income tax payable is: %.2f%n",
                taxPayable);

            System.out.print("Enter the taxable income (or -1 to end): $");
            taxableIncome = in.nextInt();
        }
        System.out.print("Bye. Have a good day!");
    }
}

```

Result

```

Enter the monthly salary (or -1 to end): $5123
Enter the age: 21
The employee's contribution is: $1024.60
The employer's contribution is: $870.91
The total contribution is: $1895.51

```

```

Enter the monthly salary (or -1 to end): $5123

```

Enter the age: 64
The employee's contribution is: 384.22
The employer's contribution is :461.07
The total contribution is: \$845.30

Enter the monthly salary (or -1 to end): \$-1
Bye. Have a good day!

4.4.8 SalesTaxCalculator

Aim: A sales tax of 7tags should include the sales tax. For example, if an item has a price tag of \$107, the actual price is \$100 and \$7 goes to the sales tax.

Write a program using a loop to continuously input the tax-inclusive price (in double); compute the actual price and the sales tax (in double); and print the results rounded to 2 decimal places. The program shall terminate in response to input of -1; and print the total price, total actual price, and total sales tax

Program

```
/**
 *author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 * @author cvrpa
 */
import java.util.Scanner;

public class SalesTaxCalculator {
    public static void main(String args[]){
        // Declare constants
        final double SALES_TAX_RATE = 0.07;
        final int SENTINEL = -1;

        // Declare variables
        double price, actualPrice, salesTax; // inputs and results
        double totalPrice = 0.0, totalActualPrice = 0.0, totalSalesTax =
            0.0;

        Scanner in = new Scanner(System.in);
        // Read the first input to "seed" the while loop
        System.out.print("Enter the tax-inclusive price in dollars (or -1
            to end): ");
        price = in.nextDouble();

        while(price != SENTINEL) {

            actualPrice = price/(1+(SALES_TAX_RATE));
            salesTax = price - actualPrice;

            totalPrice += price;
            totalActualPrice += actualPrice;
            totalSalesTax += (int)salesTax;

            System.out.printf("Actual Price is: $%.2f ", actualPrice);
            System.out.printf("Sales Tax is: $%.2f\n", salesTax);
        }
    }
}
```

```

        System.out.print("Enter the tax-inclusive price in dollars (or
        -1 to end): ");
        price = in.nextDouble();
    }
    System.out.printf("Total Price is: $%.2f%n", totalPrice);
    System.out.printf("Total actual Price is: $%.2f%n",
        totalActualPrice);
    System.out.printf("Total Sales Tax is: $%.2f%n", totalSalesTax);
}
}

```

Result

```

Enter the tax-inclusive price in dollars (or -1 to end): 107
Actual Price is: $100.00 Sales Tax is: $7.00
Enter the tax-inclusive price in dollars (or -1 to end): 214
Actual Price is: $200.00 Sales Tax is: $14.00
Enter the tax-inclusive price in dollars (or -1 to end): 321
Actual Price is: $300.00 Sales Tax is: $21.00
Enter the tax-inclusive price in dollars (or -1 to end): -1
Total Price is: $642.00
Total actual Price is: $600.00
Total Sales Tax is: $42.00

```

4.4.9 ReverseInt

Aim: Write a program that prompts user for a positive integer. The program shall read the input as int; and print the "reverse" of the input integer.

Program

```

/**
 *author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 * @author cvrpa
 */
import java.util.Scanner;

public class ReverseInt {
    public static void main(String args[]){
        // Declare variables
        int inNumber;    // to be input
        int inDigit;     // each digit
        int rev = 0;

        Scanner in = new Scanner(System.in);
        System.out.print("Enter a positive integer: ");
        inNumber = in.nextInt();

        // Extract and drop the "last" digit repeatably using a while-loop
        // with modulus/divide operations
        while (inNumber > 0) {
            inDigit = inNumber % 10; // extract the "last" digit
            rev = rev*10+inDigit;
            inNumber /= 10;          // drop "last" digit and repeat
        }
    }
}

```

```

    }
    System.out.println("The reverse is: "+rev);
}
}

```

Result

Enter a positive integer: 12345
The reverse is: 54321

Enter a positive integer: 32564
The reverse is: 46523

4.4.10 SumOfDigitsInt

Aim: Write a program that prompts user for a positive integer. The program shall read the input as int; compute and print the sum of all its digits.

Program

```

/**
 *author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 * @author cvrpa
 */
import java.util.Scanner;

public class SumOfDigitsInt {
    public static void main(String args[]){
        // Declare variables
        int inNumber;    // to be input
        int inDigit;     // each digit
        int sum = 0;

        Scanner in = new Scanner(System.in);
        System.out.print("Enter a positive integer: ");
        inNumber = in.nextInt();

        // Extract and drop the "last" digit repeatably using a while-loop
        // with modulus/divide operations
        while (inNumber > 0) {
            inDigit = inNumber % 10; // extract the "last" digit
            sum +=inDigit;
            inNumber /= 10;          // drop "last" digit and repeat
        }
        System.out.println("The sum of all digits is: "+sum);
    }
}

```

Result

Enter a positive integer: 12345
The sum of all digits is: 15

Enter a positive integer: 34261
The sum of all digits is: 16

4.4.11 InputValidation

Aim: Your program often needs to validate the user's inputs, e.g., marks shall be between 0 and 100.

Write a program that prompts user for an integer between 0-10 or 90-100. The program shall read the input as int; and repeat until the user enters a valid input.

Program

```
/**
 *author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 * @author cvrpa
 */
import java.util.Scanner;

public class InputValidation {
    public static void main(String args[]){
        // Declare variables
        int numberIn;        // to be input
        boolean isValid;     // boolean flag to control the loop

        Scanner in = new Scanner(System.in);

        // Use a do-while loop controlled by a boolean flag
        // to repeatably read the input until a valid input is entered
        isValid = false;     // default assuming input is not valid

        do{
            // Prompt and read input
            System.out.print("Enter a number between 0-10 or 90-100: ");
            numberIn = in.nextInt();
            // Validate input by setting the boolean flag accordingly
            if((numberIn>=0 && numberIn<=10) || (numberIn>=90 && numberIn
                <=100)) {
                isValid = true;    // exit the loop
                break;
            } else {
                System.out.println("Invalid input, try again..."); //
                Print error message and repeat
            }
        }while (!isValid);
        System.out.println("You have entered: "+numberIn);
    }
}
```

Result

```
Enter a number between 0-10 or 90-100: -5
Invalid input, try again...
Enter a number between 0-10 or 90-100: 45
Invalid input, try again...
Enter a number between 0-10 or 90-100: 108
Invalid input, try again...
Enter a number between 0-10 or 90-100: 99
You have entered: 99
```


4.4.12 AverageWithInputValidation

Aim: Write a program that prompts user for the mark (between 0-100 in int) of 3 students; computes the average (in double); and prints the result rounded to 2 decimal places. Your program needs to perform input validation.

Program

```
/**
 *author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 * @author cvrpa
 */
import java.util.Scanner;

public class AverageWithInputValidation {
    public static void main(String args[]){
        // Declare variables
        int numberIn;          // to be input
        double average=0.0;
        int counter=0, sum =0;

        Scanner in = new Scanner(System.in);

        do {
            // Prompt and read input
            System.out.print("Enter a number between 0-100: ");
            numberIn = in.nextInt();
            // Validate input by setting the boolean flag accordingly
            if((numberIn>=0 && numberIn<=100)) {
                sum += numberIn;
                counter++;
            } else {
                System.out.println("Invalid input, try again..."); //
                Print error message and repeat
            }
        } while (counter!=3);
        average = sum/3;
        System.out.printf("The average is: %.2f",average);
    }
}
```

Result

```
Enter a number between 0-100: -1
Invalid input, try again...
Enter a number between 0-100: 45
Enter a number between 0-100: 106
Invalid input, try again...
Enter a number between 0-100: 92
Enter a number between 0-100: 23
The average is: 53.00
```

4.5 Post-Lab Viva Voce Questions

1. Describe the role of the Scanner class in reading input data. How does it simplify the process of data input?
2. How would you read a whole line of text from a user using the Scanner class?
3. What does the break statement do in a loop, and when might you use it?
4. How can you prevent an infinite loop when using a while or do-while loop in Java?

4.6 Further Probing Experiments

1. Develop a Java program called SphereComputation that prompts user for the radius of a sphere in floating point number. The program shall read the input as double; compute the volume and surface area of the sphere in double; and print the values rounded to 2 decimal places.
2. Write a program called CylinderComputation that prompts user for the base radius and height of a cylinder in floating point number. The program shall read the inputs as double; compute the base area, surface area, and volume of the cylinder; and print the values rounded to 2 decimal places.

EXERCISES ON NESTED-LOOPS

5.1 Introduction

In this lab, we will explore the concept of nested loops, a flexible programming technique that allows us to solve more complicated problems. Just like a set of building blocks, nested loops enable us to construct complex patterns and structures within our programs. Our goal is to understand how to use nested loops effectively, mastering a skill that will empower us to tackle a wide range of challenges in Java programming. By the end of this lab, you will have a deeper understanding of how to apply nested loops to create exclusive solutions, enhancing your programming toolkit significantly. So, let's dive in and unlock the full potential of nested loops in Java!

5.2 Prelab Preparation

5.2.1 Objective

By the end of this lab, students should be able to:

1. Understand the syntax of nested loops and using them effectively to solve problems that involve multi-layered iterations.
2. learn strategies for optimizing the efficiency of the nested loop structures, ensuring that the programs run smoothly and perform well, even in scenarios involving extensive iterations.

5.2.2 Background

It is also possible to place a loop inside another loop. This is called a nested loop. The "inner loop" will be executed one time for each iteration of the "outer loop".

Syntax for Nested for loop:

```
for ( initialization; condition; increment ) {  
    for ( initialization; condition; increment ) {  
        // statement of inside loop  
    }  
    // statement of outer loop  
}
```

Syntax for Nested while loop:

```
while(condition) {  
    while(condition) {  
        // statement of inside loop  
    }  
    // statement of outer loop}
```

Syntax for Nested do-while loop:

```
do{
    do{
        // statement of inside loop
    }while(condition);
    // statement of outer loop
}while(condition);
```

Note: There is no rule that a loop must be nested inside its own type. In fact, there can be any type of loop nested inside any type and to any level.

Syntax:

```
do{
    while(condition) {
        for ( initialization; condition; increment ) {
            // statement of inside for loop
        }
        // statement of inside while loop
    }
    // statement of outer do-while loop
}while(condition);
```

5.2.3 Hardware/Software Requirements

To successfully complete this lab, students will need access to the following:

1. A computer with a suitable text editor or Integrated Development Environment (IDE) installed, such as Eclipse, IntelliJ IDEA, or Visual Studio Code.
2. Java Development Kit (JDK) installed on the computer. Ensure that the JDK is correctly configured, and the system's PATH variable is set to include the JDK's bin directory.

5.2.4 Pre-Lab Questions

1. What is a nested loop in Java, and how does it differ from a regular loop?
2. Explain the syntax of a nested "for" loop in Java with an example.
3. How are nested "while" loops implemented in Java, and why might you use them?
4. How many loops can be nested?

5.3 Experiment Procedure

1. Ensure you have access to a Java development environment on your computer. Open your Java IDE or text editor for coding.
2. Create a new Java class with a suitable name in your IDE or text editor.
3. Write Java code to solve the problem, incorporating the following elements:
 - Input: Use the Scanner class to read input from the user or define input data within the program.

- Decision: Implement conditional statements (e.g., if, else if, switch) to make decisions based on the input or other conditions.
 - Loops: Utilize looping structures (e.g., for, while, do-while) to perform repetitive tasks or iterate through data.
4. Test your code with different input values to ensure it behaves as expected. Debug and fix any issues that arise during testing.
 5. Document the problem statement, the approach used to solve it, and the code you've written in your lab notebook or report.
 6. Experiment with variations of the problem, explore different scenarios, and modify your code to handle various cases.

5.4 Experiemnts

5.4.1 SquarePattern (nested-loop)

Aim: Write a program called SquarePattern that prompts user for the size (a non-negative integer in int); and prints the following square pattern using two nested for-loops.

Program

```
/**
 *author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 * @author cvrpa
 */
import java.util.Scanner;

public class SquarePattern {
    public static void main(String args[]){
        int size;
        Scanner in = new Scanner(System.in);
        System.out.print("Enter the size: ");
        size = in.nextInt();
        for (int row = 1; row <= size; row++) {
            for (int col = 1; col <= size; col++) {
                System.out.print("# ");
            }
            System.out.println();
        }
    }
}
```

Result

```
Enter the size: 5
# # # # #
# # # # #
# # # # #
# # # # #
# # # # #
```

5.4.2 SquarePattern (nested-loop)

Aim: Write a program called CheckerPattern that prompts user for the size (a non-negative integer in int); and prints the following checkerboard pattern.

Program

```
/**
 *author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 * @author cvrpa
 */

import java.util.Scanner;

public class CheckerPattern {
    public static void main(String args[]){
        int size;
        Scanner in = new Scanner(System.in);
        System.out.print("Enter the size: ");
        size = in.nextInt();
        for (int row = 1; row <= size; row++) {
            for (int col = 1; col <= size; col++) {
                if ((row % 2) == 0) {    // row 2, 4, 6, ...
                    System.out.print(" #");
                }
                else{
                    System.out.print("# ");
                }
            }
            System.out.println();
        }
    }
}
```

Result

```
Enter the size: 8
# # # # # # # #
 # # # # # # # #
# # # # # # # #
 # # # # # # # #
# # # # # # # #
 # # # # # # # #
# # # # # # # #
 # # # # # # # #
# # # # # # # #
 # # # # # # # #
```

5.4.3 MultiplicationTable (nested-loop)

Aim: Write a program called MultiplicationTable that prompts user for the size (a positive integer in int); and prints the multiplication table

Program

```
/**
 *author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
```

```

* @author cvrpa
*/

import java.util.Scanner;

public class MultiplicationTable {
    public static void main(String args[]){
        int size;
        Scanner in = new Scanner(System.in);
        System.out.print("Enter the size: ");
        size = in.nextInt();

        System.out.printf("    * |");
        for (int row = 1; row <= size; row++) {
            System.out.printf("%4d", row);
        }

        System.out.println();
        for (int row = 0; row <= size; row++) {
            System.out.print("----");
        }
        System.out.print("--");
        System.out.println();

        for (int row = 1; row <= size; row++) {
            System.out.printf("%4d |", row);
            for (int col = 1; col <= size; col++) {
                System.out.printf("%4d", row*col);
            }
            System.out.println();
        }
    }
}

```

Result

Enter the size: 10

*	1	2	3	4	5	6	7	8	9	10
1	1	2	3	4	5	6	7	8	9	10
2	2	4	6	8	10	12	14	16	18	20
3	3	6	9	12	15	18	21	24	27	30
4	4	8	12	16	20	24	28	32	36	40
5	5	10	15	20	25	30	35	40	45	50
6	6	12	18	24	30	36	42	48	54	60
7	7	14	21	28	35	42	49	56	63	70
8	8	16	24	32	40	48	56	64	72	80
9	9	18	27	36	45	54	63	72	81	90
10	10	20	30	40	50	60	70	80	90	100

5.4.4 TriangularPattern (nested-loop)

Aim: Write a program called TriangularPatternX that prompts user for the size (a non-negative integer in int); and prints triangular patterns (Refer Syllabus)

Program

```

/**
*author Dr.C V Rama Padmaja

```

```

* Associate Professor, Dept CSE
* IARE, Dundigal.
* @author cvrpa
*/

import java.util.Scanner;

public class TriangularPattern {
    public static void main(String args[]){
        int size;
        Scanner in = new Scanner(System.in);
        System.out.print("Enter the size: ");
        size = in.nextInt();

        // For displaying (a)
        for (int row = 1; row <= size; row++) { // row = 1, 2, 3, ...,
            size
            // Inner loop to print each of the columns of a particular row
            for (int col = 1; col <= size; col++) { // col = 1, 2, 3, ...,
                size
                if (row >= col) {
                    System.out.print("# ");
                } else {
                    System.out.print(" "); // Need to print the "leading"
                    blanks
                }
            }
            // Print a newline after printing all the columns
            System.out.println();
        }

        System.out.println("\n\n");
        // For displaying (b)
        for (int row = 1; row <= size; row++) { // row = 1, 2, 3, ...,
            size
            // Inner loop to print each of the columns of a particular row
            for (int col = 1; col <= size; col++) { // col = 1, 2, 3, ...,
                size
                if (row + col <= size + 1) {
                    System.out.print("# ");
                } else {
                    System.out.print(" "); // Need to print the "leading"
                    blanks
                }
            }
            // Print a newline after printing all the columns
            System.out.println();
        }

        System.out.println("\n\n");
        // For displaying (c)
        for (int row = 1; row <= size; row++) { // row = 1, 2, 3, ...,
            size
            // Inner loop to print each of the columns of a particular row
            for (int col = 1; col <= size; col++) { // col = 1, 2, 3, ...,
                size
                if (row <= col) {

```



```

        System.out.print("# ");
    } else {
        System.out.print("  "); // Need to print the "leading"
        blanks
    }
}
// Print a newline after printing all the columns
System.out.println();
}

System.out.println("\n\n");
// For displaying (d)
for (int row = 1; row <= size; row++) { // row = 1, 2, 3, ...,
    size
    // Inner loop to print each of the columns of a particular row
    for (int col = 1; col <= size; col++) { // col = 1, 2, 3, ...,
        size
        if (row + col >= size + 1) {
            System.out.print("# ");
        } else {
            System.out.print("  "); // Need to print the "leading"
            blanks
        }
    }
    // Print a newline after printing all the columns
    System.out.println();
}
}
}

```

Result

Enter the size: 5

```

#
# #
# # #
# # # #
# # # # #

```

```

# # # # #
# # # #
# # #
# #
#

```

```

# # # # #
# # # #
# # #
# #
#

```

```

#
# #
# # #
# # # #
# # # # #

```

5.4.5 BoxPattern (nested-loop)

Aim: Write a program called BoxPatternX that prompts user for the size (a nonnegative integer in int) and print the box patterns

Program

```
/**
 *author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 * @author cvrpa
 */

import java.util.Scanner;

public class BoxPatternX {
    public static void main(String args[]){
        int size;
        Scanner in = new Scanner(System.in);
        System.out.print("Enter the size: ");
        size = in.nextInt();

        // For displaying (a)
        for (int row = 1; row <= size; row++) { // row = 1, 2, 3, ...,
            size
            // Inner loop to print each of the columns of a particular row
            for (int col = 1; col <= size; col++) { // col = 1, 2, 3, ...,
                size
                if (row == 1 || row == size || col == 1 || col == size) {
                    System.out.print("# ");
                } else {
                    System.out.print(" "); // Need to print the "leading"
                    blanks
                }
            }
            // Print a newline after printing all the columns
            System.out.println();
        }

        System.out.println("\n");
        // For displaying (b)
        for (int row = 1; row <= size; row++) { // row = 1, 2, 3, ...,
            size
            // Inner loop to print each of the columns of a particular row
            for (int col = 1; col <= size; col++) { // col = 1, 2, 3, ...,
                size
                if (row == 1 || row == size || row == col) {
                    System.out.print("# ");
                } else {
                    System.out.print(" "); // Need to print the "leading"
                    blanks
                }
            }
            // Print a newline after printing all the columns
            System.out.println();
        }

        System.out.println("\n");
    }
}
```

```

// For displaying (c)
for (int row = 1; row <= size; row++) { // row = 1, 2, 3, ...,
    size
    // Inner loop to print each of the columns of a particular row
    for (int col = 1; col <= size; col++) { // col = 1, 2, 3, ...,
        size
        if (row == 1 || row == (size-col+1) || row == size) {
            System.out.print("# ");
        } else {
            System.out.print(" "); // Need to print the "leading"
                                   blanks
        }
    }
    // Print a newline after printing all the columns
    System.out.println();
}

System.out.println("\n");
// For displaying (d)
for (int row = 1; row <= size; row++) { // row = 1, 2, 3, ...,
    size
    // Inner loop to print each of the columns of a particular row
    for (int col = 1; col <= size; col++) { // col = 1, 2, 3, ...,
        size
        if (row == 1 || row == size || row == (size-col+1) || row
            == col) {
            System.out.print("# ");
        } else {
            System.out.print(" "); // Need to print the "leading"
                                   blanks
        }
    }
    // Print a newline after printing all the columns
    System.out.println();
}

System.out.println("\n");
// For displaying (e)
for (int row = 1; row <= size; row++) { // row = 1, 2, 3, ...,
    size
    // Inner loop to print each of the columns of a particular row
    for (int col = 1; col <= size; col++) { // col = 1, 2, 3, ...,
        size
        if (row == 1 || row == size || row == (size-col+1) || row
            == col || col == 1 || col == size) {
            System.out.print("# ");
        } else {
            System.out.print(" "); // Need to print the "leading"
                                   blanks
        }
    }
    // Print a newline after printing all the columns
    System.out.println();
}
}
}

```

Result

Enter the size: 8

```
# # # # # # # #
#               #
#               #
#               #
#               #
#               #
#               #
# # # # # # # #
```

```
# # # # # # # #
#
#
#
#
#
#
# # # # # # # #
```

```
# # # # # # # #
#
#
#
#
#
#
# # # # # # # #
```

```
# # # # # # # #
#
#
#
#
#
#
# # # # # # # #
```

```
# # # # # # # #
# #           # #
#  #         #  #
#   #       #   #
#    #     #    #
#   #       #   #
#  #         #  #
# #           # #
# # # # # # # #
```

5.4.6 HillPattern (nested-loop)

Aim: Write 3 programs called HillPatternX (X = A, B, C, D) that prompts user for the size (a nonnegative integer in int); and prints the pattern

Program

```

/**
 *author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 * @author cvrpa
 */

import java.util.Scanner;

public class HillPatternX {
    public static void main(String args[]){
        int size;
        Scanner in = new Scanner(System.in);
        System.out.print("Enter the size: ");
        size = in.nextInt();

        // For displaying (a)
        System.out.println("HillPattern (a)");
        for (int row = 1; row <= size; row++) { // row = 1, 2, 3, ...,
            size
            // Inner loop to print each of the columns of a particular row
            for(int col = 1; col <= size-row; col++) {
                System.out.print(" ");
            }
            for(int col = 1; col <= 2*row-1; col++){
                System.out.print("# ");
            }
            // Print a newline after printing all the columns
            System.out.println();
        }
        System.out.println();

        // For displaying (b)
        System.out.println("HillPattern (b)");
        for (int row = 1; row <= size; row++) { // row = 1, 2, 3, ...,
            size
            // Inner loop to print each of the columns of a particular row
            for(int col = 1; col <= row-1; col++) {
                System.out.print(" ");
            }
            for(int col = 1; col <= 2*(size-row)+1; col++){
                System.out.print("# ");
            }
            // Print a newline after printing all the columns
            System.out.println();
        }
        System.out.println("\n");

        // For displaying (c)
        System.out.println("HillPattern (c)");
        for (int row = 1; row <= size; row++) { // row = 1, 2, 3, ...,
            size
            // Inner loop to print each of the columns of a particular row
            for(int col = 1; col <= size-row; col++) {
                System.out.print(" ");
            }
            for(int col = 1; col <= 2*row-1; col++){

```

```

        System.out.print("# ");
    }
    // Print a newline after printing all the columns
    System.out.println();
}
for (int row = 2; row <= size; row++) { // row = 1, 2, 3, ...,
    size
    // Inner loop to print each of the columns of a particular row
    for(int col = 1; col <= row-1; col++) {
        System.out.print(" ");
    }
    for(int col = 1; col <= 2*(size-row)+1; col++){
        System.out.print("# ");
    }
    // Print a newline after printing all the columns
    System.out.println();
}
System.out.println("\n");
}
}

```

Result

Enter the size: 6

HillPattern (a)

```

      #
     # # #
    # # # # #
   # # # # # # #
  # # # # # # # # #
 # # # # # # # # # #
# # # # # # # # # #

```

HillPattern (b)

```

# # # # # # # # # # #
 # # # # # # # # #
  # # # # # # #
   # # # # #
    # # #
     # # #
      #

```

HillPattern (c)

```

      #
     # # #
    # # # # #
   # # # # # # #
  # # # # # # # # #
 # # # # # # # # # #
# # # # # # # # # # #
 # # # # # # # # #
  # # # # # # #
   # # # # #
    # # #
     # #
      #

```

5.4.7 NumberPattern (nested-loop)

Aim: Write a program called NumberPatternX that prompts user for the size (a nonnegative integer in int); and prints the NumberPattern

Program

```
/**
 *author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 * @author cvrpa
 */

import java.util.Scanner;

public class NumberPattern {
    public static void main(String args[]){
        int size;
        Scanner in = new Scanner(System.in);
        System.out.print("Enter the size: ");
        size = in.nextInt();

        // For displaying (a)
        for (int row = 1; row <= size; row++) { // row = 1, 2, 3, ...,
            size
            // Inner loop to print each of the columns of a particular row
            for (int col = 1; col <= size; col++) { // col = 1, 2, 3, ...,
                size
                if (row >= col) {
                    System.out.print(col + " ");
                } else {
                    System.out.print(" "); // Need to print the "leading"
                    blanks
                }
            }
            // Print a newline after printing all the columns
            System.out.println();
        }

        System.out.println("\n\n");
        // For displaying (b)
        for (int row = 1; row <= size; row++) { // row = 1, 2, 3, ...,
            size
            // Inner loop to print each of the columns of a particular row
            for (int col = 1; col <= size; col++) { // col = 1, 2, 3, ...,
                size
                if (row + col <= size + 1) {
                    System.out.print(col + " ");
                } else {
                    System.out.print(" "); // Need to print the "leading"
                    blanks
                }
            }
            // Print a newline after printing all the columns
            System.out.println();
        }

        System.out.println("\n\n");
        // For displaying (c)
        for (int row = 1; row <= size; row++) { // row = 1, 2, 3, ...,
            size
            // Inner loop to print each of the columns of a particular row
```

```

        for (int col = 1; col <= size; col++) { // col = 1, 2, 3, ...,
            size
            if (row <= col) {
                System.out.print(col + " ");
            } else {
                System.out.print(" "); // Need to print the "leading"
                blanks
            }
        }
        // Print a newline after printing all the columns
        System.out.println();
    }

    System.out.println("\n\n");
    // For displaying (d)
    for (int row = 1; row <= size; row++) { // row = 1, 2, 3, ...,
        size
        // Inner loop to print each of the columns of a particular row
        for (int col = 1; col <= size; col++) { // col = 1, 2, 3, ...,
            size
            if (row + col >= size + 1) {
                System.out.print(col + " ");
            } else {
                System.out.print(" "); // Need to print the "leading"
                blanks
            }
        }
        // Print a newline after printing all the columns
        System.out.println();
    }
}
}

```

Result

Enter the size: 8

```

1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
1 2 3 4 5 6
1 2 3 4 5 6 7
1 2 3 4 5 6 7 8

```

```

1 2 3 4 5 6 7 8
1 2 3 4 5 6 7
1 2 3 4 5 6
1 2 3 4 5
1 2 3 4
1 2 3
1 2
1

```



```

1 2 3 4 5 6 7 8
  2 3 4 5 6 7 8
    3 4 5 6 7 8
      4 5 6 7 8
        5 6 7 8
          6 7 8
            7 8
              8

              8
            7 8
          6 7 8
        5 6 7 8
      4 5 6 7 8
    3 4 5 6 7 8
  2 3 4 5 6 7 8
1 2 3 4 5 6 7 8

```

5.5 Sample Viva Voce Questions

1. Explain the concept of a "loop within a loop" and why it is useful.
2. What precautions should you take to prevent infinite loops when using nested loops?
3. Provide an example of a practical application where nested loops are commonly used in Java programming.
4. What is the output of a simple nested loop program that prints a pattern of stars or numbers?

5.6 Further Probing Experiments

1. Write a program called HillPattern that prompts user for the size (a non-negative integer in int) and prints the pattern as shown:

```

# # # # # # # # # # #
# # # # #   # # # # #
# # # #   # # # # #
# # #   # # # # #
# #   # # # # #
#   # # # # #
# #   # # # # #
# # #   # # # # #
# # # #   # # # # #
# # # # # # # # # # #

```


MAGIC(SPECIAL) NUMBERS

6.1 Introduction

In this session, we will explore a unique category of numbers, such as Armstrong numbers, neon numbers, spy numbers, palindrome numbers, and more. These special numbers possess remarkable properties that make them stand out in the world of mathematics and programming. As we unravel the secrets behind these magical digits, you will gain valuable insights into the power of logic and algorithms in programming, enabling you to apply your newfound knowledge to solve complex problems and create efficient code.

6.2 Prelab Preparation

6.2.1 Objective

By the end of this lab, students should be able to:

1. Understand the unique properties and characteristics of special numbers, gaining a deep understanding of their mathematical significance and relevance in programming.
2. Learn how to identify and fix common programming errors that occur when working with special numbers.

6.2.2 Background

Number Manipulation is a fundamental concept in computer programming that involves the alteration and manipulation of numerical values to achieve specific outcomes or solve computational problems. It plays a crucial role in various programming tasks, from basic arithmetic operations to more complex mathematical algorithms. Here are some key aspects of number manipulation in programming

Arithmetic Operations:

Addition (+), Subtraction (-), Multiplication (*), Division (/), and Modulo (%): These fundamental arithmetic operations are used to manipulate numbers for various purposes, such as calculations, comparisons, and transformations.

Rounding and Truncation

Mathematical Functions: Programming languages provide functions for rounding numbers (e.g., `Math.round()`) or truncating decimals (e.g., `Math.floor()` and `Math.ceil()`). These functions are essential for ensuring precision and formatting in calculations.

Type Conversion Casting: Converting a number from one data type to another is known as casting. For instance, casting a floating-point number to an integer type involves removing the decimal portion. Explicit casting helps handle different data types during operations.

Random Number Generation

Pseudorandom Number Generators: Programming languages offer methods to generate random numbers within a specified range. This is useful for simulations, games, and cryptographic applications.

Number Decomposition

Extracting Digits: For tasks like checking special numbers (e.g., Armstrong, spy, and palindromic numbers), numbers can be decomposed into their individual digits, often using modulo and division operations.

Number Reversal

Reversing Digits: Reversing the order of digits in a number is important for tasks like checking palindromic numbers. This can be achieved through iterative operations.

Bitwise Operations (for binary numbers)

Bitwise AND (&), OR (|), XOR (^), Shifts (< and >), and Complement (~): Bit manipulation is common when working with binary numbers or performing low-level operations. Bitwise operations are used in various scenarios, including optimizing code or solving specific problems related to data encoding.

Numerical Comparisons

Relational Operators (e.g., <, >, <=, >=, ==, and !=): Comparing numbers to determine relationships or validate conditions is a fundamental aspect of programming. These comparisons help make decisions based on numerical values.

Precision and Rounding Errors

Floating-Point Arithmetic Considerations: Due to limited precision in floating-point numbers, handling rounding errors and ensuring accuracy in calculations are vital in numerical programming.

Mathematical Functions and Libraries

Math Libraries: Many programming languages provide extensive math libraries that offer a wide range of mathematical functions for more advanced number manipulation, including trigonometric, exponential, and logarithmic functions.

Number manipulation is a versatile skill that programmers rely on when working with numerical data in a wide range of applications, from scientific simulations to financial calculations and game development. It forms the foundation of many algorithms and data processing tasks, making it an essential aspect of programming expertise.

To successfully complete this lab, students will need access to the following:

1. A computer with a suitable text editor or Integrated Development Environment (IDE) installed, such as Eclipse, IntelliJ IDEA, or Visual Studio Code.
2. Java Development Kit (JDK) installed on the computer. Ensure that the JDK is correctly configured, and the system's PATH variable is set to include the JDK's bin directory.

6.2.3 Hardware/Software Requirements

To successfully complete this lab, students will need access to the following:

1. A computer with a suitable text editor or Integrated Development Environment (IDE) installed, such as Eclipse, IntelliJ IDEA, or Visual Studio Code.

2. Java Development Kit (JDK) installed on the computer. Ensure that the JDK is correctly configured, and the system's PATH variable is set to include the JDK's bin directory.

6.2.4 Pre-Lab Questions

1. When would you use casting to manipulate numbers in programming?
2. How can you round a floating-point number to the nearest integer in code?
3. What does it mean to reverse the digits of a number, and why is it useful?
4. Explain the purpose of a random number generator in programming.

6.3 Experiment Procedure

1. Ensure you have access to a Java development environment on your computer. Open your Java IDE or text editor for coding.
2. Create a new Java class with a suitable name in your IDE or text editor.
3. Write Java code to solve the problem
4. Test your code with different input values to ensure it behaves as expected. Debug and fix any issues that arise during testing.
5. Document the problem statement, the approach used to solve it, and the code you've written in your lab notebook or report.
6. Experiment with variations of the problem, explore different scenarios, and modify your code to handle various cases.

6.4 Experiemnts

6.4.1 AmicableNumbers

Aim: Two different numbers are said to be so amicable numbers if each sum of divisors is equal to the other number. Amicable Numbers are: (220, 284), (1184, 1210), (2620, 2924), (5020, 5564), (6232, 6368).

Program

```
/**
 * author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 * @author cvrpa
 */

import java.util.Scanner;

public class AmicableNumbers {
    public static void main(String[] args) {

        Scanner in = new Scanner(System.in);

        System.out.print("Enter first number: ");
        int number1 = in.nextInt();
```

```

        System.out.print("Enter second number: ");
        int number2 = in.nextInt();

        int sum1 = 1;

        for (int i = 2; i <= number1/2; i++) {
            if (number1 % i == 0) {
                sum1 = sum1 + i;
            }
        }

        int sum2 = 1;
        for (int i = 2; i <= number2/2; i++) {
            if (number2 % i == 0) {
                sum2 = sum2 + i;
            }
        }

        if((sum1 == number2) && (sum2 == number1)){
            System.out.println(number1 + " and " + number2 + " are Amicable
                Numbers");
        }
        else{
            System.out.println(number1 + " and " + number2 + " are not
                Amicable Numbers");
        }
        in.close();
    }
}

```

Result Enter first number: 220
Enter second number: 284
220 and 284 are Amicable Numbers

6.4.2 ArmstrongNumber

Aim: Armstrong number is a positive number if it is equal to the sum of cubes of its digits is called Armstrong number and if its sum is not equal to the number then it's not a Armstrong number.

Program

```

/**
 * author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 * @author cvrpa
 */

import java.util.Scanner;

public class ArmstrongNumber {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.print("Enter a number: ");

        int number = in.nextInt();
    }
}

```

```

        int originalNumber = number;
        int result = 0;

        int numberOfDigits = (int) Math.log10(number) + 1;

        while (number > 0) {
            int digit = number % 10;
            result += Math.pow(digit, numberOfDigits);
            number /= 10;
        }

        if (result == originalNumber) {
            System.out.println(originalNumber + " is an Armstrong number.");
        } else {
            System.out.println(originalNumber + " is not an Armstrong
            number.");
        }
        in.close();
    }
}

```

Result

Enter a number: 153

153 is an Armstrong number.

Enter a number: 150

150 is not an Armstrong number.

6.4.3 CapricornNumber

Aim: A number is called Capricorn or Kaprekar number whose square is divided into two parts in any conditions and parts are added, the additions of parts is equal to the number, is called Capricorn or Kaprekar number.

Program

```

/**
 * author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 * @author cvrpa
 */

import java.util.Scanner;

public class CapricornNumber {
    public static void main(String[] args)
    {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int n = scanner.nextInt();
        boolean isCapricorn = false;

        int square = n * n;
        int temp = square;
        int contDigits = 0;
    }
}

```

```

while (temp > 0)
{
    contDigits++;
    temp /= 10;
}

for (int i = 1; i < contDigits; i++)
{
    int divisor = (int) Math.pow(10, i);
    int quotient = square / divisor;
    int remainder = square % divisor;
    if (quotient + remainder == n)
    {
        isCapricorn = true;
    }
}
if (isCapricorn)
{
    System.out.println(n + " is a Capricorn/Kaprekar number");
} else
{
    System.out.println(n + "is not a Capricorn/Kaprekar number");
}
}
}

```

Result

Enter a number: 45

45 is a Capricorn/Kaprekar number

Enter a number: 297

297 is a Capricorn/Kaprekar number

6.4.4 CircularPrime

Aim: A circular prime is a prime number with the property that the number generated at each intermediate step when cyclically permuting its digits will be prime. For example, 1193 is a circular prime, since 1931, 9311 and 3119 all are also prime.

Program

```

/**
 * author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 * @author cvrpa
 */

import java.util.Scanner;

public class CircularPrime {
    public static void main(String[] args)
    {
        boolean flag = true;
        int n, num, r, c = 0;

        Scanner sc = new Scanner(System.in);
    }
}

```



```

        System.out.print("Enter a number: ");
        n = sc.nextInt();

        num = n;
        while (num > 0)
        {
            r = num % 10;
            c++;
            num = num / 10;
        }
        num = n;
        for (int i = 1; i <= c; i++)
        {
            r = num % 10;
            num = num / 10;
            num = (r * (int) Math.pow(10, c - 1)) + num;
            if (!prime(num))
            {
                flag = false;
                break;
            }
        }

        if(flag)
        {
            System.out.println(n + " is a Circular Prime");
        }
        else
        {
            System.out.println(n + " is not a Circular Prime");
        }
    }

    static boolean prime(int n)
    {
        // TODO code application logic here
        int i = 2;
        boolean flag = true;
        while (n > i)
        {
            if (n % 2 == 0)
            {
                flag = false;
                break;
            }
            i++;
        }
        return flag;
    }
}

```

Result

Enter a number: 137
 137 is a Circular Prime

Enter a number: 44
 44 is not a Circular Prime

6.4.5 HappyNumber

Aim: A happy number is a natural number in a given number base that eventually reaches 1 when iterated over the perfect digital invariant function for. Those numbers that do not end in 1 are -unhappy numbers.

Program

```
/**
 * author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 * @author cvrpa
 */

import java.util.Scanner;

public class HappyNumber {
    public static void main(String[] args)
    {
        int n, r = 1, num, sum = 0;
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number: ");
        n = sc.nextInt();
        num = n;
        while (num > 9)
        {
            while (num > 0)
            {
                r = num % 10;
                sum = sum + (r * r);
                num = num / 10;
            }
            num = sum;
            sum = 0;
        }
        if (num == 1)
        {
            System.out.println(n + " is a Happy Number");
        }
        else
        {
            System.out.println(n + " is not a Happy Number");
        }
    }
}
```

Result

Enter number: 31

31 is a Happy Number

Enter number: 32

32 is a Happy Number

Enter number: 33

33 is not a Happy Number

6.4.6 AutomorphicNumber

Aim: An Automorphic number is a number whose square “ends” in the same digits as the number itself.

Program

```
/**
 * author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 * @author cvrpa
 */

import java.util.Scanner;

public class AutomorphicNumber {
    public static void main(String[] args)
    {
        int n, sqrNum, temp, sqrNumRemainder, c = 0;
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        n = sc.nextInt();

        // get the number of digits
        temp=n;
        while (temp > 0)
        {
            temp=temp/10;
            c++;
        }

        sqrNum = n * n;
        sqrNumRemainder= sqrNum%(int)Math.pow(10, c);

        if(sqrNumRemainder==n)
        {
            System.out.println(n + " is an Automorphic Number");
        }
        else
        {
            System.out.println(n + " is not an Automorphic Number");
        }
    }
}
```

Result

Enter a number: 5

5 is an Automorphic Number

Enter a number: 25

25 is an Automorphic Number

Enter a number: 6

6 is an Automorphic Number

Enter a number: 7

7 is not an Automorphic Number

6.4.7 DisariumNumber

Aim: A number is called Disarium number if the sum of its power of the positions from left to right is equal to the number.

Program

```
/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

import java.util.Scanner;

public class DisariumNumber {
    public static void main(String[] args)
    {
        int r, n, num, digits=0,
        sum = 0;
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        n = sc.nextInt();
        num = n;
        while (num > 0)
        {
            digits++;
            num = num / 10;
        }
        num = n;
        while (num > 0)
        {
            r = num % 10;
            sum = sum + (int)Math.pow(r, digits);
            num = num / 10;
            digits--;
        }

        if(n==sum)
        {
            System.out.println(n + " is a Disarium Number");
        }
        else
        {
            System.out.println(n + " is not a Disarium Number");
        }
    }
}
```

Result

Enter a number: 135
135 is a Disarium Number

Enter a number: 130
130 is not a Disarium Number

6.4.8 Magic Number

Aim: Magic number is the if the sum of its digits recursively are calculated till a single digit. If the single digit is 1 then the number is a magic number. Magic number is very similar with Happy Number.

Program

```
/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept of CSE,
 * IARE, Dundigal.
 */

import java.util.Scanner;

public class MagicNumbers {

    public static void main(String[] args) {
        int n, r = 1, num, sum = 0;
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        n = sc.nextInt();
        num = n;
        while (num > 9)
        {
            while (num > 0)
            {
                r = num % 10;
                sum = sum + r;
                num = num / 10;
            }
            num = sum;
            sum = 0;
        }
        if (num == 1)
        {
            System.out.println(n + " is a Magic Number");
        }
        else
        {
            System.out.println(n + " is not a Magic Number");
        }
    }
}
```

Result

Enter a number: 541
541 is a Magic Number

Enter a number: 226
226 is a Magic Number

Enter a number: 150
150 is not a Magic Number

6.4.9 NeonNumber

Aim: A neon number is a number where the sum of digits of square of the number is equal to the number. For example if the input number is 9, its square is $9*9 = 81$ and sum of the digits is 9. i.e. 9 is a neon number.

Program

```
/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept of CSE,
 * IARE, Dundigal.
 */

import java.util.Scanner;

public class NeonNumber {
    public static void main(String[] args)
    {
        // TODO code application logic here
        int n,
        sqr = 1,
        sum = 0, r;
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        n = sc.nextInt();
        sqr = n * n;
        while (sqr > 0)
        {
            r = sqr % 10;
            sum = sum + r;
            sqr = sqr / 10;
        }
        if (n == sum)
        {
            System.out.println(n + " is a Neon Number");
        }
        else
        {
            System.out.println(n + " is not a Neon Number");
        }
    }
}
```

Result

Enter a number: 9
9 is a Neon Number

Enter a number: 8
8 is not a Neon Number

6.4.10 PalindromicNumber

Aim: A palindromic number is a number that remains the same when its digits are reversed.

Program

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept of CSE,
 * IARE, Dundigal.
 */

import java.util.Scanner;

public class PalindromicNumber {
    public static void main(String[] args)
    {
        int n, num, r,
        rev = 0;
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        n = sc.nextInt();
        num = n;
        while (num > 0)
        {
            r = num % 10;
            rev = (rev * 10) + r;
            num = num / 10;
        }
        if (n == rev)
        {
            System.out.println(n + " is a Palindrome Number");
        }
        else
        {
            System.out.println(n + " is not a Palindrome Number");
        }
    }
}

```

Result

Enter a number: 245
 245 is not a Palindrome Number

Enter a number: 242
 242 is a Palindrome Number

6.4.11 PerfectNumber

Aim: A perfect number is a positive integer that is equal to the sum of its positive divisors, excluding the number itself. For instance, 6 has divisors 1, 2 and 3, and $1 + 2 + 3 = 6$, so 6 is a perfect number.

Program

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept of CSE,
 * IARE, Dundigal.
 */

import java.util.Scanner;

```

```

public class PerfectNumber {
    public static void main(String[] args)
    {
        int n, sum = 0;
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        n = sc.nextInt();
        for (int i = 1; i < n; i++)
        {
            if (n % i == 0)
            {
                sum = sum + i;
            }
        }
        if (n == sum)
        {
            System.out.println(n + " is a Perfect Number");
        }
        else
        {
            System.out.println(n + "is not a Perfect Number");
        }
    }
}

```

Result

Enter a number: 6
6 is a Perfect Number

Enter a number: 5
5 is not a Perfect Number

6.4.12 SpecialNumber

Aim: A number is said to be special number when the sum of factorial of its digits is equal to the number itself. Example- 145 is a Special Number as $1!+4!+5!=145$.

Program

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept of CSE,
 * IARE, Dundigal.
 */

import java.util.Scanner;

public class SpecialNumber {
    public static void main(String[] args)
    {
        int n, num, r,
        sumOfFactorial = 0;
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        n = sc.nextInt();
        num = n;
        while (num > 0)

```



```

        {
            r = num % 10;
            int fact=1;
            for(int i=1;i<=r;i++)
            {
                fact=fact*i;
            }
            sumOfFactorial = sumOfFactorial+fact;
            num = num / 10;
        }
        if(n==sumOfFactorial)
        {
            System.out.println(n + " is a Special Number" );
        }
        else
        {
            System.out.println(n + "is not a Special Number" );
        }
    }
}

```

Result

Enter a number: 145

145 is a Special Number

Enter a number: 23

23 is not a Special Number

6.4.13 SpyNumber

Aim: A spy number is a number where the sum of its digits equals the product of its digits. For example, 1124 is a spy number, the sum of its digits is $1+1+2+4=8$ and the product of its digits is $1*1*2*4=8$.

Program

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept of CSE,
 * IARE, Dundigal.
 */

import java.util.Scanner;

public class SpyNumber {
    public static void main(String[] args)
    {
        int r, n, num, mul = 1, sum = 0;
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        n = sc.nextInt();
        num = n;
        while (num > 0)
        {
            r = num % 10;
            sum = sum + r;
            mul = mul * r;

```

```

        num = num / 10;
    }
    if (mul == sum)
    {
        System.out.println(n + " is a Spy Number");
    }
    else
    {
        System.out.println(n + " is not a Spy Number");
    }
}
}

```

Result

Enter a number: 1124

1124 is a Spy Number

Enter a number: 12

12 is not a Spy Number

6.4.14 UglyNumber

Aim: A number is said to be an Ugly number if positive numbers whose prime factors only include 2, 3, 5. For example, 6(2×3), 8($2 \times 2 \times 2$), 15(3×5) are ugly numbers while 14(2×7) is not ugly since it includes another prime factor 7. Note that 1 is typically treated as an ugly number.

Program

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept of CSE,
 * IARE, Dundigal.
 */

import java.util.Scanner;

public class UglyNumber {
    public static void main(String[] args)
    {
        int n;
        boolean flag=true;
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        n = sc.nextInt();
        int temp = n;
        while (n != 1)
        {
            if (n % 5 == 0)
            {
                n /= 5;
            }
            else if (n % 3 == 0)
            {
                n /= 3;
            }
            else if (n % 2 == 0)
            {

```

```

        n /= 2;
    }
    else
    {
        flag=false;
        break;
    }
}
if (flag)
{
    System.out.println(temp + " is an Ugly number.");
}
else
{
    System.out.println(temp + " is not an Ugly number.");
}
sc.close();
}
}

```

Result

Enter a number: 6
6 is an Ugly number.

Enter a number: 14
14 is not an Ugly number.

6.5 Sample Viva Voce Questions

1. **Question:**

Answer:

2. **Question:**

Answer:

3. **Question:**

Answer:

4. **Question:**

Answer:

6.6 Further Probing Experiments

1. The least common multiple, lowest common multiple, or smallest common multiple of two integers a and b, usually denoted by LCM(a, b), is the smallest positive integer that is divisible by both a and b.

Write a Java Program to find the LCM of two given numbers

2. Two integers a and b are said to be relatively prime, mutually prime, or coprime if the only positive integer that divides both of them is 1. Example: 13 and 15 are co prime.

Write a Java program to check whether the two given numbers are co-prime or not

EXERCISES ON STRING AND CHAR OPERATIONS

7.1 Introduction

This lab, focus on the fundamental concepts of strings and characters in the context of Java programming. Strings are an integral part of any programming language, and understanding how to work with them is crucial for developing applications that handle textual data efficiently. In Java, strings and characters offer a rich set of features and functions, making them powerful tools for developers. This lab will provide you with a comprehensive understanding of strings and characters, from basic operations to more advanced techniques. By the end of this lab, you will be well-equipped to manipulate and work with text in Java, a skill that is vital for a wide range of software development tasks.

7.2 Prelab Preparation

7.2.1 Objective

By the end of this lab, students should be able to:

1. Understand the basics of strings in Java, including string declaration, initialization, and essential string operations like concatenation and comparison.
2. Perform various operations on strings, such as modifying, extracting substrings, and transforming text to meet specific requirements.
3. Work with individual characters, including character representation, comparison, and manipulation.

7.2.2 Background

Strings and characters are integral components of programming and are particularly significant in Java. Understanding how to work with them is fundamental for developing Java applications.

User Interaction: Strings are frequently used to process and manipulate user input. Understanding how to work with strings is crucial for building interactive applications where users provide textual information.

Data Storage: Strings are used to store and represent data, both within the program and in external storage, such as databases and files. Effective string handling is essential for data management.

Text Processing: Many applications involve text processing, from simple search and replace operations to more complex data extraction and transformation tasks. Mastery of strings and characters is essential for these operations.

Output Formatting: Properly formatted strings are crucial for presenting information to users or generating reports. The ability to format strings according to specific requirements is a valuable skill.

When working with strings and characters in Java, there are several fundamental concepts and techniques that are essential for efficient text processing. These concepts form the core knowledge required for effective manipulation of strings and characters in Java. Here are some of the required concepts:

String Declaration and Initialization: Understanding how to declare and initialize strings is fundamental. In Java, strings can be declared using the `String` class, and they can be initialized using string literals or by creating new instances.

In Java, you can declare and initialize strings in several ways. Here are some common methods for declaring and initializing strings:

- Using String Literals:

You can directly create a string using double-quoted string literals.

```
String greeting = "Hello, World!";
```

- Using the new Keyword:

You can create a string object using the new keyword.

```
String message = new String("This is a new string.");
```

- Using Empty String:

You can create an empty string by assigning an empty string literal to a variable.

```
String emptyString = "";
```

- Null String:

You can also create a string with a null value.

```
String nullString = null;
```

String Concatenation: Concatenating (joining) strings together is a common operation. The `+` operator is often used for string concatenation, and the `concat()` method of the `String` class can also be used.

String Length: Finding the length of a string is essential for various operations. You can use the `length()` method to determine the number of characters in a string.

String Comparison: You'll often need to compare strings for equality. The `equals()` method is used to compare the content of two strings, while `==` is used to compare their references.

String Manipulation: Learn how to manipulate strings by using methods like `substring()`, `charAt()`, `replace()`, `toLowerCase()`, `toUpperCase()`, and many more. These methods enable you to modify and extract parts of strings.

String Parsing: Parsing involves breaking down a string into smaller components, such as tokens. The `split()` method and regular expressions are commonly used for string parsing.

String Formatting: Properly formatting strings is crucial for displaying information to users. You can use methods like `String.format()` or printf-style formatting to format strings according to specific requirements.

Character Handling: Understand the basics of working with individual characters in strings. This includes accessing characters by index, comparing characters, and converting between characters and their integer representations.

Escape Sequences: Learn common escape sequences used to represent special characters within strings, such as `\n` for a newline or `\t` for a tab.

7.2.3 Hardware/Software Requirements

To successfully complete this lab, students will need access to the following:

1. A computer with a suitable text editor or Integrated Development Environment (IDE) installed, such as Eclipse, IntelliJ IDEA, or Visual Studio Code.
2. Java Development Kit (JDK) installed on the computer. Ensure that the JDK is correctly configured, and the system's PATH variable is set to include the JDK's bin directory.

7.2.4 Pre-Lab Questions

1. Why are strings important for user interaction in Java applications?
2. How do you declare and initialize a string using string literals in Java?
3. Explain the difference between using string literals and the `new` keyword to create a string.
4. What method is used to find the length of a string in Java, and why is it important?

7.3 Experiment Procedure

1. Ensure you have access to a Java development environment on your computer. Open your Java IDE or text editor for coding.
2. Create a new Java class with a suitable name in your IDE or text editor.
3. Write Java code to solve the problem
4. Test your code with different input values to ensure it behaves as expected. Debug and fix any issues that arise during testing.
5. Document the problem statement, the approach used to solve it, and the code you've written in your lab notebook or report.
6. Experiment with variations of the problem, explore different scenarios, and modify your code to handle various cases.

7.4 Experiments, Results & Discussion

7.4.1 ReverseString (String & char)

Aim: Write a program called ReverseString, which prompts user for a String, and prints the reverse of the String by extracting and processing each character.

Program

```
/**
 *author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 * @author cvrpa
 */

import java.util.Scanner;

public class ReverseString {
    public static void main(String args[]){
        // Define variables
        String inStr;          // input String
        int inStrLen;          // length of the input String

        Scanner in = new Scanner(System.in);
        // Prompt and read input as "String"
        System.out.print("Enter a String: ");
        inStr = in.next();    // use next() to read a String
        inStrLen = inStr.length();

        // Use inStr.charAt(index) in a loop to extract each character
        // The String's index begins at 0 from the left.
        // Process the String from the right
        System.out.print("The reverse of the String " + inStr + " is ");
        for (int charIdx = inStrLen - 1; charIdx >= 0; --charIdx) {
            // charIdx = inStrLen-1, inStrLen-2, ... ,0
            System.out.print(inStr.charAt(charIdx));
        }

    }
}
```

Result

Enter a String: abcdefghijk

The reverse of the String abcdefghijk is kjihgfedcba

7.4.2 CountVowelsDigits (String & char)

Aim: Write a program called CountVowelsDigits, which prompts the user for a String, counts the number of vowels (a, e, i, o, u, A, E, I, O, U) and digits (0-9) contained in the string, and prints the counts and the percentages (rounded to 2 decimal places).

Program

```
/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
```

```

* IARE, Dundigal.
* @author cvrpa
*/

import java.util.Scanner;

public class CountVowelsDigits {
    public static void main(String args[]){
        Scanner in = new Scanner(System.in);

        // Prompt the user for input
        System.out.print("Enter a string: ");
        String input = in.nextLine();

        int vCount = 0;
        int dCount = 0;
        // Count vowels
        String vowels = "aeiouAEIOU";
        for (int i = 0; i < input.length(); i++) {
            char c = input.charAt(i);
            if (vowels.indexOf(c) != -1) {
                vCount++;
            }
            if (Character.isDigit(c)) {
                dCount++;
            }
        }

        // Calculate percentages
        int totalCharacters = input.length();
        double vowelPercentage = (double) vCount / totalCharacters * 100;
        double digitPercentage = (double) dCount / totalCharacters * 100;

        // Print the results
        System.out.printf("Number of vowels: %d (%.2f%%)\n", vCount,
            vowelPercentage);
        System.out.printf("Number of digits: %d (%.2f%%)\n", dCount,
            digitPercentage);

        // Close the scanner
        in.close();
    }
}

```

Result

```

Enter a string: testing123456
Number of vowels: 2 (15.38%)
Number of digits: 6 (46.15%)

```

7.4.3 PhoneKeyPad (String & char)

Aim: On you phone keypad, the alphabets are mapped to digits as follows:

ABC(2), DEF(3), GHI(4), JKL(5), MNO(6), PQRS(7), TUV(8), WXYZ(9).

Write a program called PhoneKeyPad, which prompts user for a String (case insensitive),

and converts to a sequence of keypad digits.

Program

```
/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 * @author cvrpa
 */

import java.util.Scanner;

public class PhoneKeyPad {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);

        // Prompt the user for input
        System.out.print("Enter a string: ");
        String input = in.nextLine().toUpperCase(); // Convert to uppercase
            for case insensitivity

        // Convert the input string to keypad digits
        StringBuilder result = new StringBuilder();
        System.out.print("The sequence of keypad digits: ");
        for (int i = 0; i < input.length(); i++) {
            char c = input.charAt(i);
            switch(c){
                case 'A': case 'B': case 'C':
                    System.out.print(2); break;
                case 'D': case 'E': case 'F':
                    System.out.print(3); break;
                case 'G': case 'H': case 'I':
                    System.out.print(4); break;
                case 'J': case 'K': case 'L':
                    System.out.print(5); break;
                case 'M': case 'N': case 'O':
                    System.out.print(6); break;
                case 'P': case 'Q': case 'R': case 'S':
                    System.out.print(7); break;
                case 'T': case 'U': case 'V':
                    System.out.print(8); break;
                case 'W': case 'X': case 'Y': case 'Z':
                    System.out.print(9); break;
                default:
                    System.out.print("");
            }
        }
        // Close the scanner
        in.close();
    }
}
```

Result

Enter a string: abcdxyhjs

The sequence of keypad digits: 222399457

7.4.4 Caesar's Code (String & char)

Aim: Caesar's Code is one of the simplest encryption techniques. Each letter in the plaintext is replaced by a letter some fixed number of position (n) down the alphabet cyclically. In this exercise, we shall pick n=3. That is, 'A' is replaced by 'D', 'B' by 'E', 'C' by 'F', ..., 'X' by 'A', ..., 'Z' by 'C'.

Write a program called CaesarCode to cipher the Caesar's code. The program shall prompt user for a plaintext string consisting of mix-case letters only; compute the ciphertext; and print the ciphertext in uppercase.

Program

```
/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept of CSE,
 * IARE, Dundigal.
 */

import java.util.Scanner;

public class CaesarsCode {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a plaintext string: ");

        String plaintext = sc.nextLine();
        plaintext = plaintext.toUpperCase();

        String ciphertext = " ";

        for (int i = 0; i < plaintext.length(); i++) {
            char originalChar = plaintext.charAt(i);

            if (Character.isLetter(originalChar)) {
                char shiftedChar = (char) (originalChar + 3);
                ciphertext += shiftedChar;
            } else {
                // If the character is not a letter, add it unchanged to
                // the ciphertext
                ciphertext += originalChar;
            }
        }
        System.out.println("The ciphertext string is:" + ciphertext);
        sc.close();
    }
}
```

Result

Enter a plaintext string: Testing

The ciphertext string is: WHVWLQJ

7.4.5 Decipher Caesar's Code (String & char)

Aim: Write a program called DecipherCaesarCode to decipher the Caesar's code described in the previous exercise. The program shall prompts user for a ciphertext string consisting of mix-case letters only; compute the plaintext; and print the plaintext in uppercase.

Program

```
/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept of CSE,
 * IARE, Dundigal.
 */

import java.util.Scanner;

public class DecipherCaesarCode {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a plaintext string: ");

        String plaintext = sc.nextLine();
        plaintext = plaintext.toUpperCase();

        String ciphertext = " ";

        for (int i = 0; i < plaintext.length(); i++) {
            char originalChar = plaintext.charAt(i);

            if (Character.isLetter(originalChar)) {
                char shiftedChar = (char) (originalChar - 3);
                ciphertext += shiftedChar;
            } else {
                // If the character is not a letter, add it unchanged to
                // the ciphertext
                ciphertext += originalChar;
            }
        }
        System.out.println("The ciphertext string is:" + ciphertext);
        sc.close();
    }
}
```

Result

Enter a plaintext string: wHVwLQJThe ciphertext string is: TESTING

7.4.6 Exchange Cipher (String & char)

Aim: This simple cipher exchanges 'A' and 'Z', 'B' and 'Y', 'C' and 'X', and so on. Write a program called ExchangeCipher that prompts user for a plaintext string consisting of mix-case letters only. Your program shall compute the ciphertext; and print the ciphertext in uppercase.

Program

```
/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept of CSE,
 * IARE, Dundigal.
 */

import java.util.Scanner;

public class ExchangeCipher {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

```

        System.out.print("Enter a plaintext string: ");

        String plaintext = sc.nextLine();
        plaintext = plaintext.toUpperCase();

        String ciphertext = " ";

        for (int i = 0; i < plaintext.length(); i++) {
            char originalChar = plaintext.charAt(i);

            if (Character.isLetter(originalChar)) {
                char shiftedChar = (char) ('A' + 'Z' - originalChar);
                ciphertext += shiftedChar;
            } else {
                // If the character is not a letter, add it unchanged to
                // the ciphertext
                ciphertext += originalChar;
            }
        }
        System.out.println("The ciphertext string is:" + ciphertext);
        sc.close();
    }
}

```

Result

Enter a plaintext string: abcXYZ

The ciphertext string is: ZYXCBA

7.4.7 TestPalindromicWord (String & char)

Aim: A word that reads the same backward as forward is called a palindrome, e.g., "mom", "dad", "racecar", "madam", and "Radar" (case-insensitive).

Write a program called TestPalindromicWord, that prompts user for a word and prints "xxx" is—is not a palindrome".

Program

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept of CSE,
 * IARE, Dundigal.
 */

import java.util.Scanner;

public class TestPalindromicPhrase {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter a phrase: ");
        String phrase = scanner.nextLine();
        scanner.close();
        boolean flag = true;

        phrase = phrase.replaceAll("[^a-zA-Z]", "").toLowerCase();
        int length = phrase.length();
        int fIdx = 0, bIdx = length - 1;
    }
}

```

```

while (fIdx < bIdx) {
    char forwardChar = phrase.charAt(fIdx);
    char backwardChar = phrase.charAt(bIdx);

    if (forwardChar != backwardChar) {
        flag = false;
        break;
    }
    ++fIdx;
    --bIdx;
}

if (flag) {
    System.out.println("The phrase is a palindromic phrase.");
} else {
    System.out.println("The phrase is not a palindromic phrase.");
}
}
}

```

Result

Enter a word: maDaM

maDaM is a palindromic word.

7.4.8 CheckBinStr (String & char)

Aim: The binary number system uses 2 symbols, 0 and 1. Write a program called CheckBinStr to verify a binary string. The program shall prompt user for a binary string; and decide if the input string is a valid binary string.

Program

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept of CSE,
 * IARE, Dundigal.
 */

import java.util.Scanner;

public class CheckBinStr {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a binary string: ");
        String binaryStr = sc.nextLine();
        boolean flag = true;

        for (char c : binaryStr.toCharArray()) {
            if (c != '0' && c != '1') {
                flag = false;
                break;
            }
        }

        if (flag) {
            System.out.println "\"" + binaryStr + "\" is a binary string");
        } else {

```

```

        System.out.println "\"" + binaryStr + "\" is NOT a binary
            string");
    }
    sc.close();
}
}

```

Result

Enter a binary string: 10101100
 "10101100" is a binary string

Enter a binary string: 10201011
 "10201011" is NOT a binary string

7.4.9 CheckHexStr (String & char)

Aim: The hexadecimal (hex) number system uses 16 symbols, 0-9 and A-F (or a-f). Write a program to verify a hex string. The program shall prompt user for a hex string; and decide if the input string is a valid hex string.

Program

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept of CSE,
 * IARE, Dundigal.
 */

import java.util.Scanner;

public class CheckHexStr {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a hex string: ");
        String hexStr = sc.nextLine();
        boolean flag = true;

        for (char c : hexStr.toCharArray()) {
            if (!Character.isDigit(c) && !((c >= 'A' && c <= 'F') || (c >=
                'a' && c <= 'f')))) {
                flag = false;
            }
        }

        if (flag) {
            System.out.println "\"" + hexStr + "\" is a hex string";
        } else {
            System.out.println "\"" + hexStr + "\" is NOT a hex string";
        }

        sc.close();
    }
}

```

Result

Enter a Hexadecimal string: 123aBc
 The equivalent binary for hexadecimal "123aBc" is: 0001 0010 0011 1010 1011 1100

Enter a Hexadecimal string: 123aBc

The equivalent binary for hexadecimal "123aBc" is: 0001 0010 0011 1010 1011 1100

7.4.10 Bin2Dec (String & char)

Aim: Write a program called Bin2Dec to convert an input binary string into its equivalent decimal number.

Program

```
/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept of CSE,
 * IARE, Dundigal.
 */

import java.util.Scanner;

public class Bin2Dec {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter a Binary string: ");
        String binaryStr = scanner.nextLine();
        boolean flag = true;

        int decimal = 0;
        int binaryLength = binaryStr.length();
        int power = 0;

        // Iterate through the binary string from right to left
        for (int i = binaryLength - 1; i >= 0; i--) {
            char digit = binaryStr.charAt(i);

            // Check if the character is '0' or '1'
            if (digit == '0' || digit == '1') {
                int bitValue = Character.getNumericValue(digit);
                decimal += bitValue * Math.pow(2, power);
                power++;
            } else {
                flag = false;
                break;
            }
        }

        if(flag){
            System.out.println("The equivalent decimal number for binary\n" + binaryStr + "\n is: " + decimal);
        }
        else{
            System.out.println("error: invalid binary string\n" + binaryStr + "\n");
        }
        scanner.close();
    }
}
```

Result

Enter a Binary string: 1011

The equivalent decimal number for binary "1011" is: 11

Enter a Binary string: 1234

error: invalid binary string "1234"

7.4.11 Hex2Dec (String & char)

Aim: Write a program called Hex2Dec to convert an input hexadecimal string into its equivalent decimal number.

Program

```
/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept of CSE,
 * IARE, Dundigal.
 */

import java.util.Scanner;

public class Hex2Dec {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter a Hexadecimal string: ");
        String hexStr = scanner.nextLine();

        int decimal = 0;

        // Convert the hex string to uppercase for consistency
        hexStr = hexStr.toUpperCase();

        for (int i = 0; i < hexStr.length(); i++) {
            char digit = hexStr.charAt(i);

            if (Character.isDigit(digit)) {
                decimal = decimal * 16 + (digit - '0');
            } else if (digit >= 'A' && digit <= 'F') {
                decimal = decimal * 16 + (digit - 'A' + 10);
            } else {
                // Invalid hexadecimal digit found
                decimal = -1;
                break;
            }
        }

        if (decimal != -1) {
            System.out.println("The equivalent decimal number for
                hexadecimal \"" + hexStr + "\" is: " + decimal);
        } else {
            System.out.println("error: invalid hexadecimal string \"" +
                hexStr + "\"");
        }
        scanner.close();
    }
}
```


Result Enter a Hexadecimal string: 1a

The equivalent decimal number for hexadecimal "1A" is: 26

Enter a Hexadecimal string: abcd

The equivalent decimal number for hexadecimal "ABCD" is: 43981

Enter a Hexadecimal string: 1g5

error: invalid hexadecimal string "1G5"

7.4.12 Oct2Dec (String & char)

Aim: Write a program called Oct2Dec to convert an input Octal string into its equivalent decimal number.

Program

```
/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept of CSE,
 * IARE, Dundigal.
 */

import java.util.Scanner;

public class Oct2Dec {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter an Octal string: ");
        String octalStr = scanner.nextLine();

        int decimal = 0;

        for (int i = 0; i < octalStr.length(); i++) {
            char digit = octalStr.charAt(i);

            if (digit >= '0' && digit <= '7') {
                decimal = decimal * 8 + (digit - '0');
            } else {
                // Invalid octal digit found
                decimal = -1;
                break;
            }
        }

        if (decimal != -1) {
            System.out.println("The equivalent decimal number \"" +
                octalStr + "\" is: " + decimal);
        } else {
            System.out.println("error: Invalid Octal string \"" + octalStr
                + "\"");
        }
        scanner.close();
    }
}
```

Result

Enter an Octal string: 234

The equivalent decimal number "234" is: 156

Enter an Octal string: 148

error: Invalid Octal string "148"

7.5 Sample Viva Voce Questions

1. Give an example of a string concatenation operation in Java.
2. Discuss the concept of character manipulation in strings and provide an example.
3. How do you compare two strings for equality in Java, and what's the distinction between `equals()` and `==` for strings?
4. How can you extract a substring from a given string, and what method is used for this purpose?

7.6 Further Probing Experiments

1. A phrase that reads the same backward as forward is also called a palindrome, e.g., "Madam, I'm Adam", "A man, a plan, a canal - Panama!" (ignoring punctuation and capitalization).

Write a program to check for palindromic phrase.

2. Write a Java program that takes a sentence as input, splits it into individual words using spaces as separators, and then displays both the original sentence and the total word count. For example, if the user enters 'Hello, World!,' the program should output 'Original Sentence: Hello, World! Total Words: 2.' Your program should handle typical sentence structures and spaces as separators.

EXERCISES ON ARRAYS

8.1 Introduction

Arrays are fundamental data structures that play a pivotal role in Java and many other programming languages. Arrays allow us to store and manipulate collections of data, making them a crucial component in a programmer's toolkit. In this lab, we will delve into the world of arrays in Java, exploring the concepts of array declaration, initialization, and manipulation. We will learn about the various types of arrays, multidimensional arrays, and how to work with them effectively. Arrays are the building blocks of many advanced data structures and algorithms, making this lab essential for your journey as a Java programmer.

8.2 Prelab Preparation

8.2.1 Objective

By the end of this lab, students should be able to:

1. Demonstrate a clear understanding of array basics, including declaration, initialization, and indexing in Java.
2. Effectively work with different array types, such as one-dimensional and multidimensional arrays, for data organization.
3. Perform array manipulation tasks, including adding, modifying, and deleting elements, as well as implementing common array operations like sorting and searching.
4. Apply their knowledge of arrays to solve practical programming problems, enhancing their problem-solving skills.

8.2.2 Background

Arrays are fundamental data structures in programming, and in Java, they play a critical role in organizing and managing data. Arrays allow us to store collections of values of the same data type, providing an efficient means to access and manipulate this data.

Declaration and Initialization:

In Java, arrays are declared by specifying the data type of the elements they will hold, followed by square brackets (`[]`) to denote that it's an array. For example, to declare an array of integers: `int[] myArray;`. Arrays can be initialized with values, either during declaration or separately.

```
int[] numbers = {1, 2, 3, 4, 5};
```

Indexing:

Arrays are zero-indexed, which means the first element is accessed at index 0, the second at index 1, and so on. Proper indexing is essential for working with arrays.

```
int firstNumber = numbers[0]; // Access the first element.
```

Array Length:

To find the number of elements in an array, you can use the length attribute.

```
int arrayLength = numbers.length; // Gets the length of the array.
```

One-Dimensional and Multidimensional Arrays:

Java supports both one-dimensional arrays (vectors) and multidimensional arrays (matrices). Multidimensional arrays can have multiple rows and columns.

```
int[][] matrix = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
```

Array Manipulation:

Arrays can be manipulated by adding, modifying, or removing elements. Common array operations include sorting and searching for specific values within an array.

```
int[] extendedArray = Arrays.copyOf(numbers, numbers.length + 1);
extendedArray[numbers.length] = 6;
```

Arrays are the foundation for many advanced data structures and algorithms. They provide a simple yet powerful way to manage large amounts of data, making them essential for tasks such as data storage, sorting, searching, and more. Understanding arrays is crucial for any Java programmer, as it opens the door to efficient data management and problem-solving in a wide range of software development projects

8.2.3 Hardware/Software Requirements

To successfully complete this lab, students will need access to the following:

1. A computer with a suitable text editor or Integrated Development Environment (IDE) installed, such as Eclipse, IntelliJ IDEA, or Visual Studio Code.
2. Java Development Kit (JDK) installed on the computer. Ensure that the JDK is correctly configured, and the system's PATH variable is set to include the JDK's bin directory.

8.2.4 Pre-Lab Questions

1. What is an array in Java, and why are they important in programming?
2. How do you declare and initialize a one-dimensional array in Java?
3. How do you access elements in an array?
4. What is the role of the length attribute in arrays, and how do you use it?

8.3 Experiment Procedure

1. Ensure you have access to a Java development environment on your computer. Open your Java IDE or text editor for coding.
2. Create a new Java class with a suitable name in your IDE or text editor.

3. Write Java code to solve the problem
4. Test your code with different input values to ensure it behaves as expected. Debug and fix any issues that arise during testing.
5. Document the problem statement, the approach used to solve it, and the code you've written in your lab notebook or report.
6. Experiment with variations of the problem, explore different scenarios, and modify your code to handle various cases.

8.4 Experiments, Results & Discussion

8.4.1 PrintArray (Array)

Aim: Write a program called PrintArray which prompts user for the number of items in an array (a non-negative integer), and saves it in an int variable called NUM_ITEMS. It then prompts user for the values of all the items and saves them in an int array called items. The program shall then print the contents of the array in the form of [x1, x2, ..., xn].

Program

```
/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept of CSE,
 * IARE, Dundigal.
 */

import java.util.Scanner;

public class PrintArray {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Prompt the user for the number of items
        System.out.print("Enter the number of items: ");
        int NUM_ITEMS = scanner.nextInt();

        // Create an array to store the items
        int[] items = new int[NUM_ITEMS];

        // Prompt the user for the values of all items
        System.out.print("Enter the value of all items (separated by space)
            : ");
        for (int i = 0; i < NUM_ITEMS; i++) {
            items[i] = scanner.nextInt();
        }

        // Print the array's contents
        System.out.print("The values are: [");
        for (int i = 0; i < NUM_ITEMS; i++) {
            System.out.print(items[i]);
            if (i < NUM_ITEMS - 1) {
                System.out.print(", ");
            }
        }
        System.out.println("]");
        scanner.close();
    }
}
```

```
    }
}
```

Result

Enter the number of items: 5

Enter the value of all items (separated by space): 23 45 34 67 58

The values are: [23, 45, 34, 67, 58]

8.4.2 PrintArrayInStars (Array)

Aim: Write a program called PrintArrayInStars which prompts user for the number of items in an array (a non-negative integer), and saves it in an int variable called NUM_ITEMS. It then prompts user for the values of all the items (non-negative integers) and saves them in an int array called items. The program shall then print the contents of the array in a graphical form, with the array index and values represented by number of stars.

Program

```
/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept of CSE,
 * IARE, Dundigal.
 */
import java.util.Scanner;

public class PrintArrayInStars {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Prompt the user for the number of items
        System.out.print("Enter the number of items: ");
        int NUM_ITEMS = scanner.nextInt();

        // Create an array to store the items
        int[] items = new int[NUM_ITEMS];

        // Prompt the user for the values of all items
        System.out.print("Enter the value of all items (separated by space)
            : ");
        for (int i = 0; i < NUM_ITEMS; i++) {
            items[i] = scanner.nextInt();
        }

        // Print the array's contents using stars
        for (int i = 0; i < NUM_ITEMS; i++) {
            System.out.print(i + ": ");
            for (int j = 0; j < items[i]; j++) {
                System.out.print("*");
            }
            System.out.print("(" + items[i] + ")\n");
        }
        scanner.close();
    }
}
```

Result

Enter the number of items: 5
Enter the value of all items (separated by space): 3 5 2 0 6
0: *******(3)
1: *********(5)
2: ******(2)
3: **(0)**
4: *********(6)

8.4.3 GradesStatistics (Array)

Aim: Write a program which prompts user for the number of students in a class (a non-negative integer), and saves it in an int variable called numStudents. It then prompts user for the grade of each of the students (integer between 0 to 100) and saves them in an int array called grades. The program shall then compute and print the average (in double rounded to 2 decimal places) and minimum/maximum (in int).

Program

```
/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept of CSE,
 * IARE, Dundigal.
 */
import java.util.Scanner;

public class GradesStatistics {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Prompt the user for the number of students
        System.out.print("Enter the number of students: ");
        int numStudents = scanner.nextInt();

        // Create an array to store the grades
        int[] grades = new int[numStudents];

        // Prompt the user for the grades of each student
        for (int i = 0; i < numStudents; i++) {
            System.out.print("Enter the grade for student " + (i + 1) + ": ");
            grades[i] = scanner.nextInt();
        }

        // Calculate the average, minimum, and maximum grades
        double sum = 0;
        int minimum = Integer.MAX_VALUE;
        int maximum = Integer.MIN_VALUE;

        for (int grade : grades) {
            sum += grade;
            if (grade < minimum) {
                minimum = grade;
            }
            if (grade > maximum) {
                maximum = grade;
            }
        }
    }
}
```

```

        }
    }

    double average = sum / numStudents;

    // Print the results
    System.out.printf("The average is: %.2f\n", average);
    System.out.println("The minimum is: " + minimum);
    System.out.println("The maximum is: " + maximum);

    scanner.close();
}
}

```

Result

```

Enter the number of students: 5
Enter the grade for student 1: 56
Enter the grade for student 2: 67
Enter the grade for student 3: 78
Enter the grade for student 4: 59
Enter the grade for student 5: 46
The average is: 61.20
The minimum is: 46
The maximum is: 78

```

8.4.4 Hex2Bin (Array for Table Lookup)

Aim: Write a program called Hex2Bin that prompts user for a hexadecimal string and print its equivalent binary string.

Hints 1. Use an array of 16 Strings containing binary strings corresponding to hexadecimal number 0-9A-F (or a-f), as follows

```

final String[] HEX\_BITS = {
    "0000", "0001", "0010", "0011",
    "0100", "0101", "0110", "0111",
    "1000", "1001", "1010", "1011",
    "1100", "1101", "1110", "1111"};

```

Program

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept of CSE,
 * IARE, Dundigal.
 */
import java.util.Scanner;

public class Hex2Bin {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        final String[] HEX_BITS = {
            "0000", "0001", "0010", "0011",
            "0100", "0101", "0110", "0111",
            "1000", "1001", "1010", "1011",
            "1100", "1101", "1110", "1111"

```



```

};

System.out.print("Enter a Hexadecimal string: ");
String temp= scanner.nextLine(); // Convert to uppercase for
    consistency
String hexString = temp.toUpperCase();
StringBuilder binaryStringBuilder = new StringBuilder();

for (char hexDigit : hexString.toCharArray()) {
    if (hexDigit >= '0' && hexDigit <= '9') {
        int index = hexDigit - '0'; // Convert from character
            '0'-'9' to integer 0-9
        binaryStringBuilder.append(HEX_BITS[index]);
        binaryStringBuilder.append(" ");
    } else if (hexDigit >= 'A' && hexDigit <= 'F') {
        int index = hexDigit - 'A' + 10; // Convert from character
            'A'-'F' to integer 10-15
        binaryStringBuilder.append(HEX_BITS[index]);
        binaryStringBuilder.append(" ");
    } else {
        System.out.println("Invalid hexadecimal digit: " + hexDigit
            );
        scanner.close();
        return;
    }
}

String binaryString = binaryStringBuilder.toString();
System.out.println("The equivalent binary for hexadecimal \"" +
    temp + "\" is: " + binaryString);
scanner.close();
}
}

```

Result

Enter a Hexadecimal string: 1abc

The equivalent binary for hexadecimal "1abc" is: 0001 1010 1011 1100

8.4.5 Dec2Hex (Array for Table Lookup)

Aim: Write a program called Dec2Hex that prompts user for a positive decimal number, read as int, and print its equivalent hexadecimal string.

Program

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept of CSE,
 * IARE, Dundigal.
 */
import java.util.Scanner;

public class Dec2Hex {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter a decimal number: ");
        int decimalNumber = scanner.nextInt();
    }
}

```

```

        if (decimalNumber < 0) {
            System.out.println("Please enter a positive decimal number.");
        } else {
            String hexadecimalString = Integer.toHexString(decimalNumber).
                toUpperCase();
            System.out.println("The equivalent hexadecimal number is " +
                hexadecimalString);
        }
        scanner.close();
    }
}

```

Result

Enter a decimal number: 1234

The equivalent hexadecimal number is 4D2

Enter a decimal number: 120

The equivalent hexadecimal number is 78

8.5 Sample Viva Voce Questions

1. What is the significance of zero-based indexing in arrays?
2. Explain the importance of proper indexing and what happens when you access an out-of-bounds index.
3. How do you initialize a multidimensional array in Java, and what's the purpose of such arrays?
4. Can you have an array without specifying its size during declaration? If yes, how?

8.6 Further Probing Experiments

1. Given an array and a key, the task is to remove all occurrences of the specified key from the array in Java.
2. Write a Java Program for a given array of integers `arr[]` of size `N` and an integer, the task is to rotate the array elements to the left by `d` positions.

EXERCISES ON METHODS

9.1 Introduction

In this lab session, we explore the fundamental role of methods as the cornerstone of Java programming. Methods are crucial for code organization, modularity, and reusability, making it easier to handle complex problem-solving and improve code maintainability. This lab covers key concepts, including method declaration, parameters, return types, and method overloading, empowering participants to create modular, efficient, and clear Java code. Upon completing this session, students will be skilled at crafting and implementing custom methods for practical problem-solving, thus setting the foundation for a deeper understanding of Java programming and its practical applications.

9.2 Prelab Preparation

9.2.1 Objective

By the end of this lab, students should be able to:

1. Gain a clear understanding of how methods in Java serve as building blocks for code organization, encapsulation, and code reuse.
2. Learn to declare and define methods, including methods with parameters, to accept and process data in your programs.
3. Understand the concept of return types, and explore the benefits of method overloading for creating flexible and versatile code.
4. Discover best practices for method naming, scope, and access modifiers to enhance code readability, maintainability, and collaboration.
5. Apply the knowledge of methods to solve practical programming problems, creating modular, reusable, and efficient code.

9.2.2 Background

In Java, a method is a block of code that performs a specific task or function. Methods are a fundamental building block of Java programs and serve as a means to organize, encapsulate, and reuse code. They are critical for creating modular and maintainable software.

1. Method Declaration

To create a method, you declare it using the following syntax:

```
[accessModifier] [static] [returnType] methodName([parameterList])
{
    // Method body
}
```

A method declaration in Java consists of an optional access modifier, an optional static keyword, a return type, the method name, an optional parameter list and the method body contains the actual code to be executed.

methodName: This is the name of the method, which should be a valid Java identifier.

Method Body: The method body is enclosed in curly braces and contains the actual code to be executed when the method is called. This code can include variables, statements, and expressions.

Example:

```
public int calculateSum(int num1, int num2) {
    return num1 + num2;
}
```

In this example, the method `calculateSum` is declared with a return type of `int` and takes two integer parameters. It returns the sum of the two numbers.

2. Parameters

Parameters are variables that receive values when the method is called. They are enclosed in parentheses in the method declaration and can be of various data types.

```
returnType methodName([parameterType parameterName, ...]) {
    // Method body
}
```

Example:

```
public void greetUser(String name) {
    System.out.println("Hello, " + name + "!");
}
```

In this example, the `greetUser` method takes a `String` parameter `name` and displays a greeting message with the provided name.

3. Return Types

Return Type specifies the data type of the value that the method will return after it has finished its execution. If the method doesn't return anything, the return type is declared as `void`.

```
returnType methodName([parameterType parameterName, ...]) {
    // Method body
    return returnValue;
}
```

Example:

```
public double calculateAverage(int[] numbers) {
    int sum = 0;
    for (int num : numbers) {
        sum += num;
    }
    return (double) sum / numbers.length;
}
```

In this example, the `calculateAverage` method returns a `double` value representing the average of an array of integers.

4. Method Overloading:

Method overloading allows the same method name to be used with different parameter lists. The method is distinguished based on the number or data types of its parameters. You can have multiple methods with the same name but different parameters.

Syntax:

```
returnType methodName(parameterType parameterName) { ... }
returnType methodName(parameterType1 parameterName1, parameterType2
    parameterName2) { ... }
```

Example:

```
int add(int num1, int num2) { ... }
double add(double num1, double num2) { ... }
```

In this example, there are two add methods with different parameter types. The Java compiler determines which method to call based on the argument types provided during the method invocation.

These concepts are essential in Java programming, as they enable developers to create modular and reusable code, enhancing code organization and efficiency. Understanding these principles is crucial for building robust and maintainable software.

Key Characteristics of Methods:

Modularity: Methods allow you to break down complex tasks into smaller, manageable components. This makes your code easier to read, understand, and maintain.

Reusability: Once a method is defined, you can call it multiple times from various parts of your program. This reusability reduces redundancy in your code and promotes a more efficient development process.

Encapsulation: Methods can hide the implementation details of a specific task. This encapsulation allows you to abstract away the complexity and provide a simple interface for other parts of your code to interact with.

9.2.3 Hardware/Software Requirements

To successfully complete this lab, students will need access to the following:

1. A computer with a suitable text editor or Integrated Development Environment (IDE) installed, such as Eclipse, IntelliJ IDEA, or Visual Studio Code.
2. Java Development Kit (JDK) installed on the computer. Ensure that the JDK is correctly configured, and the system's PATH variable is set to include the JDK's bin directory.

9.2.4 Pre-Lab Questions

1. What is a method in Java, and why is it important in programming?
2. How do you declare a method in Java, and what components are involved in the method declaration?
3. Can a method have parameters, and how are they defined in the method declaration?
4. What does it mean when the return type is void?

9.3 Experiment Procedure

1. Ensure you have access to a Java development environment on your computer. Open your Java IDE or text editor for coding.
2. Create a new Java class with a suitable name in your IDE or text editor.
3. Write Java code to solve the problem
4. Test your code with different input values to ensure it behaves as expected. Debug and fix any issues that arise during testing.
5. Document the problem statement, the approach used to solve it, and the code you've written in your lab notebook or report.
6. Experiment with variations of the problem, explore different scenarios, and modify your code to handle various cases.

9.4 Experiments, Results & Discussion

9.4.1 exponent() (method)

Aim: Write a method called `exponent(int base, int exp)` that returns an int value of base raised to the power of exp. The signature of the method is:

```
public static int exponent(int base, int exp);
```

Assume that exp is a non-negative integer and base is an integer. Do not use any Math library functions. Also write the `main()` method that prompts user for the base and exp; and prints the result.

Program

```
/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept of CSE,
 * IARE, Dundigal.
 */
import java.util.Scanner;

public class ExponentMethod {
    public static void main(String[] args) {
        // Declare variables
        int exp;    // exponent (non-negative integer)
        int base;   // base (integer)

        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the base: ");
        base = sc.nextInt();
        System.out.print("Enter the exponent: ");
        exp = sc.nextInt();

        // Print result
        System.out.println(base + " raises to the power of " + exp + " is:
            " + exponent(base, exp));
    }

    // Returns "base" raised to the power "exp"
    public static int exponent(int base, int exp) {
```

```

        int product = 1;    // resulting product

        // Multiply product and base for exp number of times
        for (int i = 1; i<=exp; i++) {
            product *= base;
        }
        return product;
    }
}

```

Result

Enter the base: 3

Enter the exponent: 4

3 raises to the power of 4 is: 81

Enter the base: 8

Enter the exponent: 2

8 raises to the power of 2 is: 64

9.4.2 isOdd() (method)

Aim: Write a boolean method called isOdd() in a class called OddEvenTest, which takes an int as input and returns true if the it is odd. The signature of the method is as follows:

```
public static boolean isOdd(int number);
```

Also write the main() method that prompts user for a number, and prints "ODD" or "EVEN". You should test for negative input.

Program

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept of CSE,
 * IARE, Dundigal.
 */
import java.util.Scanner;

public class OddEvenTest {
    public static void main(String[] args) {
        // Declare variables
        int number;
        boolean flag = true;

        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        number = sc.nextInt();

        flag = isOdd(number);
        if(flag){
            System.out.println(number + " is an even number");}
        else{
            System.out.println(number + " is an odd number");}
    }
    public static boolean isOdd(int number){
        return number%2 == 0;
    }
}

```

Result

Enter a number: 5
5 is an odd number

Enter a number: 6
6 is an even number

Enter a number: -10
-10 is an even number

9.4.3 hasEight() (method)

Aim: Write a boolean method called hasEight(), which takes an int as input and returns true if the number contains the digit 8 (e.g., 18, 168, 1288). The signature of the method is as follows:

```
public static boolean hasEight(int number);
```

Write a program called MagicSum, which prompts user for integers (or -1 to end), and produce the sum of numbers containing the digit 8.

Program

```
/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept of CSE,
 * IARE, Dundigal.
 */
import java.util.Scanner;

public class MagicSum {
    public static void main(String[] args) {
        final int SENTINEL = -1;
        int sum = 0, number = 0;
        Scanner scanner = new Scanner(System.in);

        while (number != SENTINEL) {
            System.out.print("Enter a positive integer (or -1 to end): ");
            number = scanner.nextInt();

            if (number == -1) {
                break;
            }
            if (hasEight(number)) {
                sum += number;
            }
        }
        System.out.println("The magic sum is: " + sum);
    }

    public static boolean hasEight(int number) {
        // Convert the number to a string to check for the digit 8
        String numberAsString = Integer.toString(number);

        // Check if the string contains the character '8'
        return numberAsString.contains("8");
    }
}
```


Result

```
Enter a positive integer (or -1 to end): 5
Enter a positive integer (or -1 to end): 3
Enter a positive integer (or -1 to end): 9
Enter a positive integer (or -1 to end): 8
Enter a positive integer (or -1 to end): 4
Enter a positive integer (or -1 to end): 82
Enter a positive integer (or -1 to end): -1
The magic sum is: 90
```

9.4.4 print() (Array & Method)

Aim: Write a method called print(), which takes an int array and print its contents in the form of [a1, a2, ..., an]. Take note that there is no comma after the last element. The method's signature is as follows:

```
public static void print(int[] array);
```

Also write a test driver to test this method (you should test on empty array, one-element array, and n-element array). How to handle double[] or float[]? You need to write a overloaded version for double[] and a overloaded version for float[], with the following signatures:

```
public static void print(double[] array)
public static void print(float[] array)
```

The above is known as method overloading, where the same method name can have many versions, differentiated by its parameter list.

Hints: For the first element, print its value; for subsequent elements, print commas followed by the value.

Program

```
/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept of CSE,
 * IARE, Dundigal.
 */

public class ArrayPrinter {
    public static void main(String[] args) {
        // Test cases for int[]
        int[] intArrayEmpty = {};
        int[] intArrayOneElement = {42};
        int[] intArrayMultipleElements = {1, 2, 3, 4, 5};

        // Test cases for double[]
        double[] doubleArrayEmpty = {};
        double[] doubleArrayOneElement = {3.14};
        double[] doubleArrayMultipleElements = {1.0, 2.0, 3.0, 4.0, 5.0};

        // Test cases for float[]
        float[] floatArrayEmpty = {};
        float[] floatArrayOneElement = {3.14f};
        float[] floatArrayMultipleElements = {1.0f, 2.0f, 3.0f, 4.0f, 5.0f};

        System.out.print("intArrayEmpty: ");
        print(intArrayEmpty);
```

```

        System.out.print("intArrayOneElement: ");
        print(intArrayOneElement);

        System.out.print("intArrayMultipleElements: ");
        print(intArrayMultipleElements);

        System.out.print("doubleArrayEmpty: ");
        print(doubleArrayEmpty);

        System.out.print("doubleArrayOneElement: ");
        print(doubleArrayOneElement);

        System.out.print("doubleArrayMultipleElements: ");
        print(doubleArrayMultipleElements);

        System.out.print("floatArrayEmpty: ");
        print(floatArrayEmpty);

        System.out.print("floatArrayOneElement: ");
        print(floatArrayOneElement);

        System.out.print("floatArrayMultipleElements: ");
        print(floatArrayMultipleElements);
    }

    public static void print(int[] array) {
        System.out.print("[");
        for (int i = 0; i < array.length; i++) {
            System.out.print(array[i]);
            if (i < array.length - 1) {
                System.out.print(", ");
            }
        }
        System.out.println("]");
    }

    public static void print(double[] array) {
        System.out.print("[");
        for (int i = 0; i < array.length; i++) {
            System.out.print(array[i]);
            if (i < array.length - 1) {
                System.out.print(", ");
            }
        }
        System.out.println("]");
    }

    public static void print(float[] array) {
        System.out.print("[");
        for (int i = 0; i < array.length; i++) {
            System.out.print(array[i]);
            if (i < array.length - 1) {
                System.out.print(", ");
            }
        }
        System.out.println("]");
    }
}

```

```
}
```

Result

```
intArrayEmpty: []
intArrayOneElement: [42]
intArrayMultipleElements: [1, 2, 3, 4, 5]
doubleArrayEmpty: []
doubleArrayOneElement: [3.14]
doubleArrayMultipleElements: [1.0, 2.0, 3.0, 4.0, 5.0]
floatArrayEmpty: []
floatArrayOneElement: [3.14]
floatArrayMultipleElements: [1.0, 2.0, 3.0, 4.0, 5.0]
```

9.4.5 arrayToString() (Array & Method)

Aim: Write a method called `arrayToString()`, which takes an `int` array and return a `String` in the form of `[a1, a2, ..., an]`. Take note that this method returns a `String`, the previous exercise returns `void` but prints the output. The method's signature is as follows:

```
public static String arrayToString(int[] array);
```

Also write a test driver to test this method (you should test on empty array, one-element array, and n-element array).

Notes: This is similar to the built-in function `Arrays.toString()`. You could study its source code.

Program

```
/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept of CSE,
 * IARE, Dundigal.
 */

public class ArrayToStringConverter {
    public static void main(String[] args) {
        // Test cases for int[]
        int[] intArrayEmpty = {};
        int[] intArrayOneElement = {42};
        int[] intArrayMultipleElements = {1, 2, 3, 4, 5};

        System.out.println("intArrayEmpty: " + arrayToString(intArrayEmpty)
            );
        System.out.println("intArrayOneElement: " + arrayToString(
            intArrayOneElement));
        System.out.println("intArrayMultipleElements: " + arrayToString(
            intArrayMultipleElements));
    }

    public static String arrayToString(int[] array) {
        StringBuilder result = new StringBuilder("[");

        for (int i = 0; i < array.length; i++) {
            result.append(array[i]);

            if (i < array.length - 1) {
```

```

        result.append(", ");
    }
}
result.append("]");
return result.toString();
}
}

```

Result

intArrayEmpty: []

intArrayOneElement: [42]

intArrayMultipleElements: [1, 2, 3, 4, 5]

9.4.6 contains() (Array & Method)

Aim: Write a boolean method called contains(), which takes an array of int and an int; and returns true if the array contains the given int. The method's signature is as follows:

```
public static boolean contains(int[] array, int key);
```

Also write a test driver to test this method.

Program

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept of CSE,
 * IARE, Dundigal.
 */

public class ArrayContains {
    public static void main(String[] args) {
        // Test cases
        int[] array1 = {1, 2, 3, 4, 5};
        int key1 = 3;
        int key2 = 6;

        System.out.println("Array contains " + key1 + ": " + contains(
            array1, key1)); // Should print true
        System.out.println("Array contains " + key2 + ": " + contains(
            array1, key2)); // Should print false
    }

    public static boolean contains(int[] array, int key) {
        // Iterate through the array to check if it contains the key
        for (int value : array) {
            if (value == key) {
                return true;
            }
        }
        return false;
    }
}

```

Result

Array contains 3: true

Array contains 6: false

9.4.7 search() (Array & Method)

Aim: Write a method called search(), which takes an array of int and an int; and returns the array index if the array contains the given int; or -1 otherwise. The method's signature is as follows:

```
public static int search(int[] array, int key);
```

Also write a test driver to test this method.

Program

```
/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept of CSE,
 * IARE, Dundigal.
 */

public class ArraySearch {
    public static void main(String[] args) {
        // Test cases
        int[] array = {1, 3, 5, 7, 9, 11};
        int key1 = 5;
        int key2 = 8;

        int result1 = search(array, key1);
        int result2 = search(array, key2);

        System.out.println("Index of " + key1 + ": " + result1); // Should
            print "Index of 5: 2"
        System.out.println("Index of " + key2 + ": " + result2); // Should
            print "Index of 8: -1"
    }

    public static int search(int[] array, int key) {
        // Iterate through the array to find the index of the key
        for (int i = 0; i < array.length; i++) {
            if (array[i] == key) {
                return i; // Key found, return the index
            }
        }
        return -1; // Key not found, return -1
    }
}
```

Result

Index of 5: 2

Index of 8: -1

9.4.8 equals() (Array & Method)

Aim: Write a boolean method called equals(), which takes two arrays of int and returns true if the two arrays are exactly the same (i.e., same length and same contents). The method's signature is as follows:

```
public static boolean equals(int[] array1, int[] array2)
```

Also write a test driver to test this method.

Program

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept of CSE,
 * IARE, Dundigal.
 */

public class ArrayEquals {
    public static void main(String[] args) {
        // Test cases
        int[] array1 = {1, 2, 3, 4, 5};
        int[] array2 = {1, 2, 3, 4, 5};
        int[] array3 = {1, 2, 3};
        int[] array4 = {5, 4, 3, 2, 1};

        System.out.println("Arrays 1 and 2 are equal: " + equals(array1,
            array2)); // Should print true
        System.out.println("Arrays 1 and 3 are equal: " + equals(array1,
            array3)); // Should print false
        System.out.println("Arrays 1 and 4 are equal: " + equals(array1,
            array4)); // Should print false
    }

    public static boolean equals(int[] array1, int[] array2) {
        // Check if the arrays have the same length
        if (array1.length != array2.length) {
            return false;
        }

        // Check if the contents of the arrays are the same
        for (int i = 0; i < array1.length; i++) {
            if (array1[i] != array2[i]) {
                return false;
            }
        }
        return true;
    }
}

```

Result

Arrays 1 and 2 are equal: true

Arrays 1 and 3 are equal: false

Arrays 1 and 4 are equal: false

9.4.9 copyOf() (Array & Method)

Aim: Write a boolean method called `copyOf()`, which takes an `int` Array and returns a copy of the given array. The method's signature is as follows:

```
public static int[] copyOf(int[] array)
```

Also write a test driver to test this method. Write another version for `copyOf()` which takes a second parameter to specify the length of the new array. You should truncate or pad with zero so that the new array has the required length.

```
public static int[] copyOf(int[] array, int newLength)
```

NOTES: This is similar to the built-in function `Arrays.copyOf()`.

Program

```
/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept of CSE,
 * IARE, Dundigal.
 */
import java.util.Arrays;

public class ArrayCopy {
    public static void main(String[] args) {
        // Test cases
        int[] originalArray = {1, 2, 3, 4, 5};

        // Test the first version of copyOf
        int[] copiedArray1 = copyOf(originalArray);
        System.out.println("Original Array: " + Arrays.toString(
            originalArray));
        System.out.println("Copied Array 1: " + Arrays.toString(
            copiedArray1));

        // Test the second version of copyOf with a larger length
        int[] copiedArray2 = copyOf(originalArray, 7);
        System.out.println("Copied Array 2 (larger length): " + Arrays.
            toString(copiedArray2));

        // Test the second version of copyOf with a smaller length
        int[] copiedArray3 = copyOf(originalArray, 3);
        System.out.println("Copied Array 3 (smaller length): " + Arrays.
            toString(copiedArray3));
    }

    public static int[] copyOf(int[] array) {
        int[] copiedArray = new int[array.length];
        for(int i = 0; i < array.length; i++) {
            copiedArray[i] = array[i];
        }
        return copiedArray;
    }

    public static int[] copyOf(int[] array, int newLength) {
        int[] copiedArray = new int[newLength];
        for (int i = 0; i < newLength; i++) {
            if (i < array.length) {
                copiedArray[i] = array[i];
            } else {
                copiedArray[i] = 0; // Pad with zeros for elements beyond
                the original array's length
            }
        }
        return copiedArray;
    }
}
```

Result

Original Array: [1, 2, 3, 4, 5]
Copied Array 1: [1, 2, 3, 4, 5]
Copied Array 2 (larger length): [1, 2, 3, 4, 5, 0, 0]
Copied Array 3 (smaller length): [1, 2, 3]

9.4.10 swap() (Array & Method)

Aim: Write a method called swap(), which takes two arrays of int and swap their contents if they have the same length. It shall return true if the contents are successfully swapped. The method's signature is as follows:

```
public static boolean swap(int[] array1, int[] array2)
```

Also write a test driver to test this method.

Program

```
/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept of CSE,
 * IARE, Dundigal.
 */

public class ArraySwap {
    public static void main(String[] args) {
        // Test cases
        int[] array1 = {1, 2, 3};
        int[] array2 = {4, 5, 6};
        int[] array3 = {7, 8, 9, 10};
        int[] array4 = {11, 12, 13};

        boolean result1 = swap(array1, array2); // Should swap and return
        true
        boolean result2 = swap(array2, array3); // Should not swap and
        return false
        boolean result3 = swap(array3, array4); // Should not swap and
        return false

        System.out.println("Result of swapping array1 and array2: " +
            result1);
        System.out.println("Result of swapping array2 and array3: " +
            result2);
        System.out.println("Result of swapping array3 and array4: " +
            result3);

        System.out.println("array1 after swap: " + java.util.Arrays.
            toString(array1));
        System.out.println("array2 after swap: " + java.util.Arrays.
            toString(array2));
    }

    public static boolean swap(int[] array1, int[] array2) {
        // Check if the arrays have the same length
        if (array1.length != array2.length) {
            return false;
        }

        // Swap the contents of the arrays using a temporary variable
        for (int i = 0; i < array1.length; i++) {
            int temp = array1[i];
            array1[i] = array2[i];
            array2[i] = temp;
        }

        return true;
    }
}
```



```

    }
}

```

Result

Result of swapping array1 and array2: true
 Result of swapping array2 and array3: false
 Result of swapping array3 and array4: false
 array1 after swap: [4, 5, 6]
 array2 after swap: [1, 2, 3]

9.4.11 reverse() (Array & Method)

Aim: Write a method called reverse(), which takes an array of int and reverse its contents. For example, the reverse of [1,2,3,4] is [4,3,2,1]. The method's signature is as follows:

```
public static void reverse(int[] array)
```

Take note that the array passed into the method can be modified by the method (this is called "pass by reference"). On the other hand, primitives passed into a method cannot be modified. This is because a clone is created and passed into the method instead of the original copy (this is called "pass by value").

Also write a test driver to test this method.

Hints: You might use two indexes in the loop, one moving forward and one moving backward to point to the two elements to be swapped.

```

for (int fIdx = 0, bIdx = array.length - 1; fIdx < bIdx; ++fIdx, --bIdx) {
    // Swap array[fIdx] and array[bIdx]
    // Only need to transverse half of the array elements
}

```

You need to use a temporary location to swap two storage locations.

```

// Swap item1 and item2
int item1, item2, temp;
temp = item1;
item1 = item2;
item2 = item1;
// You CANNOT simply do: item1 = item2; item2 = item2;

```

Program

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept of CSE,
 * IARE, Dundigal.
 */

public class ArrayReverse {
    public static void main(String[] args) {
        // Test cases
        int[] array1 = {1, 2, 3, 4, 5};
        int[] array2 = {15, 14, 13, 12, 11};
    }
}

```

```

        System.out.println("Original Array 1: " + java.util.Arrays.toString
            (array1));
        reverse(array1);
        System.out.println("Reversed Array 1: " + java.util.Arrays.toString
            (array1));

        System.out.println("Original Array 2: " + java.util.Arrays.toString
            (array2));
        reverse(array2);
        System.out.println("Reversed Array 2: " + java.util.Arrays.toString
            (array2));
    }

    public static void reverse(int[] array) {
        for (int fIdx = 0, bIdx = array.length - 1; fIdx < bIdx; ++fIdx, --
            bIdx) {
            // Swap array[fIdx] and array[bIdx]
            int temp = array[fIdx];
            array[fIdx] = array[bIdx];
            array[bIdx] = temp;
        }
    }
}

```

Result

Original Array 1: [1, 2, 3, 4, 5]

Reversed Array 1: [5, 4, 3, 2, 1]

Original Array 2: [15, 14, 13, 12, 11]

Reversed Array 2: [11, 12, 13, 14, 15]

9.4.12 GradesStatistics (Array & Method)

Aim: Write a program called GradesStatistics, which reads in n grades (of int between 0 and 100, inclusive) and displays the average, minimum, maximum, median and standard deviation. Display the floating-point values upto 2 decimal places.

Program

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept of CSE,
 * IARE, Dundigal.
 */

import java.util.Scanner;
import java.util.Arrays;

public class GradesStatistics {
    public static int[] grades; // Declare an int[], to be allocated later
    .
    // This array is accessible by all the methods.

    public static void main(String[] args) {
        readGrades(); // Read and save the inputs in global int[] grades
        System.out.print("The grades are: ");
        print(grades);
        System.out.printf("The average is %.2f", average(grades));
    }
}

```

```

        System.out.printf("\nThe median is %.2f", median(grades));
        System.out.println("\nThe minimum is " + min(grades));
        System.out.println("The maximum is " + max(grades));
        System.out.printf("The standard deviation is %.2f", stdDev(grades))
            ;
    }

    // Prompt user for the number of students and allocate the global "
    // grades" array.
    // Then, prompt user for grade, check for valid grade, and store in "
    // grades".
    public static void readGrades() {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number of students: ");
        int n = sc.nextInt();
        grades = new int[n];
        for(int i = 0; i<n; i++){
            System.out.print("Enter the grade for student "+(i+1)+" : ");
            grades[i] = sc.nextInt();
        }
    }

    // Print the given int array in the form of [x1, x2, x3,..., xn].
    public static void print(int[] array) {
        System.out.print("[");
        for (int i = 0; i < grades.length; i++) {
            System.out.print(grades[i]);
            if (i < array.length - 1) {
                System.out.print(", ");
            }
        }
        System.out.println("]");
    }

    // Return the average value of the given int[]
    public static double average(int[] array){
        int sum = 0;
        for (int num : array) {
            sum += num;
        }
        return (double) sum / array.length;
    }

    // Return the median value of the given int[]
    // Use Arrays.sort(anArray) to sort anArray in place.
    public static double median(int[] array){
        Arrays.sort(array);
        int middle = array.length / 2;
        if (array.length % 2 == 0) {
            // If even number of elements, take the average of the two
            // middle values
            double middle1 = array[middle - 1];
            double middle2 = array[middle];
            return (middle1 + middle2) / 2.0;
        } else {
            // If odd number of elements, return the middle value
            return array[middle];
        }
    }

```

```

    }

    // Return the maximum value of the given int[]
    public static int max(int[] array) {
        int max = array[0];    // Assume that max is the first element
        for (int num : array) {
            if (num > max)
                max = num;
        }
        return max;
    }

    // Return the minimum value of the given int[]
    public static int min(int[] array){
        int min = array[0];
        for (int num : array) {
            if (num < min)
                min = num;
        }
        return min;
    }

    // Return the standard deviation of the given int[]
    public static double stdDev(int[] array) {
        double mean = average(array);
        double sumOfSquares = 0.0;

        for (int num : array) {
            sumOfSquares += Math.pow(num - mean, 2);
        }
        double variance = sumOfSquares / array.length;
        return Math.sqrt(variance);
    }
}

```

Result

```

Enter the number of students: 5
Enter the grade for student 1: 56
Enter the grade for student 2: 46
Enter the grade for student 3: 78
Enter the grade for student 4: 64
Enter the grade for student 5: 83
The grades are: [56, 46, 78, 64, 83]
The average is 65.40
The median is 64.00
The minimum is 46
The maximum is 83
The standard deviation is 13.68

```

9.4.13 GradesHistogram (Array & Method)

Aim: Write a program called GradesHistogram, which reads in n grades (as in the previous exercise), and displays the horizontal and vertical histograms.

Program

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept of CSE,
 * IARE, Dundigal.
 */

import java.util.Scanner;

public class GradesHistogram2 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Prompt for the number of grades
        System.out.print("Enter the number of grades: ");
        int n = scanner.nextInt();

        // Initialize an array to store grades
        int[] grades = new int[n];

        // Read grades from the user
        for (int i = 0; i < n; i++) {
            System.out.print("Enter grade " + (i + 1) + ": ");
            grades[i] = scanner.nextInt();
        }

        // Display horizontal histogram
        System.out.println("Horizontal Histogram:");
        displayHorizontalHistogram(grades);

        // Display vertical histogram
        System.out.println("\nVertical Histogram:");
        displayVerticalHistogram(grades);
    }

    public static void displayHorizontalHistogram(int[] grades) {
        int n = 0;
        System.out.print(" ");
        for (int grade : grades) {
            System.out.print(n + "-" + (n+10) + ": ");
            for (int i = 0; i < grade; i++) {
                System.out.print("*");
            }
            System.out.println();
            n += 10;
        }
    }

    public static void displayVerticalHistogram(int[] grades) {
        int maxGrade = findMaxGrade(grades);

        for (int row = maxGrade; row >= 1; row--) {
            for (int grade : grades) {
                if (grade >= row) {
                    System.out.print(" *\t ");
                } else {
                    System.out.print(" \t ");
                }
            }
        }
    }
}

```

```

        System.out.println();
    }

    // Display grade labels at the bottom
    int n = 0;
    for (int grade : grades) {
        System.out.print(n + "-" + (n+10) + "\t");
        //System.out.print(grade + " ");
        n += 10;
    }
    System.out.println();
}

public static int findMaxGrade(int[] grades) {
    int maxGrade = Integer.MIN_VALUE;
    for (int grade : grades) {
        if (grade > maxGrade) {
            maxGrade = grade;
        }
    }
    return maxGrade;
}
}

```

Result

```

Enter the number of grades: 5
Enter grade 1: 4
Enter grade 2: 2
Enter grade 3: 0
Enter grade 4: 1
Enter grade 5: 5
Horizontal Histogram:
0-10: *****
10-20: **
20-30:
30-40: *
40-50: *****

Vertical Histogram:

          *
*          *
*          *
*      *   *
*      *   *
0-10  10-20  20-30  30-40  40-50

```

9.5 Sample Viva Voce Questions

1. What is the purpose of a method's body, and what type of code does it contain?
2. Can a Java program contain multiple methods, and can they have the same name?
3. What is method overloading, and how is it achieved in Java?
4. What is the primary advantage of breaking down a program into smaller methods?

9.6 Further Probing Experiments

1. Given string `str`, the task is to write a method to swap the pairs of characters of a string. If the string contains an odd number of characters then the last character remains as it is. Write driver code to test the method
2. Given a string `str`, consisting of non-alphabetical characters. The task is to develop a method to remove all those non-alphabetical characters of `str` and print the words on a new line.

EXERCISES ON COMMAND-LINE ARGUMENTS, RECURSION

10.1 Introduction

Command Line Arguments and Recursion stand out as pivotal pillars for aspiring Java programmers, offering essential tools for tackling complex challenges and crafting efficient software solutions. Command Line Arguments facilitate dynamic interaction with Java programs, enabling the provision of external input and parameters without recompilation, while Recursion empowers programmers to tackle intricate problems by breaking them down into manageable subproblems, fostering elegant and concise code solutions. Mastery of these concepts not only enhances problem-solving skills but also cultivates a deeper understanding of the versatility and sophistication of the Java language, laying a robust foundation for further exploration and advancement in Java programming and beyond.

10.2 Prelab Preparation

10.2.1 Objective

By the end of this lab, students should be able to:

1. **Familiarization with Command Line Arguments:** Introduce students to the concept of command line arguments in Java, enabling them to understand how external input can be passed to Java programs during runtime.
2. **Hands-on Practice with Command Line Arguments:** Provide practical exercises to students to manipulate and process command line arguments within Java programs, reinforcing their understanding through implementation.
3. **Understanding Recursion Principles:** Educate students on the principles and characteristics of recursion in programming, including the concept of base cases, recursive calls, and stack memory management.
4. **Application of Recursion in Problem Solving:** Engage students in solving various problems using recursive algorithms, encouraging them to break down complex problems into smaller subproblems and develop recursive solutions.
5. **Comparative Analysis of Recursive and Iterative Approaches:** Guide students in comparing recursive and iterative solutions to common problems, highlighting the advantages and limitations of recursion and fostering critical thinking about algorithm design and efficiency.

10.2.2 Background

Command Line Arguments: They are strings passed to the program after its name and can be accessed within the Java program using the args parameter in the main method. Command

line arguments allow users to customize the behavior of the program at runtime by providing input data, configuration parameters, or options without the need for recompilation.

Syntax: Command line arguments are passed to a Java program as strings in the main method's args parameter.

```
public class Main {
    public static void main(String[] args) {
        // Access command line arguments here
    }
}
```

Accessing Command Line Arguments: Command line arguments can be accessed inside the main method using the args array.

```
public class Main {
    public static void main(String[] args) {
        // Accessing command line arguments
        for (String arg : args) {
            System.out.println(arg);
        }
    }
}
```

Usage Tips:

1. Command line arguments are separated by spaces.
2. Each argument is treated as a separate string.
3. Quotation marks can be used to include spaces within an argument.
4. Command line arguments are passed as strings, so they need to be parsed if they represent numerical or other non-string values.
5. Command line arguments provide a convenient way to customize the behavior of Java programs at runtime, making them more flexible and adaptable to different scenarios.
6. They are commonly used in various applications, such as command-line utilities, batch processing scripts, and application configuration.

Recursion: Recursion is a programming technique in which a function or method calls itself directly or indirectly to solve a problem by breaking it down into smaller, similar subproblems. It involves solving a problem by reducing it to simpler instances of the same problem.

When implementing recursion, it is essential to consider the following:

1. **Base Case:** Define a condition that serves as the stopping point for the recursive calls.
2. **Recursive Step:** Break down the problem into smaller subproblems and make a recursive call to solve them.
3. **Termination:** Ensure that the recursive calls eventually lead to the base case to prevent infinite recursion.

Syntax

```
public class MyClass {
    public static void recursiveMethod(parameters) {
        // Base case - condition that stops the recursion
        if (base case condition) {
            // Handle base case
        } else {
            // Recursive step - call the method recursively with
            // modified parameters
            recursiveMethod(modified parameters);
        }
    }

    public static void main(String[] args) {
        // Call the recursive method
        recursiveMethod(initial parameters);
    }
}
```

10.2.3 Hardware/Software Requirements

To successfully complete this lab, students will need access to the following:

1. A computer with a suitable text editor or Integrated Development Environment (IDE) installed, such as Eclipse, IntelliJ IDEA, or Visual Studio Code.
2. Java Development Kit (JDK) installed on the computer. Ensure that the JDK is correctly configured, and the system's PATH variable is set to include the JDK's bin directory.

10.2.4 Pre-Lab Questions

1. What is the purpose of command line arguments?
2. How can command line arguments be accessed within a Java program?
3. What is recursion in programming?
4. How does recursion differ from iteration?
5. What are the two essential components of a recursive function or method?

10.3 Experiment Procedure

1. Setting Up:

- Open IDE or text editor.
- Create Java project/file.

2. Experiment 1: Command Line Arguments

- Write Java program with command line arguments.
- Access/process arguments.
- Compile and execute program with different arguments.

3. Experiment 2: Recursive Algorithms

- Choose problem(s) for recursive solution.
- Design recursive algorithms.
- Implement algorithms as Java methods.
- Test methods with various input values.
- Compare recursive solutions with iterative counterparts.

4. Conclusion

- Summarize key insights from experiments.
- Discuss advantages, limitations, and best practices.

Note: Document observations, code, and analysis for reference.

10.4 Experiemnts

10.4.1 Arithmetic (Command-Line Arguments)

Aim: Write a program called Arithmetic that takes three command-line arguments: two integers followed by an arithmetic operator (+, -, * or /). The program shall perform the corresponding operation on the two integers and print the result. For example:

Program

```
/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

public class JavaCMLArg {

    public static void main(String[] args) {
        int operand1, operand2;
        char theOperator;

        // Check if there are 3 command-line arguments in the
        // String[] args by using length variable of array.
        if (args.length != 3) {
            System.err.println("Usage: java Arithmetic int1 int2 op");
            return;
        }

        // Convert the 3 Strings args[0], args[1], args[2] to int and
        // char.
        // Use the Integer.parseInt(aStr) to convert a String to an int
        .
        operand1 = Integer.parseInt(args[0]);
        operand2 = Integer.parseInt(args[1]);

        // Get the operator, assumed to be the first character of
        // the 3rd string. Use method charAt() of String.
        theOperator = args[2].charAt(0);
        System.out.print(args[0] + args[2] + args[1] + "=");

        switch(theOperator) {
            case ('-') -> System.out.println(operand1 - operand2);
```

```

        case ('+') -> System.out.println(operand1 + operand2);
        case ('*') -> System.out.println(operand1 * operand2);
        case ('/') -> {
            // Check for division by zero
            if (operand2 == 0) {
                System.err.println("Error: Division by zero!");
                return;
            }
            System.out.println(operand1 / operand2);
        }
        default -> System.err.println("Error: Invalid operator!");
    }
}
}

```

Result

12345+4567=16912

10.4.2 FactorialRecursive

Aim: In programming, a recursive function (or method) calls itself. The classical example is factorial(n), which can be defined recursively as $f(n)=n*f(n-1)$. Nonetheless, it is important to take note that a recursive function should have a terminating condition (or base case), in the case of factorial, $f(0)=1$. Hence, the full definition is:

factorial(n) = 1, for n = 0
factorial(n) = n * factorial(n-1), for all n $\geq$ 1

Program

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

import java.util.Scanner;

public class Factorial {

    // Recursive method to calculate factorial
    public static int factorial(int n) {
        // Base case: factorial of 0 is 1
        if (n == 0) {
            return 1;
        }
        // Recursive step: n * factorial(n-1)
        return n * factorial(n - 1);
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter a number to find its factorial: ");
        int number = scanner.nextInt();

        // Check if the number is negative
        if (number < 0) {

```

```

        System.out.println("Factorial is not defined for negative
            numbers.");
        return;
    }

    // Call the factorial method and display the result
    int result = factorial(number);
    System.out.println("Factorial of " + number + " is: " + result)
        ;

    scanner.close();
}
}

```

Result

Testcase 1

Enter a number to find its factorial: 5
 Factorial of 5 is: 120

Testcase 2

Enter a number to find its factorial: 0
 Factorial of 0 is: 1

Testcase 3

Enter a number to find its factorial: -2
 Factorial is not defined for negative numbers.

10.4.3 Fibonacci (Recursive)

Aim: Write a recursive method to compute the Fibonacci number of n, defined as follows:

$F(0) = 0$

$F(1) = 1$

$F(n) = F(n-1) + F(n-2)$ for $n \geq 2$

Program

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */
import java.util.Scanner;

public class Fibonacci {

    // Recursive method to compute the Fibonacci number of n
    public static int fibonacci(int n) {
        // Base case: Fibonacci of 0 and 1 is 1
        if (n == 0 || n == 1) {
            return 1;
        }
        // Recursive step: Fibonacci(n) = Fibonacci(n-1) + Fibonacci(n-2)
        return fibonacci(n - 1) + fibonacci(n - 2);
    }
}

```

```

    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter the value of n to compute Fibonacci
            number: ");
        int n = scanner.nextInt();

        // Check if n is non-negative
        if (n < 0) {
            System.out.println("Fibonacci number is not defined for
                negative values of n.");
            return;
        }

        // Call the fibonacci method and display the result
        int result = fibonacci(n);
        System.out.println("Fibonacci number of " + n + " is: " +
            result);

        scanner.close();
    }
}

```

Result

Enter the value of n to compute Fibonacci number: 5

Fibonacci number of 5 is: 8

10.4.4 Length of a Running Number Sequence (Recursive)

Aim: A special number sequence is defined as follows:

$S(1) = 1$

$S(2) = 12$

$S(3) = 123$

$S(4) = 1234$

..... $S(9) = 123456789$ // length is 9

$S(10) = 12345678910$ // length is 11

$S(11) = 1234567891011$ // length is 13

$S(12) = 123456789101112$ // length is 15

.....

Write a recursive method to compute the length of $S(n)$, defined as follows:

$len(1) = 1$

$len(n) = len(n-1) + numOfDigits(n)$

Program

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

import java.util.Scanner;

```

```

public class SpecialNumberSeq {

    // Recursive method to compute the length of S(n)
    public static int lengthOfSpecialSequence(int n) {
        // Base case: length of S(1) is 1
        if (n == 1) {
            return 1;
        }
        // Recursive step: length(n) = length(n-1) + numOfDigits(n)
        return lengthOfSpecialSequence(n - 1) + numOfDigits(n);
    }

    // Method to count the number of digits in a number
    public static int numOfDigits(int num) {
        if (num < 10) {
            return 1;
        }
        return 1 + numOfDigits(num / 10);
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter the value of n to compute the length of
            S(n): ");
        int n = scanner.nextInt();

        // Check if n is positive
        if (n <= 0) {
            System.out.println("Invalid input. n must be a positive
                integer.");
            return;
        }

        // Call the lengthOfSpecialSequence method and display the
        // result
        int result = lengthOfSpecialSequence(n);
        System.out.println("Length of S(" + n + ") is: " + result);

        scanner.close();
    }
}

```

Result

Testcase 1

Enter the value of n to compute the length of S(n): 8

Length of S(8) is: 8

Testcase 2

Enter the value of n to compute the length of S(n): 12

Length of S(12) is: 15

10.4.5 GCD (Recursive)

Aim: Write a recursive method called gcd() to compute the greatest common divisor of two given integers.

```
public static void int gcd(int a, int b)
```

gcd(a,b) = a, if b = 0

gcd(a,b) = gcd(b, remainder(a,b)), if b $\neq$ 0

Program

```
/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

import java.util.Scanner;

public class GCDCalculator {

    // Recursive method to compute GCD
    public static int gcd(int a, int b) {
        if (b == 0) {
            return Math.abs(a); // Handle negative inputs safely
        } else {
            return gcd(b, a % b);
        }
    }

    // Main method
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Taking input from the user
        System.out.print("Enter the first number: ");
        int num1 = scanner.nextInt();

        System.out.print("Enter the second number: ");
        int num2 = scanner.nextInt();

        int result = gcd(num1, num2);

        System.out.println("GCD of " + num1 + " and " + num2 + " is: "
            + result);

        scanner.close();
    }
}
```

Result

Testcase 1

Enter the first number: 6

Enter the second number: 7

GCD of 6 and 7 is: 1

Testcase 2

Enter the first number: 6

Enter the second number: 4

GCD of 6 and 4 is: 2

10.5 Sample Viva Voce Questions

1. What is the significance of the args parameter in the main method?
2. Can command line arguments be modified or manipulated within a Java program?
3. How do you ensure that a recursive function does not result in infinite recursion?
4. Can you provide an example of a simple recursive function in Java?
5. What are some common problems that are well-suited for recursion?

10.6 Further Probing Experiments

1. You are climbing a staircase. It takes n steps to reach the top.

Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

MORE (DIFFICULT) EXERCISES

11.1 Introduction

This lab is designed to provide you with hands-on experience in Java programming through a series of guided exercises. Java is a versatile and widely-used programming language known for its portability, security features, and object-oriented programming paradigm. Whether you're a beginner looking to learn the basics of Java or an experienced programmer seeking to enhance your skills, this lab offers an opportunity to deepen your understanding of Java programming concepts and techniques.

11.2 Prelab Preparation

11.2.1 Objective

By the end of this lab, students should be able to:

1. Familiarize with fundamental Java programming concepts such as variables, data types, operators, control structures, and methods.
2. Apply object-oriented programming principles in Java, including encapsulation, inheritance, and polymorphism.
3. Solve problems related to dates, Pascal's Triangle, and guessing games using Java programming constructs.
4. Enhance their ability to design, implement, test, and debug Java programs effectively.
5. Cultivate critical thinking and problem-solving skills by encouraging you to analyze problems, formulate algorithms, and implement solutions in Java.

11.2.2 Background

ChatGPT Background:

In this lab, we will build upon the foundational Java programming concepts covered in earlier labs and apply them to solve problems related to trigonometric series, Pascal's Triangle, patterns, utility dates, and guessing numbers. These problem domains require a deeper understanding of Java programming techniques and algorithms to effectively implement solutions.

Trigonometric Series: Trigonometric series involve mathematical functions such as sine, cosine, and tangent, which can be represented as infinite series. Implementing algorithms to calculate and manipulate trigonometric series requires a strong understanding of mathematical concepts and their translation into Java programming logic.

Pascal's Triangle: Pascal's Triangle is a triangular array of binomial coefficients, where each number is the sum of the two directly above it. Solving problems related to Pascal's

Triangle involves understanding recursive algorithms, dynamic programming techniques, and two-dimensional array manipulation in Java.

Patterns: Creating and manipulating patterns involves generating sequences of characters or numbers that follow a specific rule or structure. Implementing algorithms to generate patterns requires knowledge of loops, conditional statements, and string manipulation in Java.

Utility Dates: Utility dates involve working with dates and performing operations such as validation, calculation of leap years, determination of the day of the week, and formatting dates. Solving problems related to utility dates requires familiarity with Java's date and time APIs, as well as modular arithmetic concepts.

Guess Numbers: Guessing number games involve generating random numbers, accepting user input, and providing feedback based on the user's guesses. Implementing guessing number games requires knowledge of random number generation, user input handling, conditional statements, and loop structures in Java.

By applying the concepts learned in earlier labs to these specific problem domains, you will deepen your understanding of Java programming and enhance your problem-solving skills. Throughout the lab, you will have the opportunity to practice algorithmic thinking, develop efficient Java programs, and gain hands-on experience in tackling a variety of programming challenges.

11.2.3 Hardware/Software Requirements

To successfully complete this lab, students will need access to the following:

1. A computer with a suitable text editor or Integrated Development Environment (IDE) installed, such as Eclipse, IntelliJ IDEA, or Visual Studio Code.
2. Java Development Kit (JDK) installed on the computer. Ensure that the JDK is correctly configured, and the system's PATH variable is set to include the JDK's bin directory.

11.2.4 Pre-Lab Questions

1. How do you compute the value of each number in Pascal's Triangle?
2. How do you calculate the value of a trigonometric series for a given input angle?
3. What is a leap year, and how do you determine if a year is a leap year?
4. How would you format a date to display it in a specific format in Java?

11.3 Experiment Procedure

1. Study each concept thoroughly.
2. Write Java methods based on the studied concepts.
3. Test the implemented methods with relevant test cases.
4. Explore variations in implementations or input scenarios.
5. Record experiment procedures and observations for documentation.

11.4 Experiemnts

11.4.1 JDK Source Code

Aim: Extract the source code of the class Math from the JDK source code. Study how constants such as E and PI are defined. Also study how methods such as abs(), max(), min(), toDegree(), etc, are written.

Also study the "Integer.java", "String.java".

Program

```
/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

public class JavaBuiltInMethods {
    public static void main(String[] args) {
        // Demonstrate usage of Math class methods
        double x = 3.5;
        double y = 2.0;

        System.out.println("Math.abs(-10.5) = " + Math.abs(-10.5)); //
            Absolute value
        System.out.println("Math.max(3, 5) = " + Math.max(3, 5)); //
            Maximum of two values
        System.out.println("Math.min(3, 5) = " + Math.min(3, 5)); //
            Minimum of two values
        System.out.println("Math.ceil(3.5) = " + Math.ceil(x)); //
            Round up to the nearest integer
        System.out.println("Math.floor(3.5) = " + Math.floor(x)); //
            Round down to the nearest integer
        System.out.println("Math.pow(2, 3) = " + Math.pow(2, 3)); //
            Power function
        System.out.println("Math.sqrt(25) = " + Math.sqrt(25)); //
            Square root
        System.out.println("Math.sin(0) = " + Math.sin(0)); // Sine
            function
        System.out.println("Math.cos(0) = " + Math.cos(0)); // Cosine
            function
        System.out.println("Math.tan(0) = " + Math.tan(0)); // Tangent
            function
        System.out.println("Math.log(Math.E) = " + Math.log(Math.E));
            // Natural logarithm
        System.out.println("Math.log10(100) = " + Math.log10(100)); //
            Base 10 logarithm
        System.out.println("Math.PI = " + Math.PI); // Value of PI
        System.out.println("Math.E = " + Math.E); // Value of E (Euler'
            s number)

        // Demonstrate usage of Integer class methods
        int intValue = 123;
        String strValue = "456";

        System.out.println("Integer.parseInt(\"456\") = " + Integer.
            parseInt(strValue)); // Parse string to integer
        System.out.println("Integer.toString(123) = " + Integer.
```

```

        toString(intValue)); // Convert integer to string

// Demonstrate usage of String class methods
String str1 = "Hello";
String str2 = "World";

System.out.println("str1.length() = " + str1.length()); //
    Length of the string
System.out.println("str1.concat(\" World\") = " + str1.concat("
    World")); // Concatenation
System.out.println("str1.charAt(0) = " + str1.charAt(0)); //
    Character at index
System.out.println("str1.toUpperCase() = " + str1.toUpperCase()
    ); // Convert to upper case
System.out.println("str1.toLowerCase() = " + str1.toLowerCase()
    ); // Convert to lower case
System.out.println("str1.equals(\"Hello\") = " + str1.equals("
    Hello")); // Compare strings
System.out.println("str1.indexOf(\"l\") = " + str1.indexOf("l")
    ); // Index of substring
System.out.println("str1.substring(1, 3) = " + str1.substring
    (1, 3)); // Substring
    }
}

```

Result

```

Math.abs(-10.5) = 10.5
Math.max(3, 5) = 5
Math.min(3, 5) = 3
Math.ceil(3.5) = 4.0
Math.floor(3.5) = 3.0
Math.pow(2, 3) = 8.0
Math.sqrt(25) = 5.0
Math.sin(0) = 0.0
Math.cos(0) = 1.0
Math.tan(0) = 0.0
Math.log(Math.E) = 1.0
Math.log10(100) = 2.0
Math.PI = 3.141592653589793
Math.E = 2.718281828459045
Integer.parseInt("456") = 456
Integer.toString(123) = 123
str1.length() = 5
str1.concat(" World") = Hello World
str1.charAt(0) = H
str1.toUpperCase() = HELLO
str1.toLowerCase() = hello
str1.equals("Hello") = true
str1.indexOf("l") = 2
str1.substring(1, 3) = el

```

11.4.2 Matrices (2D Arrays)

Aim: Similar to Math class, write a Matrix library that supports matrix operations (such as addition, subtraction, multiplication) via 2D arrays. The operations shall support both double and int. Also write a test class to exercise all the operations programmed.

Hints:

```
public class Matrix {
    // Method signatures
    public static void print(int[][] m);
    public static void print(double[][] m);
    public static boolean haveSameDimension(int[][] m1, int[][] m2);
    // Used in add(), subtract()
    public static boolean haveSameDimension(double[][] m1, double[][]
        m2);
    public static int[][] add(int[][] m1, int[][] m2);
    public static double[][] add(double[][] m1, double[][] m2);
    public static int[][] subtract(int[][] m1, int[][] m2);
    public static double[][] subtract(double[][] m1, double[][] m2);
    public static int[][] multiply(int[][] m1, int[][] m2);
    public static double[][] multiply(double[][] m1, double[][] m2);
    .....
}
```

Program

Matrix.java

```
/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

public class Matrix {

    public static void print(int[][] m) {
        for (int[] row : m) {
            for (int value : row) {
                System.out.print(value + " ");
            }
            System.out.println();
        }
    }

    public static void print(double[][] m) {
        for (double[] row : m) {
            for (double value : row) {
                System.out.print(value + " ");
            }
            System.out.println();
        }
    }

    public static boolean haveSameDimension(int[][] m1, int[][] m2) {
        return m1.length == m2.length && m1[0].length == m2[0].length;
    }

    public static boolean haveSameDimension(double[][] m1, double[][]
        m2) {
```

```

        return m1.length == m2.length && m1[0].length == m2[0].length;
    }

    public static int[][] add(int[][] m1, int[][] m2) {
        if (!haveSameDimension(m1, m2)) {
            throw new IllegalArgumentException("Matrices must have the
                same dimensions for addition.");
        }
        int[][] result = new int[m1.length][m1[0].length];
        for (int i = 0; i < m1.length; i++) {
            for (int j = 0; j < m1[0].length; j++) {
                result[i][j] = m1[i][j] + m2[i][j];
            }
        }
        return result;
    }

    public static double[][] add(double[][] m1, double[][] m2) {
        if (!haveSameDimension(m1, m2)) {
            throw new IllegalArgumentException("Matrices must have the
                same dimensions for addition.");
        }
        double[][] result = new double[m1.length][m1[0].length];
        for (int i = 0; i < m1.length; i++) {
            for (int j = 0; j < m1[0].length; j++) {
                result[i][j] = m1[i][j] + m2[i][j];
            }
        }
        return result;
    }

    public static int[][] subtract(int[][] m1, int[][] m2) {
        if (!haveSameDimension(m1, m2)) {
            throw new IllegalArgumentException("Matrices must have the
                same dimensions for subtraction.");
        }
        int[][] result = new int[m1.length][m1[0].length];
        for (int i = 0; i < m1.length; i++) {
            for (int j = 0; j < m1[0].length; j++) {
                result[i][j] = m1[i][j] - m2[i][j];
            }
        }
        return result;
    }

    public static double[][] subtract(double[][] m1, double[][] m2) {
        if (!haveSameDimension(m1, m2)) {
            throw new IllegalArgumentException("Matrices must have the
                same dimensions for subtraction.");
        }
        double[][] result = new double[m1.length][m1[0].length];
        for (int i = 0; i < m1.length; i++) {
            for (int j = 0; j < m1[0].length; j++) {
                result[i][j] = m1[i][j] - m2[i][j];
            }
        }
        return result;
    }
}

```

```

public static int[][] multiply(int[][] m1, int[][] m2) {
    if (m1[0].length != m2.length) {
        throw new IllegalArgumentException("Number of columns of
            the first matrix must be equal to the number of rows of
            the second matrix for multiplication.");
    }
    int[][] result = new int[m1.length][m2[0].length];
    for (int i = 0; i < m1.length; i++) {
        for (int j = 0; j < m2[0].length; j++) {
            for (int k = 0; k < m2.length; k++) {
                result[i][j] += m1[i][k] * m2[k][j];
            }
        }
    }
    return result;
}

public static double[][] multiply(double[][] m1, double[][] m2) {
    if (m1[0].length != m2.length) {
        throw new IllegalArgumentException("Number of columns of
            the first matrix must be equal to the number of rows of
            the second matrix for multiplication.");
    }
    double[][] result = new double[m1.length][m2[0].length];
    for (int i = 0; i < m1.length; i++) {
        for (int j = 0; j < m2[0].length; j++) {
            for (int k = 0; k < m2.length; k++) {
                result[i][j] += m1[i][k] * m2[k][j];
            }
        }
    }
    return result;
}
}

```

MatrixTest.java

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

public class MatrixTest {
    public static void main(String[] args) {
        int[][] matrix1 = {
            {1, 2, 3},
            {4, 5, 6},
            {7, 8, 9}
        };

        int[][] matrix2 = {
            {9, 8, 7},
            {6, 5, 4},
            {3, 2, 1}
        };

        double[][] matrix3 = {

```



```

        {1.5, 2.5},
        {3.5, 4.5}
    };

    double[][] matrix4 = {
        {2.5, 3.5},
        {4.5, 5.5}
    };

    System.out.println("Matrix 1:");
    printMatrix(matrix1);

    System.out.println("Matrix 2:");
    printMatrix(matrix2);

    System.out.println("Matrix 3:");
    printMatrix(matrix3);

    System.out.println("Matrix 4:");
    printMatrix(matrix4);

    System.out.println("Addition of Matrix 1 and Matrix 2:");
    int[][] resultInt = Matrix.add(matrix1, matrix2);
    printMatrix(resultInt);

    System.out.println("Subtraction of Matrix 1 and Matrix 2:");
    resultInt = Matrix.subtract(matrix1, matrix2);
    printMatrix(resultInt);

    System.out.println("Multiplication of Matrix 1 and Matrix 2:");
    resultInt = Matrix.multiply(matrix1, matrix2);
    printMatrix(resultInt);

    System.out.println("Addition of Matrix 3 and Matrix 4:");
    double[][] resultDouble = Matrix.add(matrix3, matrix4);
    printMatrix(resultDouble);

    System.out.println("Subtraction of Matrix 3 and Matrix 4:");
    resultDouble = Matrix.subtract(matrix3, matrix4);
    printMatrix(resultDouble);

    System.out.println("Multiplication of Matrix 3 and Matrix 4:");
    resultDouble = Matrix.multiply(matrix3, matrix4);
    printMatrix(resultDouble);
}

private static void printMatrix(int[][] matrix) {
    for (int[] row : matrix) {
        for (int value : row) {
            System.out.printf("%4d ", value);
        }
        System.out.println();
    }
    System.out.println();
}

private static void printMatrix(double[][] matrix) {
    for (double[] row : matrix) {

```

```

        for (double value : row) {
            System.out.printf("%6.2f ", value);
        }
        System.out.println();
    }
    System.out.println();
}

```

Result

Matrix 1:

```

1 2 3
4 5 6
7 8 9

```

Matrix 2:

```

9 8 7
6 5 4
3 2 1

```

Matrix 3:

```

1.50 2.50
3.50 4.50

```

Matrix 4:

```

2.50 3.50
4.50 5.50

```

Addition of Matrix 1 and Matrix 2:

```

10 10 10
10 10 10
10 10 10

```

Subtraction of Matrix 1 and Matrix 2:

```

-8 -6 -4
-2 0 2
4 6 8

```

Multiplication of Matrix 1 and Matrix 2:

```

30 24 18
84 69 54
138 114 90

```

Addition of Matrix 3 and Matrix 4:

```

4.00 6.00
8.00 10.00

```

Subtraction of Matrix 3 and Matrix 4:

```

-1.00 -1.00
-1.00 -1.00

```

Multiplication of Matrix 3 and Matrix 4:
 15.00 19.00
 29.00 37.00

11.4.3 PrintAnimalPattern (Special Characters and Escape Sequences)

Aim: Write a program called PrintAnimalPattern, which uses println() to produce this pattern

```

      ' _ '
      (OO)

 /===== \
/  ||  %%  ||
*  ||----||
  ¥¥      ¥¥
  ""      ""
  
```

Program

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

import java.io.PrintStream;
import java.io.UnsupportedEncodingException;

public class PrintAnimalPattern {
    public static void main(String[] args) {

        try {
            PrintStream out = new PrintStream(System.out, true, "UTF
            -8");

            // Printing pattern using println()
            out.println("      ' _ '");
            out.println("      (\u00A9\u00A9)");
            out.println(" /===== \");
            out.println(" /  ||  %%  ||");
            out.println("*  ||----||");
            out.println("  \u00A5\u00A5      \u00A5\u00A5");
            out.println("  \"\"      \"");

        } catch (UnsupportedEncodingException e) {
            System.out.println("Something went wrong");
        }

    }
}
  
```

Result

```

      ' _ '
      (OO)
/=====\\
/  ||  %%  ||
*  ||----||
  ¥¥      ¥¥
  ""      ""

```

11.4.4 Print Patterns (nested-loop)

Aim: Write a method to print each of the followings patterns using nested loops in a class called PrintPatterns. The program shall prompt user for the size of the pattern.

Program

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

public class Patterns {
    public static void main(String[] args) {
        patternA();
        patternB();
        patternC();
        patternD();
        patternE();
        patternF();
        patternG();
        patternH();
        patternI();
        patternJ();
    }

    // Pattern A
    public static void patternA() {
        int spaces = 0;
        for (int i = 1; i <= 9; i++) {
            for (int j = 1; j <= spaces; j++) {
                System.out.print(" ");
            }
            for (int k = 1; k <= 11 - 2 * spaces; k++) {
                System.out.print("# ");
            }
            spaces++;
            System.out.println();
        }
        System.out.println("(a)\n");
    }

    // Pattern B
    public static void patternB() {
        for (int i = 1; i <= 8; i++) {
            for (int j = 1; j <= i; j++) {

```

```

        System.out.print(j + " ");
    }
    System.out.println();
}
System.out.println();
System.out.println("                (b) \n");
}

// Pattern E
public static void patternC() {
    for (int i = 1; i <= 8; i++) {
        for (int j = 8; j >= i; j--) {
            System.out.print(j + " ");
        }
        System.out.println();
    }
    System.out.println();
    System.out.println("                (e) \n");
}

// Pattern F
public static void patternD() {
    for (int i = 8; i >= 1; i--) {
        for (int j = 1; j <= i; j++) {
            System.out.print(j + " ");
        }
        System.out.println();
    }
    System.out.println();
    System.out.println("                (f) \n");
}

// Pattern G
public static void patternE() {
    for (int i = 8; i >= 1; i--) {
        for (int j = 8; j >= i; j--) {
            System.out.print(j + " ");
        }
        System.out.println();
    }
    System.out.println();
    System.out.println("                (g) \n");
}

// Pattern H
public static void patternF() {
    for (int i = 1; i <= 8; i++) {
        for (int j = 1; j <= i; j++) {
            System.out.print(" ");
        }
        for (int k = i; k <= 8; k++) {
            System.out.print(k + " ");
        }
        System.out.println();
    }
    System.out.println();
    System.out.println("                (h) \n");
}
}

```

```

// Pattern I
public static void patternG() {
    for (int i = 8; i >= 1; i--) {
        for (int j = 1; j <= i; j++) {
            System.out.print("  ");
        }
        for (int k = i; k <= 8; k++) {
            System.out.print(k + " ");
        }
        System.out.println();
    }
    System.out.println();
    System.out.println("                (i)\n");
}

// Pattern J
public static void patternH() {
    for (int i = 1; i <= 8; i++) {
        for (int j = 1; j <= i; j++) {
            System.out.print(j + " ");
        }
        System.out.println();
    }
    for (int i = 7; i >= 1; i--) {
        for (int j = 1; j <= i; j++) {
            System.out.print(j + " ");
        }
        System.out.println();
    }
    System.out.println();
    System.out.println("                (j)\n");
}

// Pattern K
public static void patternI() {
    for (int i = 8; i >= 1; i--) {
        for (int j = 1; j <= i; j++) {
            System.out.print(j + " ");
        }
        System.out.println();
    }
    for (int i = 2; i <= 8; i++) {
        for (int j = 1; j <= i; j++) {
            System.out.print(j + " ");
        }
        System.out.println();
    }
    System.out.println();
    System.out.println("                (k)\n");
}

// Pattern L
public static void patternJ() {
    for (int i = 1; i <= 8; i++) {
        // Print spaces
        for (int j = 1; j <= 8 - i; j++) {
            System.out.print("  ");
        }
    }
}

```

```

        // Print numbers in increasing order
        for (int j = i; j <= 2 * i - 1; j++) {
            System.out.printf("%4d", j % 10);
        }

        // Print numbers in decreasing order
        for (int j = 2 * i - 2; j >= i; j--) {
            System.out.printf("%4d", j % 10);
        }

        // Move to the next line
        System.out.println();
    }

}

```

Result

```

# # # # # # # # # # #
# # # # # # # # #
# # # # # # #
# # # # #
# # #
#

```

(a)

```

1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
1 2 3 4 5 6
1 2 3 4 5 6 7
1 2 3 4 5 6 7 8

```

(b)

```

8 7 6 5 4 3 2 1
8 7 6 5 4 3 2
8 7 6 5 4 3
8 7 6 5 4
8 7 6 5
8 7 6
8 7
8

```

(c)

```

1 2 3 4 5 6 7 8
1 2 3 4 5 6 7
1 2 3 4 5 6
1 2 3 4 5
1 2 3 4
1 2 3
1 2

```

1

(d)

8
8 7
8 7 6
8 7 6 5
8 7 6 5 4
8 7 6 5 4 3
8 7 6 5 4 3 2
8 7 6 5 4 3 2 1

(e)

1 2 3 4 5 6 7 8
2 3 4 5 6 7 8
3 4 5 6 7 8
4 5 6 7 8
5 6 7 8
6 7 8
7 8
8

(f)

8
7 8
6 7 8
5 6 7 8
4 5 6 7 8
3 4 5 6 7 8
2 3 4 5 6 7 8
1 2 3 4 5 6 7 8

(g)

1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
1 2 3 4 5 6
1 2 3 4 5 6 7
1 2 3 4 5 6 7 8
1 2 3 4 5 6 7
1 2 3 4 5 6
1 2 3 4 5
1 2 3 4
1 2 3
1 2
1

(h)

1 2 3 4 5 6 7 8
1 2 3 4 5 6 7
1 2 3 4 5 6


```

1 2 3 4 5
1 2 3 4
1 2 3
1 2
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
1 2 3 4 5 6
1 2 3 4 5 6 7
1 2 3 4 5 6 7 8

```

(i)

```

          1
        2 3 2
      3 4 5 4 3
    4 5 6 7 6 5 4
  5 6 7 8 9 8 7 6 5
6 7 8 9 0 1 0 9 8 7 6
7 8 9 0 1 2 3 2 1 0 9 8 7
8 9 0 1 2 3 4 5 4 3 2 1 0 9 8

```

11.4.5 Print Triangles (nested-loop)

Aim: Write a method to print each of the following patterns using nested-loops in a class called PrintTriangles. The program shall prompt user for the number of rows. The signatures of the methods are:

```
public static void printXxx(int numRows); // Xxx is the pattern's name
```

Program

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

public class DisplayPatterns {
    public static void main(String[] args) {
        int rows = 8;

        // Displaying powers of 2 pattern
        System.out.println("Powers of 2 pattern:");
        printPowersOf2Pattern(rows);

        // Displaying Pascal's triangle in triangle format
        System.out.println("\nPascal's triangle (triangle format):");
        printPascalsTriangle(rows);

        // Displaying Pascal's triangle in pyramid format
        System.out.println("\nPascal's triangle (pyramid format):");
        printPascalsPyramid(rows);
    }
}

```

```

// Method to display powers of 2 pattern
public static void printPowersOf2Pattern(int rows) {
    for (int i = 1; i <= rows; i++) {
        // Print spaces
        for (int j = 1; j <= rows - i; j++) {
            System.out.print("    ");
        }

        // Print powers of 2 in increasing order
        int num = 1;
        for (int j = 1; j <= i; j++) {
            System.out.printf("%4d", num);
            num *= 2;
        }

        // Print powers of 2 in decreasing order
        num /= 4;
        for (int j = i - 1; j >= 1; j--) {
            System.out.printf("%4d", num);
            num /= 2;
        }

        // Move to the next line
        System.out.println();
    }
}

// Method to generate Pascal's triangle
public static int[][] generatePascalsTriangle(int rows) {
    int[][] triangle = new int[rows][rows];
    for (int i = 0; i < rows; i++) {
        triangle[i] = new int[i + 1];
        triangle[i][0] = 1;
        triangle[i][i] = 1;
        for (int j = 1; j < i; j++) {
            triangle[i][j] = triangle[i - 1][j - 1] + triangle[i - 1][j];
        }
    }
    return triangle;
}

// Method to display Pascal's triangle in triangle format
public static void printPascalsTriangle(int rows) {
    int[][] triangle = generatePascalsTriangle(rows);
    for (int i = 0; i < triangle.length; i++) {
        for (int j = 0; j < triangle[i].length; j++) {
            System.out.printf("%4d", triangle[i][j]);
        }
        System.out.println();
    }
}

// Method to display Pascal's triangle in pyramid format
public static void printPascalsPyramid(int rows) {
    int[][] triangle = generatePascalsTriangle(rows);
    int[][] formattedPyramid = new int[rows][2 * rows - 1];
    for (int i = 0; i < rows; i++) {

```

```

        for (int j = 0; j < triangle[i].length; j++) {
            formattedPyramid[i][rows - 1 - i + 2 * j] = triangle[i][j];
        }
    }
    for (int i = 0; i < formattedPyramid.length; i++) {
        for (int j = 0; j < formattedPyramid[i].length; j++) {
            if (formattedPyramid[i][j] == 0) {
                System.out.print("    ");
            } else {
                System.out.printf("%4d", formattedPyramid[i][j]);
            }
        }
        System.out.println();
    }
}
}

```

Result

Powers of 2 pattern:

```

                1
              1 2 1
            1 2 4 2 1
          1 2 4 8 4 2 1
        1 2 4 8 16 8 4 2 1
      1 2 4 8 16 32 16 8 4 2 1
    1 2 4 8 16 32 64 32 16 8 4 2 1
  1 2 4 8 16 32 64 128 64 32 16 8 4 2 1

```

Pascal's triangle (triangle format):

```

1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
1 5 10 10 5 1
1 6 15 20 15 6 1
1 7 21 35 35 21 7 1

```

Pascal's triangle (pyramid format):

```

                1
              1 1
            1 2 1
          1 3 3 1
        1 4 6 4 1
      1 5 10 10 5 1
    1 6 15 20 15 6 1
  1 7 21 35 35 21 7 1
1 8 28 56 70 56 28 8 1

```

11.4.6 Trigonometric Series

Aim: Write a method to compute $\sin(x)$ and $\cos(x)$ using the following series expansion, in a class called `TrigonometricSeries`. The signatures of the methods are:

```

public static double sin(double x, int numTerms);
public static double cos(double x, int numTerms);

```

$$\sin(x) = \sum_{n=0}^{\infty} \frac{(-1)^n x^{2n+1}}{(2n+1)!} = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$$

$$\cos(x) = \sum_{n=0}^{\infty} \frac{(-1)^n x^{2n}}{(2n)!} = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \dots$$

Compare the values computed using the series with the JDK methods Math.sin(), Math.cos() at x=0, $\pi/6$, $\pi/4$, $\pi/3$, $\pi/2$ using various numbers of terms.

Program

```
/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

public class TrigonometricSeries {
    // Method to compute factorial
    private static double factorial(int n) {
        double result = 1;
        for (int i = 2; i <= n; i++) {
            result *= i;
        }
        return result;
    }

    // Method to compute sin(x) using series expansion
    public static double sin(double x, int numTerms) {
        double result = 0;
        for (int i = 0; i < numTerms; i++) {
            int sign = i % 2 == 0 ? 1 : -1;
            double term = sign * Math.pow(x, 2 * i + 1) / factorial(2 *
                i + 1);
            result += term;
        }
        return result;
    }

    // Method to compute cos(x) using series expansion
    public static double cos(double x, int numTerms) {
        double result = 0;
        for (int i = 0; i < numTerms; i++) {
            int sign = i % 2 == 0 ? 1 : -1;
            double term = sign * Math.pow(x, 2 * i) / factorial(2 * i);
            result += term;
        }
        return result;
    }

    public static void main(String[] args) {
        double[] angles = {0, Math.PI / 6, Math.PI / 4, Math.PI / 3,
            Math.PI / 2};
        int[] numTermsArray = {5, 10, 15};

        System.out.println("Angle(deg)\tNum Terms\tSin(x)\t\tMath.sin(x)
            \tCos(x)\t\tMath.cos(x)");

        for (double angleRad : angles) {
            double angleDeg = Math.toDegrees(angleRad); // Convert
                radians to degrees
            for (int numTerms : numTermsArray) {
```

```

        double sinValue = sin(angleRad, numTerms);
        double mathSinValue = Math.sin(angleRad);
        double cosValue = cos(angleRad, numTerms);
        double mathCosValue = Math.cos(angleRad);
        System.out.printf("%.2f\t\t\t%d\t\t\t%.8f\t\t%.8f\t\t%.8f\t\t%.8f\n",
            angleDeg, numTerms, sinValue, mathSinValue,
            cosValue, mathCosValue);
    }
}
}

```

Result

Angle(deg)	Num Terms	Sin(x)	Math.sin(x)	Cos(x)	Math.cos(x)
0.00	5	0.00000000	0.00000000	1.00000000	1.00000000
0.00	10	0.00000000	0.00000000	1.00000000	1.00000000
0.00	15	0.00000000	0.00000000	1.00000000	1.00000000
30.00	5	0.50000000	0.50000000	0.86602540	0.86602540
30.00	10	0.50000000	0.50000000	0.86602540	0.86602540
30.00	15	0.50000000	0.50000000	0.86602540	0.86602540
45.00	5	0.70710678	0.70710678	0.70710681	0.70710678
45.00	10	0.70710678	0.70710678	0.70710678	0.70710678
45.00	15	0.70710678	0.70710678	0.70710678	0.70710678
60.00	5	0.86602545	0.86602540	0.50000043	0.50000000
60.00	10	0.86602540	0.86602540	0.50000000	0.50000000
60.00	15	0.86602540	0.86602540	0.50000000	0.50000000
90.00	5	1.00000354	1.00000000	0.00002474	0.00000000
90.00	10	1.00000000	1.00000000	-0.00000000	0.00000000
90.00	15	1.00000000	1.00000000	0.00000000	0.00000000

11.4.7 Exponential Series

Aim: Write a method to compute e and exp(x) using the following series expansion, in a class called ExponentialSeries. The signatures of the methods are:

```

public static double exp(int numTerms);    // x in radians
public static double exp(double x, int numTerms);

```

$$e = \sum_{n=0}^{\infty} \frac{1}{n!} = 1 + \frac{1}{1!} + \frac{1}{2!} + \frac{1}{3!} + \dots$$

$$\exp(x) = \sum_{n=0}^{\infty} \frac{x^n}{n!} = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots$$

Program

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

public class ExponentialSeries {
    // Method to compute factorial
    private static double factorial(int n) {
        double result = 1;
        for (int i = 2; i <= n; i++) {
            result *= i;
        }
    }
}

```

```

    }
    return result;
}

// Method to compute e using series expansion
public static double exp(int numTerms) {
    double result = 0;
    for (int i = 0; i < numTerms; i++) {
        result += Math.pow(1, i) / factorial(i);
    }
    return result;
}

// Method to compute exp(x) using series expansion
public static double exp(double x, int numTerms) {
    double result = 0;
    for (int i = 0; i < numTerms; i++) {
        result += Math.pow(x, i) / factorial(i);
    }
    return result;
}

public static void main(String[] args) {
    int numTerms = 100;
    double e = exp(numTerms);
    double mathE = Math.exp(1); // Using Math library for e
    System.out.println("e (using " + numTerms + " terms): " + e);
    System.out.println("e (using Math library): " + mathE);

    double x = 2.5; // Example value of x
    double expValue = exp(x, numTerms);
    double mathExpValue = Math.exp(x); // Using Math library for
    exp(x)
    System.out.println("exp(" + x + ") (using " + numTerms + "
        terms): " + expValue);
    System.out.println("exp(" + x + ") (using Math library): " +
        mathExpValue);
}
}

```

Result

e (using 100 terms): 2.7182818284590455
 e (using Math library): 2.718281828459045
 exp(2.5) (using 100 terms): 12.182493960703471
 exp(2.5) (using Math library): 12.182493960703473

11.4.8 Special Series

Aim: Write a method to compute the sum of the series in a class called SpecialSeries. The signature of the method is:

```
public static double specialSeries(double x, int numTerms);
```

$$\text{sum} = x + \frac{1}{2} \cdot \frac{x^3}{3} + \frac{1 \cdot 3}{2 \cdot 4} \cdot \frac{x^5}{5} + \frac{1 \cdot 3 \cdot 5}{2 \cdot 4 \cdot 6} \cdot \frac{x^7}{7} + \frac{1 \cdot 3 \cdot 5 \cdot 7}{2 \cdot 4 \cdot 6 \cdot 8} \cdot \frac{x^9}{9} + \frac{1 \cdot 3 \cdot 5 \cdot 7 \cdot 9}{2 \cdot 4 \cdot 6 \cdot 8 \cdot 10} \cdot \frac{x^{11}}{11} + \frac{1 \cdot 3 \cdot 5 \cdot 7 \cdot 9 \cdot 11}{2 \cdot 4 \cdot 6 \cdot 8 \cdot 10 \cdot 12} \cdot \frac{x^{13}}{13} + \dots$$

Program

```
public class SeriesEvaluation {
    public static void main(String[] args) {
        double x = 0.5; // Define the value of x within the range -1 <= x
                           <= 1
        int n = 10; // Define the number of terms in the series

        double sum = x; // Initialize the sum with the first term (x)

        // Loop to calculate and add each subsequent term to the sum
        for (int i = 1; i <= n; i++) {
            double numerator = 1; // Initialize the numerator of the term
            double denominator = 1; // Initialize the denominator of the
                term

            // Calculate the numerator
            for (int j = 1; j <= 2 * i - 1; j += 2) {
                numerator *= j;
            }

            // Calculate the denominator
            for (int j = 2; j <= 2 * i; j += 2) {
                denominator *= j;
            }

            // Calculate the current term and add it to the sum
            sum += (numerator / denominator) * Math.pow(x, 2 * i + 1);
        }

        System.out.println("Sum of the series: " + sum);
    }
}
```

Result

Sum of the series: 0.5773502428146458

11.4.9 FactorialInt (Handling Overflow)

Aim: Write a program called FactorialInt to list all the factorials that can be expressed as an int (i.e., 32-bit signed integer in the range of [-2147483648, 2147483647]). Your output shall look like:

The factorial of 1 is 1
The factorial of 2 is 2
...
The factorial of 12 is 479001600
The factorial of 13 is out of range

Program

```
/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

public class FactorialInt {
```

```

public static void main(String[] args) {
    // Loop through numbers from 1 to the maximum value where factorial
    // can be represented as an int
    for (int i = 1; i <= 13; i++) {
        int factorial = calculateFactorial(i);
        if (factorial != -1) {
            System.out.println("The factorial of " + i + " is " +
                               factorial);
        } else {
            System.out.println("The factorial of " + i + " is out of
                               range");
            break; // Stop the loop when overflow occurs
        }
    }
}

// Method to calculate factorial of a number
private static int calculateFactorial(int n) {
    int result = 1;
    for (int i = 2; i <= n; i++) {
        // Check for overflow
        if (Integer.MAX_VALUE / result < i) {
            return -1; // Overflow occurred
        }
        result *= i;
    }
    return result;
}
}

```

Result

The factorial of 1 is 1
 The factorial of 2 is 2
 The factorial of 3 is 6
 The factorial of 4 is 24
 The factorial of 5 is 120
 The factorial of 6 is 720
 The factorial of 7 is 5040
 The factorial of 8 is 40320
 The factorial of 9 is 362880
 The factorial of 10 is 3628800
 The factorial of 11 is 39916800
 The factorial of 12 is 479001600
 The factorial of 13 is out of range

11.4.10 FibonacciInt (Handling Overflow)

Aim: Write a program called FibonacciInt to list all the Fibonacci numbers, which can be expressed as an int (i.e., 32-bit signed integer in the range of [-2147483648, 2147483647]). The output shall look like:

$F(0) = 1$
 $F(1) = 1$

$F(2) = 2$

...

$F(45) = 1836311903$

$F(46)$ is out of the range of int

Program

```
/* @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */
public class FibonacciInt {

    public static void main(String[] args) {
        // Initial Fibonacci numbers
        int prev = 1;
        int current = 1;

        // Print initial Fibonacci numbers
        System.out.println("F(0) = " + prev);
        System.out.println("F(1) = " + current);

        // Calculate and print Fibonacci numbers until overflow
        for (int i = 2; ; i++) {
            int next = prev + current;
            if (next < 0 || prev > Integer.MAX_VALUE - current) {
                System.out.println("F(" + i + ") is out of the range of int
                    ");
                break; // Stop when overflow occurs
            }
            System.out.println("F(" + i + ") = " + next);
            prev = current;
            current = next;
        }
    }
}
```

Result

$F(0) = 1$

$F(1) = 1$

$F(2) = 2$

$F(3) = 3$

$F(4) = 5$

$F(5) = 8$

$F(6) = 13$

$F(7) = 21$

$F(8) = 34$

$F(9) = 55$

$F(10) = 89$

$F(11) = 144$

$F(12) = 233$

$F(13) = 377$

$F(14) = 610$

$F(15) = 987$

$F(16) = 1597$

F(17) = 2584
 F(18) = 4181
 F(19) = 6765
 F(20) = 10946
 F(21) = 17711
 F(22) = 28657
 F(23) = 46368
 F(24) = 75025
 F(25) = 121393
 F(26) = 196418
 F(27) = 317811
 F(28) = 514229
 F(29) = 832040
 F(30) = 1346269
 F(31) = 2178309
 F(32) = 3524578
 F(33) = 5702887
 F(34) = 9227465
 F(35) = 14930352
 F(36) = 24157817
 F(37) = 39088169
 F(38) = 63245986
 F(39) = 102334155
 F(40) = 165580141
 F(41) = 267914296
 F(42) = 433494437
 F(43) = 701408733
 F(44) = 1134903170
 F(45) = 1836311903
 F(46) is out of the range of int

11.4.11 Number System Conversion

Aim: Write a method call toRadix() which converts a positive integer from one radix into another. The method has the following header:

```
public static String toRadix(String in, int inRadix, int outRadix)
```

Write a program called NumberConversion, which prompts the user for an input string, an input radix, and an output radix, and display the converted number.

Program

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

import java.util.Scanner;
import java.math.BigInteger;

public class NumberConversion {

```

```

public static void main(String[] args) {
    // Prompt the user for input
    Scanner scanner = new Scanner(System.in);
    System.out.print("Enter a number and radix: ");
    String input = scanner.next();
    System.out.print("Enter the input radix: ");
    int inputRadix = scanner.nextInt();
    System.out.print("Enter the output radix: ");
    int outputRadix = scanner.nextInt();

    // Convert the number
    String result = toRadix(input, inputRadix, outputRadix);

    // Display the result
    System.out.println "\"" + input + "\" in radix " + inputRadix + "
        is \"" + result + "\" in radix " + outputRadix + ".");
}

public static String toRadix(String in, int inRadix, int outRadix) {
    // Convert the input string to a BigInteger using the specified
    // radix
    BigInteger number = new BigInteger(in, inRadix);

    // Convert the BigInteger to a string using the specified output
    // radix
    return number.toString(outRadix);
}
}

```

Result

Test case 1:

Enter a number and radix: 1289
Enter the input radix: 10
Enter the output radix: 8
"1289" in radix 10 is "2411" in radix 8.

Test case 2:

Enter a number and radix: 1245
Enter the input radix: 8
Enter the output radix: 2
"1245" in radix 8 is "1010100101" in radix 2.

Test case 3:

Enter a number and radix: 1289
Enter the input radix: 10
Enter the output radix: 2
"1289" in radix 10 is "10100001001" in radix 2.

Test case 4:

Enter a number and radix: 1A9D
Enter the input radix: 16
Enter the output radix: 10
"1A9D" in radix 16 is "6813" in radix 10.

Test case 5:

Enter a number and radix: F

Enter the input radix: 16

Enter the output radix: 2

"F" in radix 16 is "1111" in radix 2.

11.4.12 NumberGuess

Aim: Write a program called NumberGuess to play the number guessing game. The program shall generate a random number between 0 and 99. The player inputs his/her guess, and the program shall response with "Try higher", "Try lower" or "You got it in n trials" accordingly.

Program

```
/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

import java.util.Scanner;
import java.util.Random;

public class NumberGuess {

    public static void main(String[] args) {
        Random random = new Random();
        int randomNumber = random.nextInt(100); // Generate a random number
            between 0 and 99
        int trials = 0;

        Scanner scanner = new Scanner(System.in);

        System.out.println("Guess the number between 0 and 99");

        while (true) {
            System.out.print("Key in your guess: ");
            int guess = scanner.nextInt();
            trials++;

            if (guess < randomNumber) {
                System.out.println("Try higher");
            } else if (guess > randomNumber) {
                System.out.println("Try lower");
            } else {
                System.out.println("You got it in " + trials + " trials!");
                break; // Exit the loop when the correct number is guessed
            }
        }
        scanner.close();
    }
}
```

Result

Guess the number between 0 and 99

Key in your guess: 9

```

Try higher
Key in your guess: 50
Try higher
Key in your guess: 75
Try lower
Key in your guess: 65
Try lower
Key in your guess: 55
Try higher
Key in your guess: 60
Try higher
Key in your guess: 63
You got it in 7 trials!

```

11.4.13 WordGuess

Aim: Write a program called WordGuess to guess a word by trying to guess the individual characters. The word to be guessed shall be provided using the command-line argument.

Program

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

import java.util.Scanner;

public class WordGuess {

    public static void main(String[] args) {
        if (args.length == 0) {
            System.out.println("Usage: java WordGuess <word>");
            return;
        }

        String wordToGuess = args[0];
        char[] guessedWord = new char[wordToGuess.length()];
        for (int i = 0; i < guessedWord.length; i++) {
            guessedWord[i] = '_';
        }

        int trials = 0;
        Scanner scanner = new Scanner(System.in);

        System.out.println("Welcome to Word Guess Game!");

        while (true) {
            System.out.print("Trial " + (trials + 1) + ": ");
            for (char c : guessedWord) {
                System.out.print(c);
            }
            System.out.println();

            System.out.print("Key in one character or your guess word: ");

```

```

String input = scanner.nextLine();

if (input.equals(wordToGuess)) {
    System.out.println("Congratulations!");
    System.out.println("You got it in " + (trials + 1) + "
        trials");
    break;
} else if (input.length() == 1) {
    char guessChar = input.charAt(0);
    boolean correctGuess = false;
    for (int i = 0; i < wordToGuess.length(); i++) {
        if (wordToGuess.charAt(i) == guessChar) {
            guessedWord[i] = guessChar;
            correctGuess = true;
        }
    }
    if (!correctGuess) {
        trials++;
    }
} else {
    System.out.println("Invalid input! Please enter one
        character or your guess word.");
}

// Check if the guessed word matches the word to be guessed
if (String.valueOf(guessedWord).equals(wordToGuess)) {
    System.out.println("Congratulations!");
    System.out.println("The word you guessed is: " + args[0]);
    System.out.println("You got it in " + (trials + 1) + "
        trials");
    break;
}

scanner.close();
}
}

```

Result

Welcome to Word Guess Game!

Trial 1: ____

Key in one character or your guess word: T

Trial 2: ____

Key in one character or your guess word: S

Trial 3: ____

Key in one character or your guess word: H

Trial 4: ____

Key in one character or your guess word: R

Trial 4: _R_

Key in one character or your guess word: D

Trial 5: _R_

Key in one character or your guess word: A

Trial 5: _AR_

Key in one character or your guess word: E

Trial 5: _ARE

Key in one character or your guess word: I
Congratulations!
The word you guessed is: IARE
You got it in 5 trials

11.4.14 DateUtil

Aim: Complete the following methods in a class called DateUtil:

- `boolean isLeapYear(int year):`

returns true if the given year is a leap year. A year is a leap year if it is divisible by 4 but not by 100, or it is divisible by 400.

- `boolean isValidDate(int year, int month, int day):`

returns true if the given year, month and day constitute a given date. Assume that year is between 1 and 9999, month is between 1 (Jan) to 12 (Dec) and day shall be between 1 and 28—29—30—31 depending on the month and whether it is a leap year.

- `int getDayOfWeek(int year, int month, int day):`

returns the day of the week, where 0 for SUN, 1 for MON, ..., 6 for SAT, for the given date. Assume that the date is valid.

- `String toString(int year, int month, int day):`

prints the given date in the format "xxxd day d mmm yyyy", e.g., "Tuesday 14 Feb 2012". Assume that the given date is valid.

Program

```
/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

public class DateUtil {

    // Month's name for printing
    public static String[] strMonths = {"Jan", "Feb", "Mar", "Apr", "May",
        "Jun", "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"};

    // Number of days in each month (for non-leap years)
    public static int[] daysInMonths = {31, 28, 31, 30, 31, 30, 31, 31, 30,
        31, 30, 31};

    // Returns true if the given year is a leap year
    public static boolean isLeapYear(int year) {
        return (year % 4 == 0 && year % 100 != 0) || year % 400 == 0;
    }

    // Return true if the given year, month, day is a valid date
    public static boolean isValidDate(int year, int month, int day) {
        if (year < 1 || year > 9999 || month < 1 || month > 12 || day < 1)
```

```

        return false;

        int maxDays = daysInMonths[month - 1];
        if (month == 2 && isLeapYear(year))
            maxDays = 29;

        return day <= maxDays;
    }

    // Return the day of the week, 0:Sun, 1:Mon, ..., 6:Sat
    public static int getDayOfWeek(int year, int month, int day) {
        int[] centuryTable = {4, 2, 0, 6}; // Century Table: 1700-1799: 4,
            1800-1899: 2, 1900-1999: 0, 2000-2099: 6
        int[] monthTableNonLeap = {0, 3, 3, 6, 1, 4, 6, 2, 5, 0, 3, 5}; //
            Month Table for non-leap years
        int[] monthTableLeap = {6, 2, 3, 6, 1, 4, 6, 2, 5, 0, 3, 5}; //
            Month Table for leap years

        // Step 1: Get the number from the century table
        int firstTwoDigits = year / 100;
        int centuryNumber = centuryTable[(firstTwoDigits - 17) % 4]; //
            Adjust for 1700 being the start of the cycle

        // Step 2: Add to the last two digits of the year
        int lastTwoDigits = year % 100;

        // Step 3: Add to "the last two digits of the year divide by 4,
            truncate the fractional part"
        int lastTwoDigitsDividedBy4 = lastTwoDigits / 4;

        // Step 4: Add to the number obtained from the month table
        int monthTableNumber = isLeapYear(year) ? monthTableLeap[month - 1]
            : monthTableNonLeap[month - 1];

        // Step 5: Add to the day
        int sum = centuryNumber + lastTwoDigits + lastTwoDigitsDividedBy4 +
            monthTableNumber + day;

        // Step 6: The sum modulus 7 gives the day of the week
        return sum % 7;
    }

    // Return String "xxxdays d mmm yyyy" (e.g., Wednesday 29 Feb 2012)
    public static String toString(int year, int month, int day) {
        int dayOfWeekIndex = getDayOfWeek(year, month, day);
        String dayOfWeek = getDayOfWeekString(dayOfWeekIndex);
        String monthName = strMonths[month - 1];
        return dayOfWeek + " " + day + " " + monthName + " " + year;
    }

    // Helper method to get the day of the week string
    private static String getDayOfWeekString(int dayOfWeekIndex) {
        String[] daysOfWeek = {"Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "
            Sat"};
        return daysOfWeek[dayOfWeekIndex];
    }

    // Test Driver

```



```

public static void main(String[] args) {
    System.out.println(isLeapYear(1900)); // false
    System.out.println(isLeapYear(2000)); // true
    System.out.println(isLeapYear(2011)); // false
    System.out.println(isLeapYear(2012)); // true

    System.out.println(isValidDate(2012, 2, 29)); // true
    System.out.println(isValidDate(2011, 2, 29)); // false
    System.out.println(isValidDate(2099, 12, 31)); // true
    System.out.println(isValidDate(2099, 12, 32)); // false

    System.out.println(getDayOfWeekString(getDayOfWeek(1982, 4, 24)));
        // Sat
    System.out.println(getDayOfWeekString(getDayOfWeek(2000, 1, 1)));
        // Sat
    System.out.println(getDayOfWeekString(getDayOfWeek(2054, 6, 19)));
        // Fri
    System.out.println(getDayOfWeekString(getDayOfWeek(2012, 2, 17)));
        // Fri

    System.out.println(toString(2012, 2, 14)); // Tuesday 14 Feb 2012
}
}

```

Result

```

false
true
false
true
true
false
true
false
Sat
Sat
Fri
Fri
Tue 14 Feb 2012

```

11.5 Sample Viva Voce Questions

1. Explain the usage of the `java.util.Calendar` and `java.time.LocalDate` classes for working with dates in Java.
2. Discuss the differences between the legacy `Date` class and the modern `LocalDate` class in Java, and why the latter is preferred for date manipulation.
3. How do you approach the generation of Pascal's Triangle using nested loops or recursion in Java?
4. Discuss the use of two-dimensional arrays or lists in Java to represent and store Pascal's Triangle efficiently.

11.6 Further Probing Experiments

1. Modify your program called FactorialInt to list all the factorial that can be expressed as a long (64-bit signed integer)(FactorialLong). The maximum value for long is kept in a constant called Long.MAX_VALUE.
2. Write a similar program called TribonacciInt for Tribonacci numbers.(Refer 11.10)

EXERCISES ON CLASSES AND OBJECTS

12.1 Introduction

This lab is designed to provide you with hands-on experience in working with classes and objects in the Java programming language. Classes and objects are fundamental concepts in object-oriented programming, and mastering them is crucial for developing robust and maintainable software applications. In this lab, you will apply your knowledge of object-oriented programming and Java syntax to create and manipulate classes and objects. The lab is designed to enhance your understanding of class and object concepts and enable you to proficiently define and work with them in a generic context.

12.2 Prelab Preparation

12.2.1 Objective

By the end of this lab, students should be able to:

1. Gain a deep understanding of the concepts of classes and objects in Java.
2. Define and implement classes that encapsulate data and behavior.
3. Create and manipulate objects from these classes to represent and interact with real-world entities.
4. Practice designing and using constructors, methods, and attributes within classes.
5. Develop problem-solving skills by solving practical exercises using classes and objects

12.2.2 Background

In Java, a class is a blueprint or template for creating objects. An object, on the other hand, is an instance of a class, representing a real-world entity or concept. Objects are created based on the structure defined in a class, which includes attributes (fields) and methods (functions). Let's explore classes and objects in Java with syntax and examples:

1. Class Definition:

To define a class in Java, you use the `class` keyword, followed by the class name. Here's a simple class definition:

```
public class Car {  
    // Attributes (Fields)  
    String make;  
    String model;  
    int year;  
  
    // Methods
```

```

        public void start() {
            System.out.println("The car is starting.");
        }

        public void drive() {
            System.out.println("The car is in motion.");
        }
    }
}

```

In this example, we've defined a class called "Car" with attributes (make, model, year) and methods (start, drive). The public keyword indicates that the class is accessible from other classes.

2. Object Creation:

To create an object from a class, you use the new keyword followed by the class name and a constructor if one exists. Here's how you create a "Car" object:

```

public class Main {
    public static void main(String[] args) {
        // Creating a Car object
        Car myCar = new Car();

        // Setting object attributes
        myCar.make = "Toyota";
        myCar.model = "Camry";
        myCar.year = 2020;

        // Calling object methods
        myCar.start();
        myCar.drive();
    }
}

```

In this example, we've created a "Car" object named myCar and assigned values to its attributes. We then called the start and drive methods on the object.

3. Constructors:

Constructors are special methods used to initialize objects when they are created. They have the same name as the class and are called using the new keyword. Here's a class with a constructor:

```

public class Person {
    String name;
    int age;

    // Constructor
    public Person(String n, int a) {
        name = n;
        age = a;
    }
}

```

To create a "Person" object using the constructor:

```

public class Main {
    public static void main(String[] args) {
        // Creating a Person object using a constructor
        Person person1 = new Person("Alice", 30);
    }
}

```

```

        Person person2 = new Person("Bob", 25);

        System.out.println(person1.name + " is " + person1.age
            + " years old.");
        System.out.println(person2.name + " is " + person2.age
            + " years old.");
    }
}

```

In this example, we've defined a parameterized constructor for the "Person" class, and we create two "Person" objects by passing values to the constructor.

4. Accessing Object Members:

You can access object attributes and methods using the dot notation (objectName.attribute or objectName.method()). For example:

```

String carMake = myCar.make;
myCar.drive();

```

These are the fundamental concepts of classes and objects in Java. Classes serve as blueprints for creating objects, and objects encapsulate data (attributes) and behavior (methods) to represent real-world entities or concepts.

5. The new Keyword:

In Java, the new keyword is used to create new instances (objects) of a class. It dynamically allocates memory for an object and invokes the class constructor to initialize the object. Here's the syntax and an example:

Syntax: `ClassName objectName = new ClassName();`

Example:

```

// Creating an instance of the class "Person"
Person person = new Person();

```

In this example, we use the new keyword to create a new "Person" object named person.

6. Access Specifiers:

Access specifiers are keywords in Java that control the visibility and accessibility of class members (fields, methods, and nested classes). There are four main access specifiers: public, private, protected, and default (no specifier). Here's an explanation of each:

public: Members marked as public are accessible from anywhere.

private: Members marked as private are only accessible within the same class.

protected: Members marked as protected are accessible within the same class, subclass, and package.

Default (no specifier): Members with no access specifier are accessible only within the same package.

12.2.3 Hardware/Software Requirements

To successfully complete this lab, students will need access to the following:

1. A computer with a suitable text editor or Integrated Development Environment (IDE) installed, such as Eclipse, IntelliJ IDEA, or Visual Studio Code.
2. Java Development Kit (JDK) installed on the computer. Ensure that the JDK is correctly configured, and the system's PATH variable is set to include the JDK's bin directory.

12.2.4 Pre-Lab Questions

1. Can you explain the "has-a" relationship in composition with an example?
2. What are the advantages of using composition in Java?
3. How does composition contribute to code reusability?
4. What are some common pitfalls or challenges when using composition?

12.3 Experiment Procedure

1. **Design Classes:** Create classes for simpler components.
2. **Implement Composition:** Create a class for the complex object and inject instances of simpler components.
3. **Test Composition:** Verify functionality by testing interactions between complex object and components.
4. **Experiment:** Modify configurations to observe effects on complex object behavior.
5. **Benefits:** Discuss benefits such as code reusability and modularity.
6. **Conclusion:** Summarize insights gained from the experiment.

12.4 Experiemnts

12.4.1 The Rectangle Class

A class called Rectangle, which models a rectangle with a length and a width (in float), is designed as shown in the following class diagram. Write the Rectangle class.

The expected output is:

```
Rectangle[length=1.2,width=3.4]
Rectangle[length=1.0,width=1.0]
Rectangle[length=5.6,width=7.8]
length is: 5.6
width is: 7.8
area is: 43.68
perimeter is: 26.80
```

Program

Rectangle.java

```
/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept of CSE,
 * IARE, Dundigal.
```

Hints:

Rectangle	
-length:float = 1.0f -width:float = 1.0f	
+Rectangle() +Rectangle(length:float,width:float) +getLength():float +setLength(length:float):void +getWidth():float +setWidth(width:float):void +getArea():double +getPerimeter():double +toString():String •	"Rectangle[length=?,width=?]"

```
*/  
  
public class Rectangle {  
    private float length;  
    private float width;  
  
    // Default constructor  
    public Rectangle() {  
        length = 1.0f;  
        width = 1.0f;  
    }  
  
    // Parameterized constructor  
    public Rectangle(float length, float width) {  
        this.length = length;  
        this.width = width;  
    }  
  
    // Getter for length  
    public float getLength() {  
        return length;  
    }  
  
    // Setter for length  
    public void setLength(float length) {  
        this.length = length;  
    }  
  
    // Getter for width  
    public float getWidth() {  
        return width;  
    }  
  
    // Setter for width  
    public void setWidth(float width) {  
        this.width = width;  
    }  
}
```

```

    }

    // Method to calculate the area
    public double getArea() {
        return length * width;
    }

    // Method to calculate the perimeter
    public double getPerimeter() {
        return 2 * (length + width);
    }

    @Override
    public String toString() {
        return "Rectangle[length=" + length + ",width=" + width + "];"
    }

    public static void main(String[] args) {
        Rectangle rectangle1 = new Rectangle(1.2f, 3.4f);
        Rectangle rectangle2 = new Rectangle();
        Rectangle rectangle3 = new Rectangle(5.6f, 7.8f);

        System.out.println(rectangle1);
        System.out.println(rectangle2);
        System.out.println(rectangle3);

        System.out.printf("length is: %.1f\n",rectangle3.getLength());
        System.out.printf("width is: %.1f\n",rectangle3.getWidth());
        System.out.printf("area is: %.2f\n",rectangle3.getArea());
        System.out.printf("perimeter is: %.2f\n",rectangle3.getPerimeter());
        ;
    }
}

```

Result

```

Rectangle[length=1.2,width=3.4]
Rectangle[length=1.0,width=1.0]
Rectangle[length=5.6,width=7.8]
length is: 5.6
width is: 7.8
area is: 43.68
perimeter is: 26.80

```

12.4.2 The Employee Class

A class called Employee, which models an employee with an ID, name and salary, is designed as shown in the following class diagram. The method raiseSalary(percent) increases the salary by the given percentage. Write the Employee class.

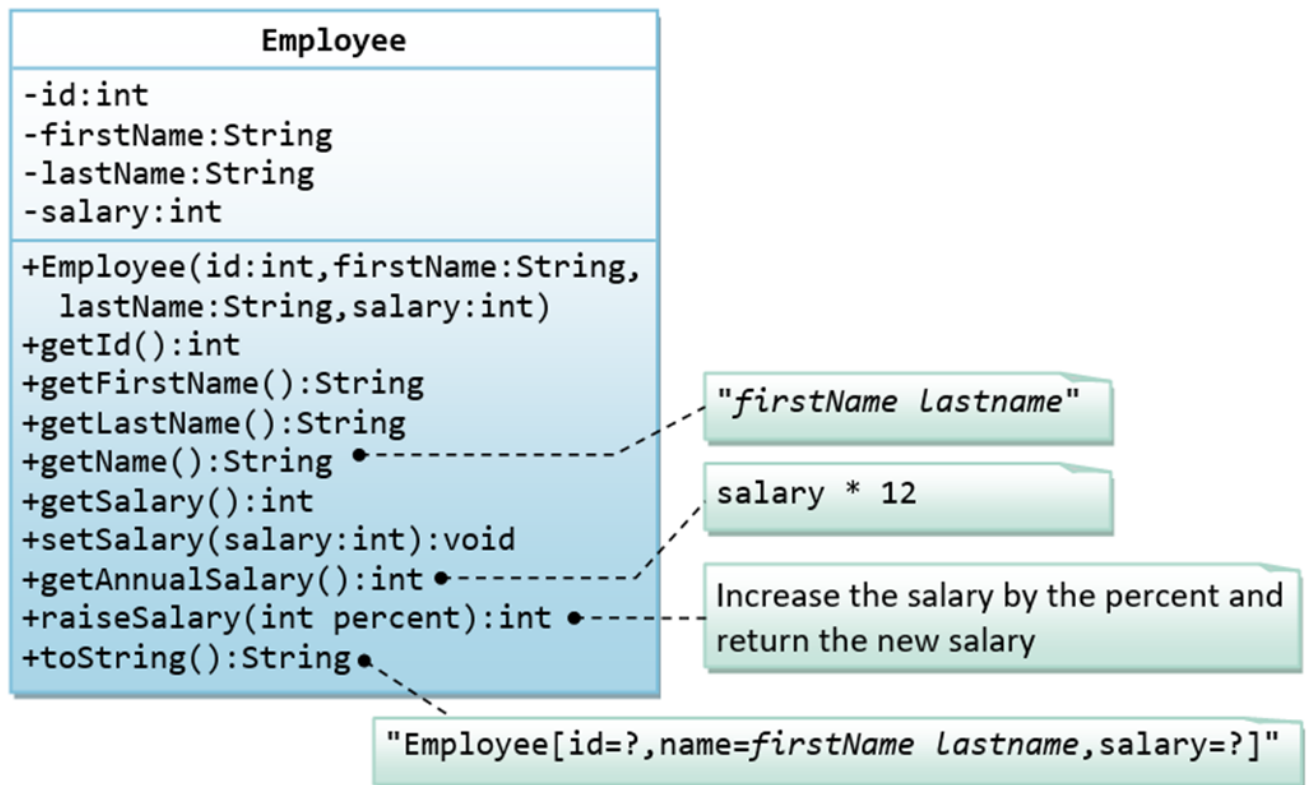
The expected output is:

```

Employee[id=8,name=Peter Tan,salary=2500]
Employee[id=8,name=Peter Tan,salary=999]
id is: 8
firstname is: Peter
lastname is: Tan
salary is: 999
name is: Peter Tan

```


Hints:



```
annual salary is: 11988
1098
Employee[id=8,name=Peter Tan,salary=1098]
```

Program

```
/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept of CSE,
 * IARE, Dundigal.
 */

public class Employee {
    private int id;
    private String firstName;
    private String lastName;
    private int salary;

    // Constructor
    public Employee(int id, String firstName, String lastName, int salary)
    {
        this.id = id;
        this.firstName = firstName;
        this.lastName = lastName;
        this.salary = salary;
    }

    // Getter for ID
    public int getId() {
```

```

        return id;
    }

    // Getter for First Name
    public String getFirstName() {
        return firstName;
    }

    // Getter for Last Name
    public String getLastName() {
        return lastName;
    }

    // Getter for Salary
    public int getSalary() {
        return salary;
    }

    // Setter for Salary
    public void setSalary(int salary) {
        this.salary = salary;
    }

    // Method to get the annual salary
    public int getAnnualSalary() {
        return salary * 12;
    }

    // Method to raise the salary by a given percentage
    public int raiseSalary(int percent) {
        double increase = (salary * percent) / 100.0;
        salary += increase;
        return salary;
    }

    @Override
    public String toString() {
        return "Employee[id=" + id + ",name=" + firstName + " " + lastName
            + ",salary=" + salary + "]";
    }

    public static void main(String[] args) {
        Employee employee = new Employee(8, "Peter", "Tan", 2500);

        System.out.println(employee);

        employee.setSalary(999);

        System.out.println(employee);

        System.out.println("id is: " + employee.getId());
        System.out.println("firstname is: " + employee.getFirstName());
        System.out.println("lastname is: " + employee.getLastName());
        System.out.println("salary is: " + employee.getSalary());
        System.out.println("name is: " + employee.getFirstName() + " " +
            employee.getLastName());
        System.out.println("annual salary is: " + employee.getAnnualSalary
            ());
    }

```

```

        System.out.println(employee.raiseSalary(10)); // Increasing salary
            by 10%
        System.out.println(employee);
    }
}

```

Result

```

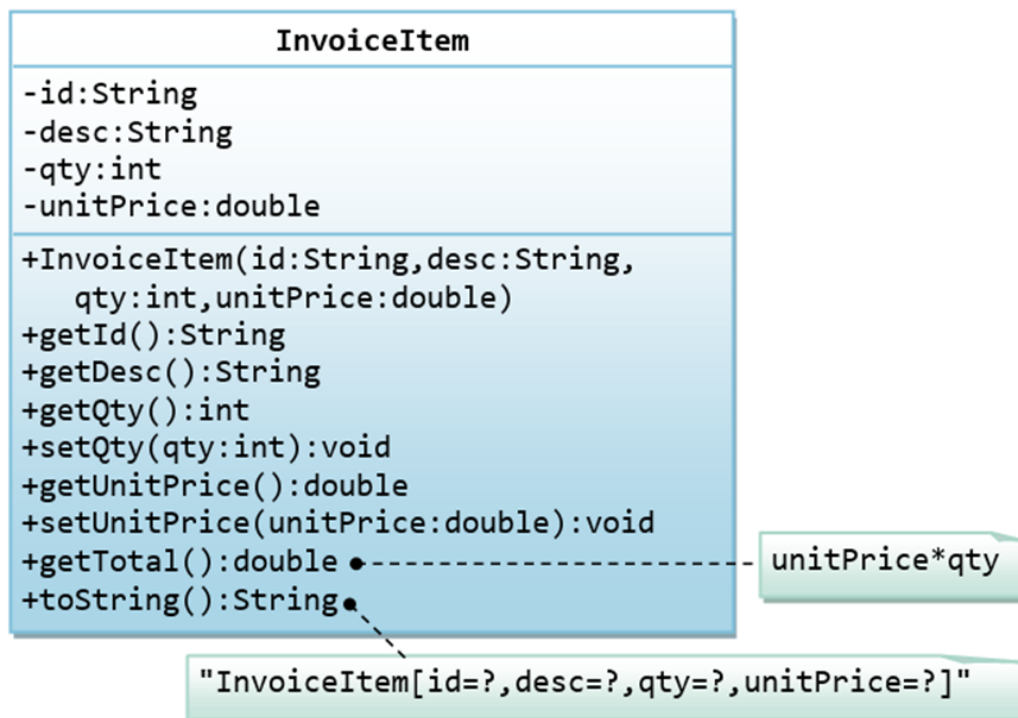
Employee[id=8,name=Peter Tan,salary=2500]
Employee[id=8,name=Peter Tan,salary=999]
id is: 8
firstname is: Peter
lastname is: Tan
salary is: 999
name is: Peter Tan
annual salary is: 11988
1098
Employee[id=8,name=Peter Tan,salary=1098]

```

12.4.3 The InvoiceItem Class

A class called InvoiceItem, which models an item of an invoice, with ID, description, quantity and unit price, is designed as shown in the following class diagram. Write the InvoiceItem class.

Hints:



The expected output is:

```

InvoiceItem[id=A101,desc=Pen Red,qty=888,unitPrice=0.08]
InvoiceItem[id=A101,desc=Pen Red,qty=999,unitPrice=0.99]
id is: A101
desc is: Pen Red

```

```
    qty is: 999
    unitPrice is: 0.99
    The total is: 989.01
```

Program

```
/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept of CSE,
 * IARE, Dundigal.
 */

public class InvoiceItem {
    private String id;
    private String desc;
    private int qty;
    private double unitPrice;

    // Constructor
    public InvoiceItem(String id, String desc, int qty, double unitPrice) {
        this.id = id;
        this.desc = desc;
        this.qty = qty;
        this.unitPrice = unitPrice;
    }

    // Getter for ID
    public String getId() {
        return id;
    }

    // Getter for Description
    public String getDesc() {
        return desc;
    }

    // Getter for Quantity
    public int getQty() {
        return qty;
    }

    // Setter for Quantity
    public void setQty(int qty) {
        this.qty = qty;
    }

    // Getter for Unit Price
    public double getUnitPrice() {
        return unitPrice;
    }

    // Setter for Unit Price
    public void setUnitPrice(double unitPrice) {
        this.unitPrice = unitPrice;
    }

    // Method to calculate the total price
    public double getTotal() {
        return qty * unitPrice;
    }
}
```

```

    }

    @Override
    public String toString() {
        return "InvoiceItem[id=" + id + ",desc=" + desc + ",qty=" + qty +
            ",unitPrice=" + unitPrice + "]";
    }

    public static void main(String[] args) {
        InvoiceItem item = new InvoiceItem("A101", "Pen Red", 888, 0.08);

        System.out.println(item);
        item.setQty(999);
        item.setUnitPrice(0.99);
        System.out.println(item);

        System.out.println("id is: " + item.getId());
        System.out.println("desc is: " + item.getDesc());
        System.out.println("qty is: " + item.getQty());
        System.out.println("unitPrice is: " + item.getUnitPrice());
        System.out.println("The total is: " + item.getTotal());
    }
}

```

Result

InvoiceItem[id=A101,desc=Pen Red,qty=888,unitPrice=0.08]

InvoiceItem[id=A101,desc=Pen Red,qty=999,unitPrice=0.99]

id is: A101

desc is: Pen Red

qty is: 999

unitPrice is: 0.99

The total is: 989.01

12.4.4 The Account Class

A class called Account, which models a bank account of a customer, is designed as shown in the following class diagram. The methods credit(amount) and debit(amount) add or subtract the given amount to the balance. The method transferTo(anotherAccount, amount) transfers the given amount from this Account to the given anotherAccount. Write the Account class.

The expected output is:

Account[id=A101,name=Tan Ah Teck,balance=88]

Account[id=A102,name=Kumar,balance=0]

ID: A101

Name: Tan Ah Teck

Balance: 88

Account[id=A101,name=Tan Ah Teck,balance=188]

Account[id=A101,name=Tan Ah Teck,balance=138]

Amount exceeded balance

Account[id=A101,name=Tan Ah Teck,balance=138]

Account[id=A101,name=Tan Ah Teck,balance=38]

Account[id=A102,name=Kumar,balance=100]

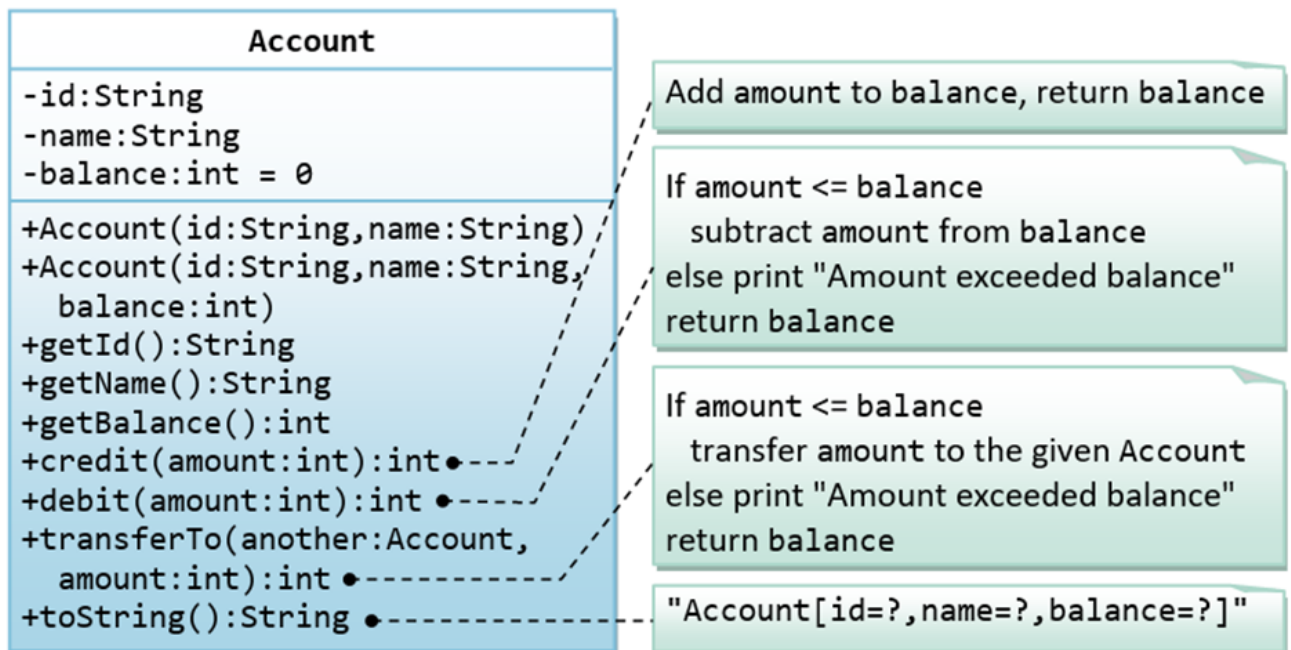
Program

```

/**
 * @author Dr.C V Rama Padmaja

```

Hints:



```
* Associate Professor, Dept of CSE,  
* IARE, Dundigal.  
*/
```

```
public class Account {  
    private String id;  
    private String name;  
    private int balance;  
  
    // Constructor without initial balance  
    public Account(String id, String name) {  
        this.id = id;  
        this.name = name;  
        this.balance = 0;  
    }  
  
    // Constructor with initial balance  
    public Account(String id, String name, int balance) {  
        this.id = id;  
        this.name = name;  
        this.balance = balance;  
    }  
  
    // Getter for ID  
    public String getId() {  
        return id;  
    }  
  
    // Getter for Name  
    public String getName() {  
        return name;  
    }  
}
```

```

// Getter for Balance
public int getBalance() {
    return balance;
}

// Method to add amount to the balance
public int credit(int amount) {
    balance += amount;
    return balance;
}

// Method to subtract amount from the balance
public int debit(int amount) {
    if (amount <= balance) {
        balance -= amount;
    } else {
        System.out.println("Amount exceeded balance");
    }
    return balance;
}

// Method to transfer amount to another account
public int transferTo(Account another, int amount) {
    if (amount <= balance) {
        balance -= amount;
        another.credit(amount);
    } else {
        System.out.println("Amount exceeded balance");
    }
    return balance;
}

@Override
public String toString() {
    return "Account[id=" + id + ",name=" + name + ",balance=" + balance
        + "]\n";
}

public static void main(String[] args) {
    Account account1 = new Account("A101", "Tan Ah Teck", 88);
    Account account2 = new Account("A102", "Kumar");

    System.out.println(account1);
    System.out.println(account2);

    System.out.println("ID: " + account1.getId());
    System.out.println("Name: " + account1.getName());
    System.out.println("Balance: " + account1.getBalance());

    account1.credit(100);
    System.out.println(account1);

    account1.debit(50);
    System.out.println(account1);

    account1.debit(200); // This should print "Amount exceeded balance"
    System.out.println(account1);
}

```

```

        account1.transferTo(account2, 100);
        System.out.println(account1);
        System.out.println(account2);
    }
}

```

Result

```

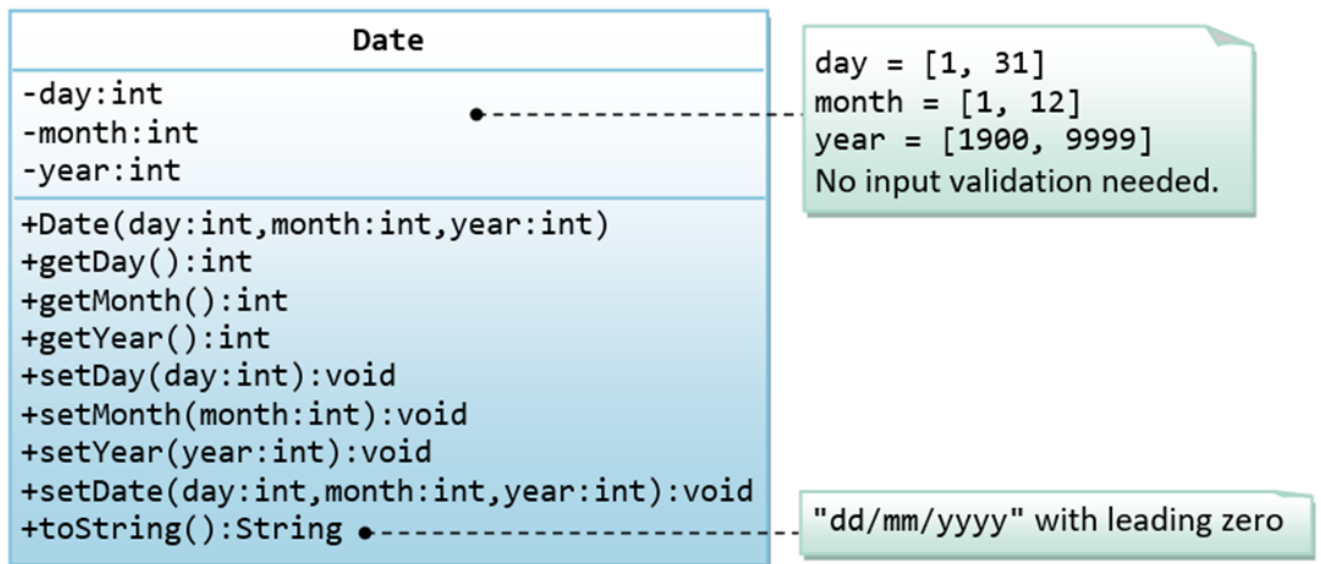
Account[id=A101,name=Tan Ah Teck,balance=88]
Account[id=A102,name=Kumar,balance=0]
ID: A101
Name: Tan Ah Teck
Balance: 88
Account[id=A101,name=Tan Ah Teck,balance=188]
Account[id=A101,name=Tan Ah Teck,balance=138]
Amount exceeded balance
Account[id=A101,name=Tan Ah Teck,balance=138]
Account[id=A101,name=Tan Ah Teck,balance=38]
Account[id=A102,name=Kumar,balance=100]

```

12.4.5 The Date Class

A class called Date, which models a calendar date, is designed as shown in the following class diagram. Write the Date class.

Hints:



The expected output is:

```

01/02/2014
09/12/2099
Month: 12
Day: 9
Year: 2099
03/04/2016

```

Program


```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept of CSE,
 * IARE, Dundigal.
 */;
public class Date {
    private int day;
    private int month;
    private int year;

    // Constructor
    public Date(int day, int month, int year) {
        this.day = day;
        this.month = month;
        this.year = year;
    }

    // Getter for day
    public int getDay() {
        return day;
    }

    // Getter for month
    public int getMonth() {
        return month;
    }

    // Getter for year
    public int getYear() {
        return year;
    }

    // Setter for day
    public void setDay(int day) {
        this.day = day;
    }

    // Setter for month
    public void setMonth(int month) {
        this.month = month;
    }

    // Setter for year
    public void setYear(int year) {
        this.year = year;
    }

    // Method to set the date
    public void setDate(int day, int month, int year) {
        this.day = day;
        this.month = month;
        this.year = year;
    }

    @Override
    public String toString() {
        // Format day, month, and year to have leading zeros if needed
        String dayStr = String.format("%02d", day);

```

```

        String monthStr = String.format("%02d", month);
        String yearStr = String.format("%04d", year);

        return dayStr + "/" + monthStr + "/" + yearStr;
    }

    public static void main(String[] args) {
        Date date1 = new Date(1, 2, 2014);
        Date date2 = new Date(9, 12, 2099);

        System.out.println(date1);
        System.out.println(date2);

        System.out.println("Month: " + date2.getMonth());
        System.out.println("Day: " + date2.getDay());
        System.out.println("Year: " + date2.getYear());

        date1.setDate(3, 4, 2016);
        System.out.println(date1);
    }
}

```

Result

```

01/02/2014
09/12/2099
Month: 12
Day: 9
Year: 2099
03/04/2016

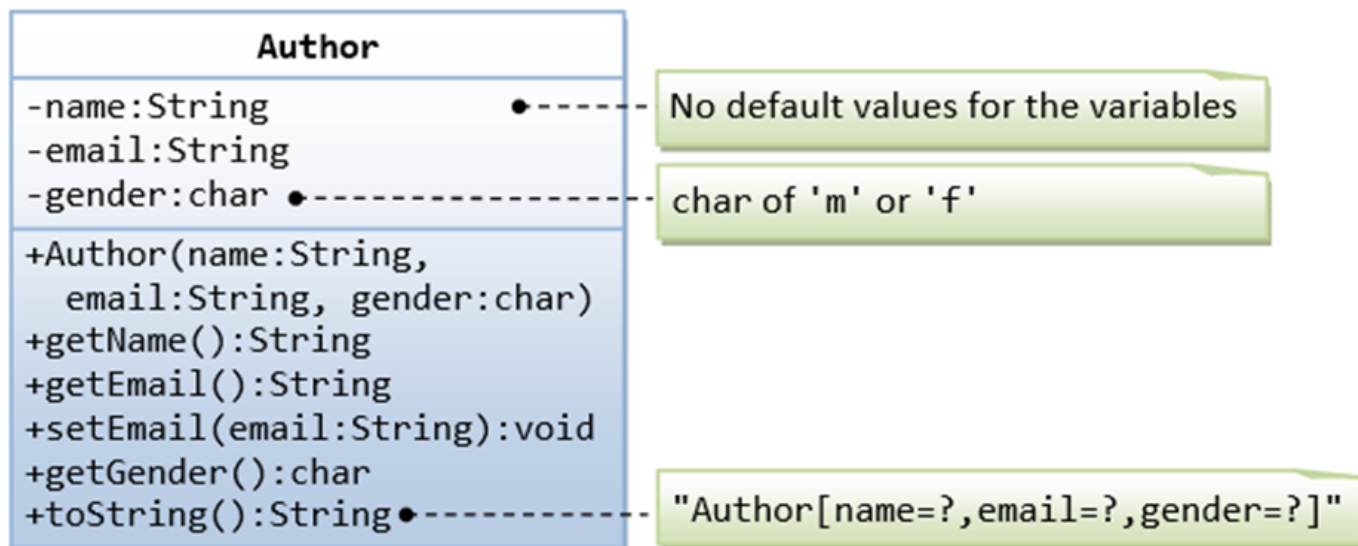
```

12.5 Sample Viva Voce Questions

1. What is the difference between public, private, and protected access modifiers in Java?
2. What is the naming convention for constructors in Java?
3. What are attributes in a class, and how are they different from methods?
4. How do you set the values of object attributes in Java?

12.6 Further Probing Experiments

1. A class called Author, which models a author with name, email and gender, is designed as shown in the following class diagram. Write the Author class.



EXERCISES ON INHERITANCE

13.1 Introduction

In this lab, we will explore the concept of inheritance, a fundamental principle in object-oriented programming (OOP) using Java. Inheritance allows a class to inherit properties and behaviors from another class, enabling code reuse and creating a hierarchical relationship between classes. Through this lab, you will gain a deeper understanding of how inheritance enhances code organization, promotes reusability, and facilitates the creation of robust software solutions in Java.

13.2 Prelab Preparation

13.2.1 Objective

By the end of this lab, students should be able to:

1. **Understanding Inheritance:** Gain a clear understanding of what inheritance is in the context of object-oriented programming and its importance in Java.
2. **Implementing Inheritance:** Learn how to create Java classes that inherit properties and behaviors from parent classes using the `extends` keyword.
3. **Exploring Superclasses and Subclasses:** Understand the concepts of superclass and subclass relationships and how they are established through inheritance.
4. **Overriding Methods:** Explore method overriding and understand how it allows subclasses to provide specific implementations of methods inherited from their superclass.
5. **Using Polymorphism:** Learn how inheritance contributes to polymorphism, allowing objects of a subclass to be treated as objects of their superclass.

13.2.2 Background

Object-oriented programming (OOP) is a programming paradigm that revolves around the concept of objects, which encapsulate data and behavior. Inheritance is a core principle of OOP that facilitates code reuse and promotes the creation of hierarchical relationships between classes.

In Java, inheritance is achieved using the `extends` keyword, which allows a subclass to inherit properties and behaviors from a superclass. This relationship establishes an "is-a" relationship between the subclass and superclass, where the subclass is a more specialized version of the superclass.

Superclasses and subclasses are used to represent generalization-specialization relationships, with superclasses containing common attributes and behaviors shared by subclasses. Subclasses can extend the functionality of their superclass by adding new attributes or behaviors, or by

overriding methods inherited from the superclass.

Method overriding is a key feature of inheritance in Java, where a subclass provides a specific implementation of a method defined in its superclass. This allows subclasses to customize behavior while still maintaining the same method signature as the superclass.

Overall, inheritance in Java plays a crucial role in organizing code, promoting reusability, and facilitating polymorphism, ultimately contributing to the development of scalable and maintainable software systems.

13.2.3 Hardware/Software Requirements

To successfully complete this lab, students will need access to the following:

1. A computer with a suitable text editor or Integrated Development Environment (IDE) installed, such as Eclipse, IntelliJ IDEA, or Visual Studio Code.
2. Java Development Kit (JDK) installed on the computer. Ensure that the JDK is correctly configured, and the system's PATH variable is set to include the JDK's bin directory.

13.2.4 Pre-Lab Questions

1. What is inheritance in Java?
2. How is inheritance implemented in Java?
3. What is method overriding in Java?
4. What is the significance of the super keyword in Java inheritance?

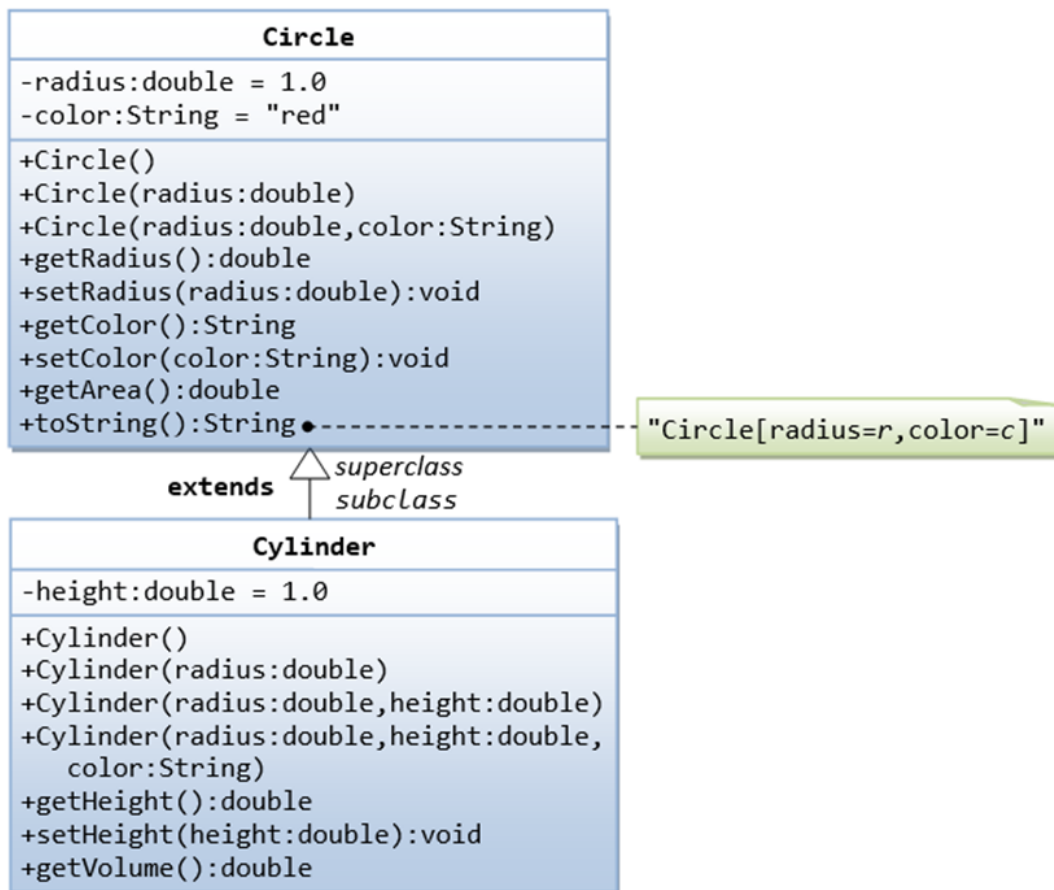
13.3 Experiment Procedure

1. **Setup:** Create a new Java project in your IDE.
2. **Define Classes:** Create a superclass and subclasses with common and specialized attributes and methods.
3. **Implement Inheritance:** Use the extends keyword to establish inheritance relationships.
4. **Explore Super/Subclasses:** Instantiate objects of superclass and subclasses, observe inherited behavior.
5. **Method Overriding:** Override methods in subclasses to provide specific implementations.
6. **Polymorphism:** Demonstrate polymorphism by treating subclass objects as superclass objects.
7. **Testing:** Experiment with different configurations, observe effects on object behavior.
8. **Documentation:** Document observations, challenges, and insights gained.
9. **Conclusion:** Summarize key takeaways regarding inheritance in Java.

13.4 Experiments

13.4.1 The Circle and Cylinder Classes

Aim: In this exercise, a subclass called Cylinder is derived from the superclass Circle as shown in the class diagram (where an arrow pointing up from the subclass to its superclass). Study how the subclass Cylinder invokes the superclass' constructors (via `super()` and `super(radius)`) and inherits the variables and methods from the superclass Circle.



You can reuse the Circle class that you have created in the previous exercise. Make sure that you keep "Circle.class" in the same directory.

Hints

```
private double height; // private variable

// Constructor with default color, radius and height
public Cylinder() {
    super(); // call superclass no-arg constructor Circle()
    height = 1.0;
}
// Constructor with default radius, color but given height
public Cylinder(double height) {
    super(); // call superclass no-arg constructor Circle()
    this.height = height;
}
// Constructor with default color, but given radius, height
public Cylinder(double radius, double height) {
```

```

        super(radius); // call superclass constructor Circle(r)
        this.height = height;
    }

    // A public method for retrieving the height
    public double getHeight() {
        return height;
    }

    // A public method for computing the volume of cylinder
    // use superclass method getArea() to get the base area
    public double getVolume() {
        return getArea()*height;
    }
}

```

Method Overriding and "Super": The subclass Cylinder inherits getArea() method from its superclass Circle. Try overriding the getArea() method in the subclass Cylinder to compute the surface area ($=2\pi \times \text{radius} \times \text{height} + 2 \times \text{base-area}$) of the cylinder instead of base area. That is, if getArea() is called by a Circle instance, it returns the area. If getArea() is called by a Cylinder instance, it returns the surface area of the cylinder.

If you override the getArea() in the subclass Cylinder, the getVolume() no longer works. This is because the getVolume() uses the overridden getArea() method found in the same class. (Java runtime will search the superclass only if it cannot locate the method in this class). Fix the getVolume().

Hints:

After overriding the getArea() in subclass Cylinder, you can choose to invoke the getArea() of the superclass Circle by calling super.getArea().

TRY

Provide a toString() method to the Cylinder class, which overrides the toString() inherited from the superclass Circle, e.g.,

```

@Override
public String toString() {          // in Cylinder class
    return "Cylinder: subclass of " + super.toString() // use Circle's
        toString()
        + " height=" + height;
}

```

Try out the toString() method in TestCylinder.

Program

Circle.java

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

// Circle class
public class Circle {

```

```

private double radius;
private String color;

public Circle() {
    radius = 1.0;
    color = "red";
}

public Circle(double radius) {
    this.radius = radius;
    color = "red";
}

public Circle(double radius, String color) {
    this.radius = radius;
    this.color = color;
}

public double getRadius() {
    return radius;
}

public void setRadius(double radius) {
    this.radius = radius;
}

public String getColor() {
    return color;
}

public void setColor(String color) {
    this.color = color;
}

public double getArea() {
    return Math.PI * radius * radius;
}

@Override
public String toString() {
    return "Circle[radius=" + radius + ",color=" + color + "]";
}
}

```

Cylinder.java

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

// Cylinder class
public class Cylinder extends Circle {
    private double height;

    public Cylinder() {
        super();
    }
}

```



```

        height = 1.0;
    }

    public Cylinder(double height) {
        super();
        this.height = height;
    }

    public Cylinder(double radius, double height) {
        super(radius);
        this.height = height;
    }

    public Cylinder(double radius, double height, String color) {
        super(radius, color);
        this.height = height;
    }

    public double getHeight() {
        return height;
    }

    @Override
    public double getArea() {
        // Override getArea() to compute the surface area of the cylinder
        return 2 * Math.PI * getRadius() * (getRadius() + height);
    }

    public double getVolume() {
        // Fix getVolume() to use the superclass getArea() method
        return super.getArea() * height;
    }

    @Override
    public String toString() {
        // Override toString() to include information from the superclass
        return "Cylinder: subclass of " + super.toString() + " height=" +
            height;
    }

    public static void main(String[] args) {
        // Test the Circle class
        Circle circle = new Circle(2.0, "blue");
        System.out.println(circle);
        System.out.println("Area: " + circle.getArea());

        // Test the Cylinder class
        Cylinder cylinder = new Cylinder(2.0, 3.0, "green");
        System.out.println(cylinder);
        System.out.println("Area: " + cylinder.getArea());
        System.out.println("Volume: " + cylinder.getVolume());
    }
}

```

TestCylinder.java

```

/**
 * @author Dr.C V Rama Padmaja

```

```

* Associate Professor, Dept CSE
* IARE, Dundigal.
*/
public class TestCylinder {
    public static void main(String[] args) {
        // Test the Circle class
        Circle circle = new Circle(2.0, "blue");
        System.out.println(circle);
        System.out.println("Area: " + circle.getArea());

        // Test the Cylinder class
        Cylinder cylinder = new Cylinder(2.0, 3.0, "green");
        System.out.println(cylinder);
        System.out.println("Area: " + cylinder.getArea());
        System.out.println("Volume: " + cylinder.getVolume());
    }
}

```

Result

```

Circle[radius=2.0,color=blue]
Area: 12.566370614359172
Cylinder: subclass of Circle[radius=2.0,color=green] height=3.0
Area: 62.83185307179586
Volume: 37.69911184307752

```

13.4.2 Superclass Person and its subclasses

Aim: Write the classes as shown in the following class diagram. Mark all the overridden methods with annotation @Override.

Program

Person.java

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

// Person class
public class Person {
    private String name;
    private String address;

    public Person(String name, String address) {
        this.name = name;
        this.address = address;
    }

    public String getName() {
        return name;
    }
}

```

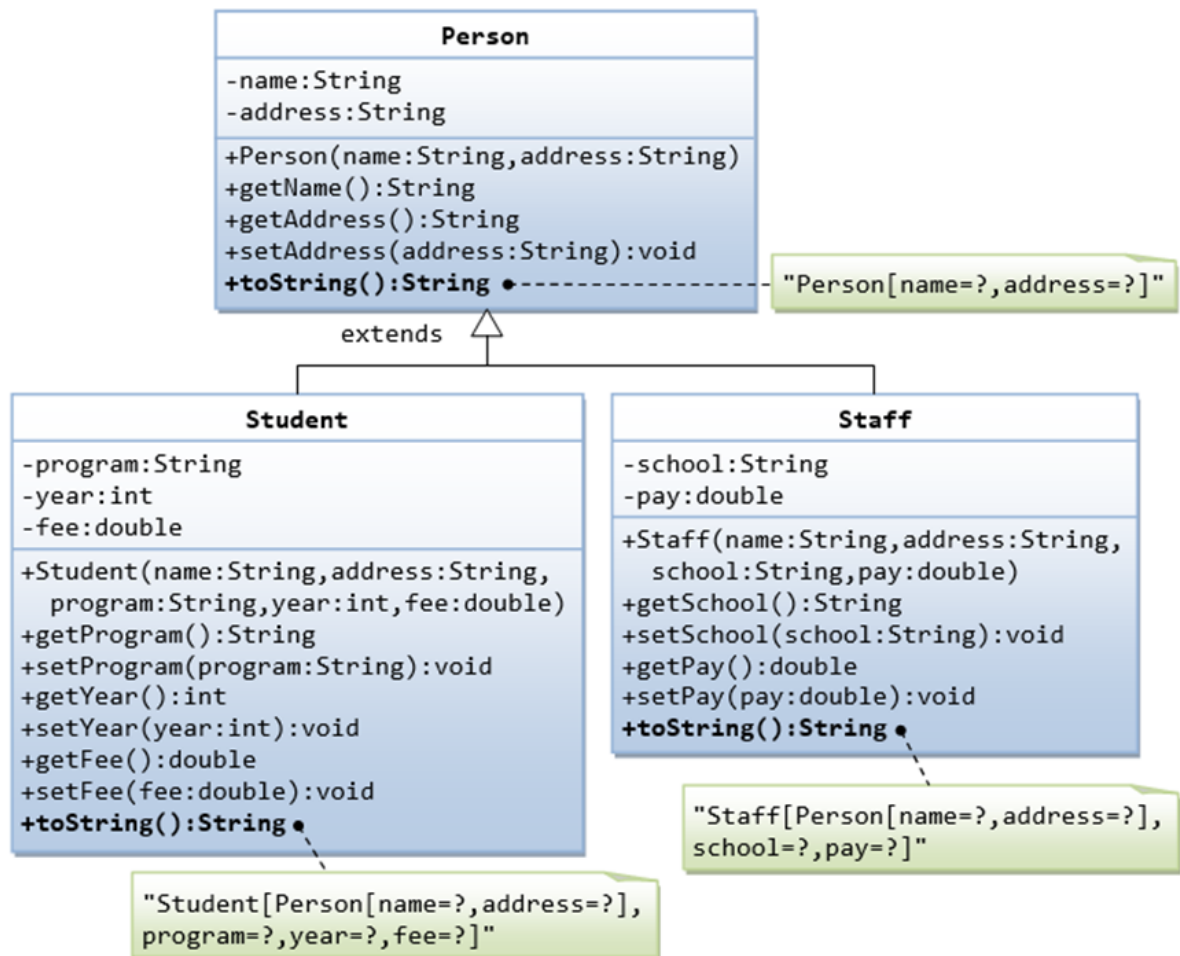
```

public String getAddress() {
    return address;
}

public void setAddress(String address) {
    this.address = address;
}

@Override
public String toString() {
    return "Person[name=" + name + ", address=" + address + "];"
}
}

```



Student.java

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

// Student class (subclass of Person)
public class Student extends Person {
    private String program;
    private int year;

```

```

private double fee;

public Student(String name, String address, String program, int year,
    double fee) {
    super(name, address);
    this.program = program;
    this.year = year;
    this.fee = fee;
}

public String getProgram() {
    return program;
}

public void setProgram(String program) {
    this.program = program;
}

public int getYear() {
    return year;
}

public void setYear(int year) {
    this.year = year;
}

public double getFee() {
    return fee;
}

public void setFee(double fee) {
    this.fee = fee;
}

@Override
public String toString() {
    return "Student[" + super.toString() + ", program=" + program + ",
        year=" + year + ", fee=" + fee + "]";
}
}

```

Staff.java

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

// Staff class (subclass of Person)
public class Staff extends Person {
    private String school;
    private double pay;

    public Staff(String name, String address, String school, double pay) {
        super(name, address);
        this.school = school;
        this.pay = pay;
    }
}

```

```

    }

    public String getSchool() {
        return school;
    }

    public void setSchool(String school) {
        this.school = school;
    }

    public double getPay() {
        return pay;
    }

    public void setPay(double pay) {
        this.pay = pay;
    }

    @Override
    public String toString() {
        return "Staff[" + super.toString() + ", school=" + school + ", pay"
            + " + pay + " ]";
    }
}

```

TestPerson.java

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

// TestPerson class
public class TestPerson {
    public static void main(String[] args) {
        Person person = new Person("John Doe", "123 Main St");
        Student student = new Student("Alice Smith", "456 Oak St", "
            Computer Science", 2, 1500.0);
        Staff staff = new Staff("Bob Johnson", "789 Pine St", "High School
            ", 3000.0);

        System.out.println(person);
        System.out.println(student);
        System.out.println(staff);
    }
}

```

Result

Person[name=John Doe, address=123 Main St]
 Student[Person[name=Alice Smith, address=456 Oak St], program=Computer Science, year=2,
 fee=1500.0]
 Staff[Person[name=Bob Johnson, address=789 Pine St], school=High School, pay=3000.0]

13.4.3 Point2D and Point3D

Aim: Write the classes as shown in the following class diagram. Mark all the overridden methods with annotation `@Override`.

Hints:

1. You cannot assign floating-point literal say 1.1 (which is a double) to a float variable, you need to add a suffix f, e.g. 0.0f, 1.1f.
2. The instance variables x and y are private in Point2D and cannot be accessed directly in the subclass Point3D. You need to access via the public getters and setters. For example,

```
public void setXYZ(float x, float y, float z) {
    setX(x);      // or super.setX(x), use setter in superclass
    setY(y);
    this.z = z;
}
```

3. The method `getXY()` shall return a float array

```
public float[] getXY() {
    float[] result = new float[2]; // construct an array of 2
    elements
    result[0] = ...
    result[1] = ...
    return result; // return the array
}
```

Program

Point2D

```
/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

// Point2D class
public class Point2D {
    private float x;
    private float y;

    public Point2D(float x, float y) {
        this.x = x;
        this.y = y;
    }

    public Point2D() {
        this(0.0f, 0.0f);
    }

    public float getX() {
        return x;
    }

    public void setX(float x) {
        this.x = x;
    }
}
```

```

    }

    public float getY() {
        return y;
    }

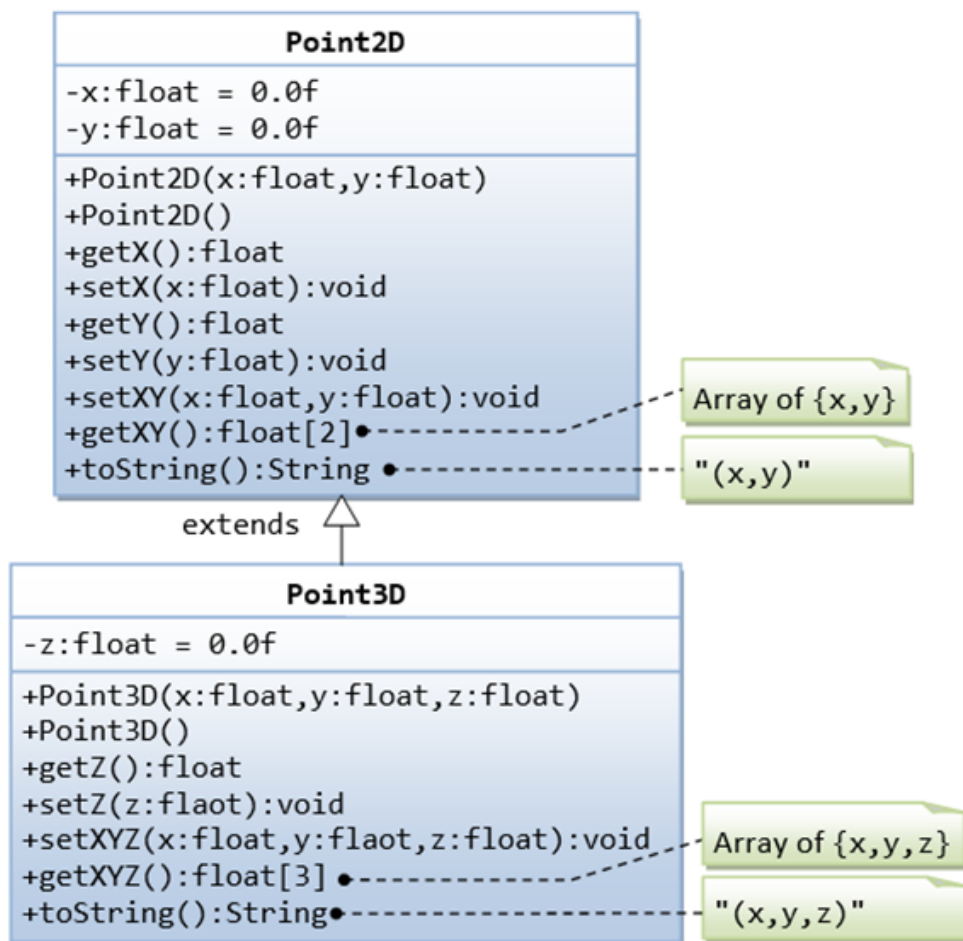
    public void setY(float y) {
        this.y = y;
    }

    public float[] getXY() {
        return new float[]{x, y};
    }

    public void setXY(float x, float y) {
        this.x = x;
        this.y = y;
    }

    @Override
    public String toString() {
        return "(" + x + "," + y + ")";
    }
}

```



Point3D

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

// Point3D class (subclass of Point2D)
public class Point3D extends Point2D {
    private float z;

    public Point3D(float x, float y, float z) {
        super(x, y);
        this.z = z;
    }

    public Point3D() {
        this(0.0f, 0.0f, 0.0f);
    }

    public float getZ() {
        return z;
    }

    public void setZ(float z) {
        this.z = z;
    }

    public void setXYZ(float x, float y, float z) {
        setX(x);
        setY(y);
        this.z = z;
    }

    public float[] getXYZ() {
        return new float[]{getX(), getY(), z};
    }

    @Override
    public String toString() {
        return "(" + getX() + "," + getY() + "," + z + ")";
    }
}

```

TestPoint3D

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

public class TestPoint3D {
    public static void main(String[] args) {
        // Testing Point2D
        Point2D point2D = new Point2D(2.5f, 3.0f);
        System.out.println("Point2D: " + point2D);
        System.out.println("X: " + point2D.getX());
    }
}

```



```

        System.out.println("Y: " + point2D.getY());
        System.out.println("XY Array: " + arrayToString(point2D.getXY()));

        // Testing Point3D
        Point3D point3D = new Point3D(1.0f, 2.0f, 3.0f);
        System.out.println("\nPoint3D: " + point3D);
        System.out.println("Z: " + point3D.getZ());
        System.out.println("XYZ Array: " + arrayToString(point3D.getXYZ()));
        ;

        // Testing setXYZ method
        point3D.setXYZ(4.0f, 5.0f, 6.0f);
        System.out.println("\nAfter setXYZ: " + point3D);

        // Testing Point2D methods on Point3D
        Point2D pointAs2D = point3D; // Upcasting
        System.out.println("\nPoint3D as Point2D: " + pointAs2D);
    }

    // Helper method to convert array to String for printing
    private static String arrayToString(float[] arr) {
        StringBuilder result = new StringBuilder("[");
        for (float value : arr) {
            result.append(value).append(", ");
        }
        result.delete(result.length() - 2, result.length()); // Removing
            the trailing comma and space
        result.append("]");
        return result.toString();
    }
}

```

Result

Point2D: (2.5,3.0)

X: 2.5

Y: 3.0

XY Array: [2.5, 3.0]

Point3D: (1.0,2.0,3.0)

Z: 3.0

XYZ Array: [1.0, 2.0, 3.0]

After setXYZ: (4.0,5.0,6.0)

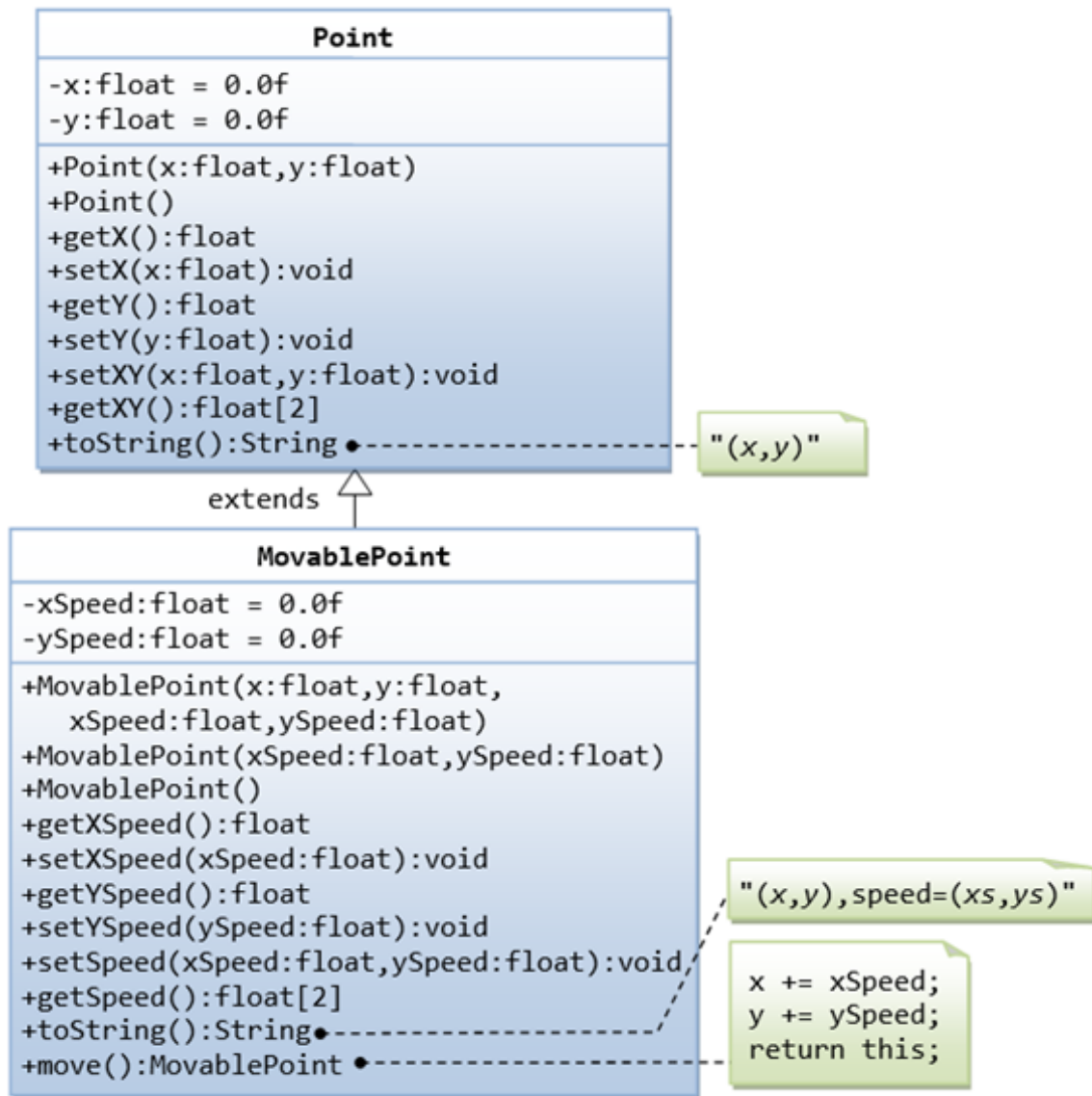
Point3D as Point2D: (4.0,5.0,6.0)

13.4.4 Point and MovablePoint

Aim: Write the classes as shown in the following class diagram. Mark all the overridden methods with annotation `@Override`.

Hints:

1. You cannot assign floating-point literal say 1.1 (which is a double) to a float variable, you need to add a suffix f, e.g. 0.0f, 1.1f.
2. The instance variables x and y are private in Point and cannot be accessed directly in the subclass MovablePoint. You need to access via the public getters and setters. For example, you cannot write `x += xSpeed`, you need to write `setX(getX() + xSpeed)`.



Program

Point.java

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

// Point class
  
```

```

public class Point {
    private float x;
    private float y;

    public Point(float x, float y) {
        this.x = x;
        this.y = y;
    }

    public Point() {
        this(0.0f, 0.0f);
    }

    public float getX() {
        return x;
    }

    public void setX(float x) {
        this.x = x;
    }

    public float getY() {
        return y;
    }

    public void setY(float y) {
        this.y = y;
    }

    public float[] getXY() {
        return new float[]{x, y};
    }

    public void setXY(float x, float y) {
        this.x = x;
        this.y = y;
    }

    @Override
    public String toString() {
        return "(" + x + "," + y + ")";
    }
}

```

MovablePoint.java

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */
// MovablePoint class
public class MovablePoint extends Point {
    private float xSpeed;
    private float ySpeed;

    public MovablePoint(float x, float y, float xSpeed, float ySpeed) {
        super(x, y);
    }
}

```

```

        this.xSpeed = xSpeed;
        this.ySpeed = ySpeed;
    }

    public MovablePoint(float xSpeed, float ySpeed) {
        this(0.0f, 0.0f, xSpeed, ySpeed);
    }

    public MovablePoint() {
        this(0.0f, 0.0f, 0.0f, 0.0f);
    }

    public float getXSpeed() {
        return xSpeed;
    }

    public void setXSpeed(float xSpeed) {
        this.xSpeed = xSpeed;
    }

    public float getYSpeed() {
        return ySpeed;
    }

    public void setYSpeed(float ySpeed) {
        this.ySpeed = ySpeed;
    }

    public float[] getSpeed() {
        return new float[]{xSpeed, ySpeed};
    }

    public void setSpeed(float xSpeed, float ySpeed) {
        this.xSpeed = xSpeed;
        this.ySpeed = ySpeed;
    }

    @Override
    public String toString() {
        return super.toString() + ",speed=(" + xSpeed + "," + ySpeed + ")";
    }

    public MovablePoint move() {
        setX(getX() + xSpeed);
        setY(getY() + ySpeed);
        return this;
    }
}

```

TestMovablePoint.java

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

public class TestMovablePoint {

```

```

public static void main(String[] args) {
    Point point = new Point(2.5f, 3.5f);
    System.out.println("Point: " + point);
    System.out.println("X coordinate: " + point.getX());
    System.out.println("Y coordinate: " + point.getY());
    System.out.println("Coordinates array: " + arrayToString(point.
        getXY()));

    MovablePoint movablePoint = new MovablePoint(1.5f, 2.5f, 0.5f, 1.5f
    );
    System.out.println("\nMovablePoint: " + movablePoint);
    System.out.println("X Speed: " + movablePoint.getXSpeed());
    System.out.println("Y Speed: " + movablePoint.getYSpeed());
    System.out.println("Speed array: " + arrayToString(movablePoint.
        getSpeed()));

    System.out.println("\nMoving MovablePoint...");
    movablePoint.move();
    System.out.println("Updated MovablePoint: " + movablePoint);
}

private static String arrayToString(float[] array) {
    StringBuilder result = new StringBuilder("[");
    for (float value : array) {
        result.append(value).append(", ");
    }
    result.delete(result.length() - 2, result.length()).append("]");
    return result.toString();
}
}

```

Result

Point: (2.5,3.5)

X coordinate: 2.5

Y coordinate: 3.5

Coordinates array: [2.5, 3.5]

MovablePoint: (1.5,2.5),speed=(0.5,1.5)

X Speed: 0.5

Y Speed: 1.5

Speed array: [0.5, 1.5]

Moving MovablePoint...

Updated MovablePoint: (2.0,4.0),speed=(0.5,1.5)

13.4.5 Superclass Shape and its subclasses Circle, Rectangle and Square

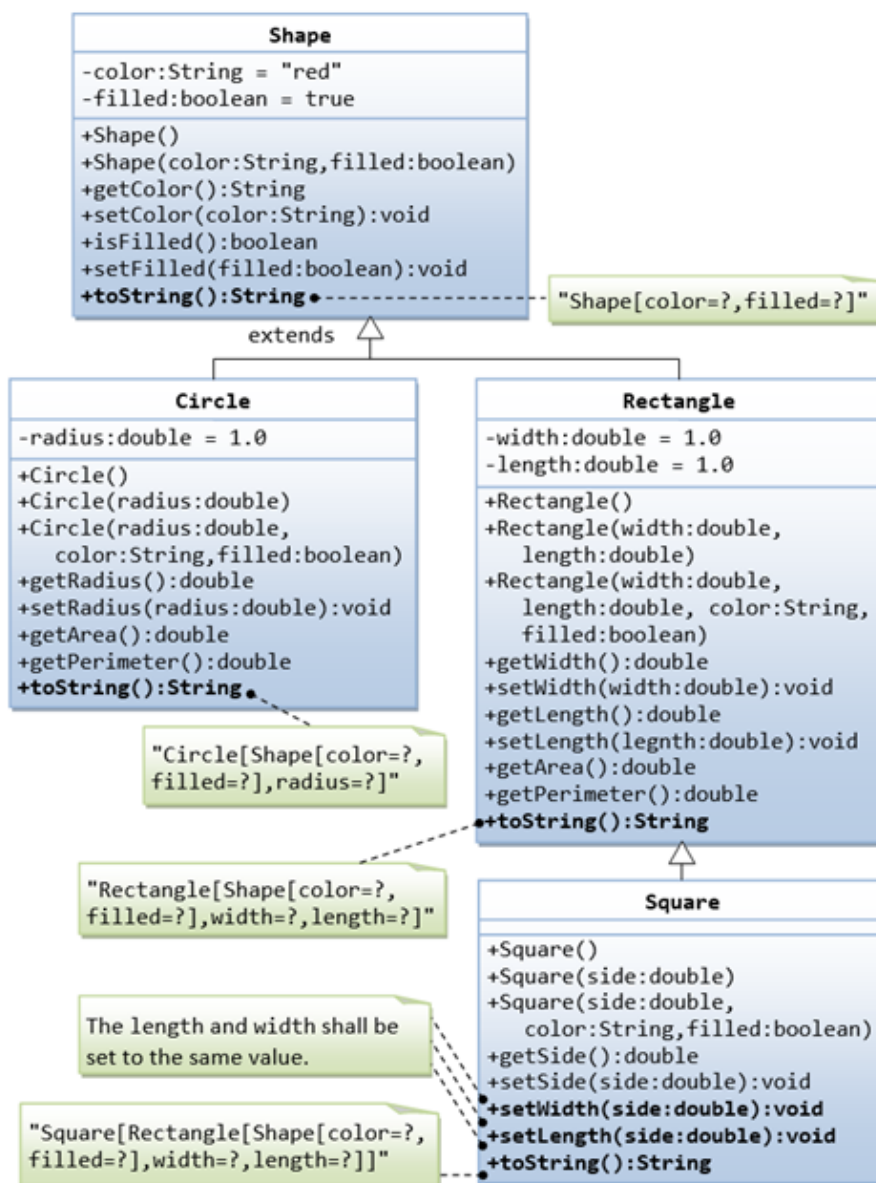
Aim: Write a superclass called Shape (as shown in the class diagram) and a test program to test all the methods defined in Shape. Write two subclasses of Shape called Circle and Rectangle, as shown in the class diagram.

Write a class called Square, as a subclass of Rectangle. Convince yourself that Square can be modeled as a subclass of Rectangle. Square has no instance variable, but inherits the instance variables width and length from its superclass Rectangle. Provide the appropriate constructors (as shown in the class diagram).

Hints:

```
public Square(double side) {  
    super(side, side); // Call superclass Rectangle(double, double)  
}
```

1. Override the toString() method to return "A Square with side=xxx, which is a subclass of yyy", where yyy is the output of the toString() method from the superclass.
2. Do you need to override the getArea() and getPerimeter()? Try them out.
3. Override the setLength() and setWidth() to change both the width and length, so as to maintain the square geometry.



Program

Shape.java

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */
public class Shape {
    private String color;
    private boolean filled;

    // Default constructor
    public Shape() {
        this.color = "red";
        this.filled = true;
    }

    // Parameterized constructor
    public Shape(String color, boolean filled) {
        this.color = color;
        this.filled = filled;
    }

    // Getter for color
    public String getColor() {
        return color;
    }

    // Setter for color
    public void setColor(String color) {
        this.color = color;
    }

    // Getter for filled
    public boolean isFilled() {
        return filled;
    }

    // Setter for filled
    public void setFilled(boolean filled) {
        this.filled = filled;
    }

    // toString method
    public String toString() {
        return "Shape[color=" + color + ",filled=" + filled + "]";
    }
}

```

Circle2.java

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

public class Circle2 extends Shape {
    private double radius;
}

```

```

// Constructors
public Circle2() {
    this.radius = 1.0;
}

public Circle2(double radius) {
    this.radius = radius;
}

public Circle2(double radius, String color, boolean filled) {
    super(color, filled);
    this.radius = radius;
}

// Getter for radius
public double getRadius() {
    return radius;
}

// Setter for radius
public void setRadius(double radius) {
    this.radius = radius;
}

// Methods to calculate area and perimeter
public double getArea() {
    return Math.PI * radius * radius;
}

public double getPerimeter() {
    return 2 * Math.PI * radius;
}

// toString method
public String toString() {
    return "Circle[" + super.toString() + ",radius=" + radius + "];"
}
}

```

Rectangle.java

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

public class Rectangle extends Shape {
    private double width;
    private double length;

    // Constructors
    public Rectangle() {
        this.width = 1.0;
        this.length = 1.0;
    }
}

```



```

public Rectangle(double width, double length) {
    this.width = width;
    this.length = length;
}

public Rectangle(double width, double length, String color, boolean
    filled) {
    super(color, filled);
    this.width = width;
    this.length = length;
}

// Getters and setters for width and length
public double getWidth() {
    return width;
}

public void setWidth(double width) {
    this.width = width;
}

public double getLength() {
    return length;
}

public void setLength(double length) {
    this.length = length;
}

// Methods to calculate area and perimeter
public double getArea() {
    return width * length;
}

public double getPerimeter() {
    return 2 * (width + length);
}

// toString method
public String toString() {
    return "Rectangle[" + super.toString() + ",width=" + width + ",
        length=" + length + "];"
}
}

```

Square.java

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

public class Square extends Rectangle {
    // Constructors
    public Square() {
        super();
    }
}

```

```

public Square(double side) {
    super(side, side);
}

public Square(double side, String color, boolean filled) {
    super(side, side, color, filled);
}

// Getter and setter for side
public double getSide() {
    return getWidth(); // or getLength() since they are the same
}

public void setSide(double side) {
    setWidth(side);
    setLength(side);
}

// toString method
public String toString() {
    return "Square[" + super.toString() + "]\n";
}
}

```

TestShapes.java

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

public class TestShapes {
    public static void main(String[] args) {
        // Test Shape class
        Shape shape = new Shape("green", false);
        System.out.println("Shape: " + shape);
        System.out.println();

        // Test Circle class
        Circle2 circle = new Circle2(5.0, "blue", true);
        System.out.println("Circle: " + circle);
        System.out.println("Area: " + circle.getArea());
        System.out.println("Perimeter: " + circle.getPerimeter());
        System.out.println();

        // Test Rectangle class
        Rectangle rectangle = new Rectangle(4.0, 6.0, "yellow", true);
        System.out.println("Rectangle: " + rectangle);
        System.out.println("Area: " + rectangle.getArea());
        System.out.println("Perimeter: " + rectangle.getPerimeter());
        System.out.println();

        // Test Square class
        Square square = new Square(3.0, "purple", true);
        System.out.println("Square: " + square);
        System.out.println("Area: " + square.getArea());
    }
}

```

```

        System.out.println("Perimeter: " + square.getPerimeter());
        System.out.println();
    }
}

```

Result

Shape: Shape[color=green, filled=false]

Circle: Circle[Shape[color=blue, filled=true], radius=5.0]

Area: 78.53981633974483

Perimeter: 31.41592653589793

Rectangle: Rectangle[Shape[color=yellow, filled=true], width=4.0, length=6.0]

Area: 24.0

Perimeter: 20.0

Square: Square[Rectangle[Shape[color=purple, filled=true], width=3.0, length=3.0]]

Area: 9.0

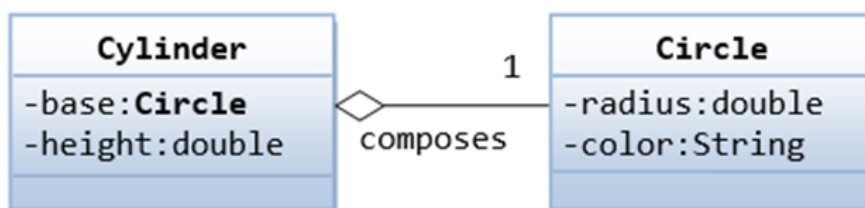
Perimeter: 12.0

13.5 Sample Viva Voce Questions

1. How does inheritance promote code reuse in Java?
2. What is the difference between superclass and subclass in Java?
3. How is method overriding achieved in Java?
4. How does inheritance contribute to polymorphism in Java?

13.6 Further Probing Experiments

1. Design a MyRectangle class which is composed of two MyPoint instances as its top-left and bottom-right corners. Draw the class diagrams, write the codes, and write the test drivers.
2. The Circle and Cylinder Classes Using Composition
Try rewriting the Circle-Cylinder of the previous exercise using composition (as shown in the class diagram) instead of inheritance. That is, "a cylinder is composed of a base circle and a height".



Which design (inheritance or composition) is better?

EXERCISES ON POLYMORPHISM, ABSTRACT CLASSES AND INTERFACES

14.1 Introduction

In this lab, our primary focus will be on mastering advanced object-oriented programming concepts in Java, specifically polymorphism, abstract classes, and interfaces. These concepts play a pivotal role in creating Java applications that are not only flexible and scalable but also easy to maintain. Throughout a series of engaging exercises and examples, this lab is designed to sharpen your skills in polymorphism, provide a deeper understanding of abstract classes, and showcase the power of interfaces. Let's dive into the heart of these concepts and discover how they contribute to the development of resilient and adaptable software solutions.

14.2 Prelab Preparation

14.2.1 Objective

By the end of this lab, students should be able to:

1. **Understanding Polymorphism:** Gain a comprehensive understanding of polymorphism and its types, including compile-time polymorphism (method overloading) and runtime polymorphism (method overriding).
2. **Implementing Polymorphism:** Learn how to leverage polymorphism to create flexible and extensible Java code that can adapt to different data types and scenarios.
3. **Exploring Abstract Classes:** Understand the concept of abstract classes and how they serve as blueprints for concrete subclasses, allowing for code reuse and abstraction.
4. **Using Abstract Classes:** Explore how abstract classes can be used to define common behavior and attributes shared among related classes while leaving specific implementations to concrete subclasses.
5. **Understanding Interfaces:** Gain insight into the concept of interfaces and their role in defining contracts for classes, promoting loose coupling and flexibility in Java applications.
6. **Implementing Interfaces:** Learn how to define and implement interfaces in Java to provide a common set of methods that can be implemented by multiple classes, enabling polymorphism and code reuse.

14.2.2 Background

Object-oriented programming (OOP) is a programming paradigm centered around the concept of objects, which encapsulate data and behavior. Polymorphism, abstract classes, and interfaces are advanced OOP concepts that enhance code organization, reusability, and flexibility.

Polymorphism allows objects to be treated as instances of their superclass or interface, enabling dynamic method invocation and code flexibility. Method overloading and method overriding are two forms of polymorphism in Java, achieved through compile-time and runtime polymorphism, respectively.

Abstract classes serve as templates or blueprints for concrete subclasses, defining common behavior and attributes shared among related classes. They cannot be instantiated directly but can be extended by concrete subclasses, which provide specific implementations for abstract methods.

Interfaces provide a way to define contracts for classes, specifying a set of methods that implementing classes must provide. Interfaces enable loose coupling and flexibility in Java applications, allowing classes to interact without being tightly coupled to each other's implementations.

In this lab, you will gain hands-on experience implementing polymorphism, abstract classes, and interfaces in Java through a series of exercises and examples, deepening your understanding of these important concepts and their practical applications in software development.

14.2.3 Hardware/Software Requirements

To successfully complete this lab, students will need access to the following:

1. A computer with a suitable text editor or Integrated Development Environment (IDE) installed, such as Eclipse, IntelliJ IDEA, or Visual Studio Code.
2. Java Development Kit (JDK) installed on the computer. Ensure that the JDK is correctly configured, and the system's PATH variable is set to include the JDK's bin directory.

14.2.4 Pre-Lab Questions

1. What is polymorphism in Java, and why is it important in object-oriented programming?
2. Differentiate between compile-time polymorphism and runtime polymorphism in Java.
3. Can you provide an example of method overloading and method overriding in Java?
4. What is an interface in Java, and how does it differ from an abstract class?

14.3 Experiment Procedure

1. **Setup:** Create Java project "PolymorphismLab" in IDE.
2. **Polymorphism:** Create class with overloaded methods, extend class and override methods for runtime polymorphism.
3. **Abstract Classes:** Define abstract class with abstract and concrete methods, create subclasses with specific implementations.
4. **Interfaces:** Define interface with methods, implement in multiple classes, demonstrate interface-based polymorphism.
5. **Combined Exercise:** Design system using polymorphism, abstract classes, and interfaces.
6. **Testing:** Verify implementations, ensure correctness and adherence to Java conventions.

7. **Documentation:** Document process, challenges, and observations.
8. **Conclusion:** Summarize learnings and practical applications of polymorphism, abstract classes, and interfaces in Java.

14.4 Experiemnts

14.5 Experiments

14.5.1 Abstract Superclass Shape and Its Concrete Subclasses

Aim: Rewrite the superclass Shape and its subclasses Circle, Rectangle and Square, as shown in the class diagram.

Shape is an abstract class containing 2 abstract methods: `getArea()` and `getPerimeter()`, where its concrete subclasses must provide its implementation. All instance variables shall have protected access, i.e., accessible by its subclasses and classes in the same package. Mark all the overridden methods with annotation `@Override`.

In this exercise, Shape shall be defined as an abstract class, which contains:

- Two protected instance variables `color(String)` and `filled(boolean)`. The protected variables can be accessed by its subclasses and classes in the same package. They are denoted with a '#' sign in the class diagram.
- Getter and setter for all the instance variables, and `toString()`.
- Two abstract methods `getArea()` and `getPerimeter()` (shown in italics in the class diagram).

The subclasses Circle and Rectangle shall override the abstract methods `getArea()` and `getPerimeter()` and provide the proper implementation. They also override the `toString()`.

Write a test class to test these statements involving polymorphism and explain the outputs. Some statements may trigger compilation errors. Explain the errors, if any.

```
Shape s1 = new Circle(5.5, "red", false); // Upcast Circle to Shape
System.out.println(s1);                  // which version?
System.out.println(s1.getArea());         // which version?
System.out.println(s1.getPerimeter());    // which version?
System.out.println(s1.getColor());
System.out.println(s1.isFilled());
System.out.println(s1.getRadius());

Circle c1 = (Circle)s1;                   // Downcast back to Circle
System.out.println(c1);
System.out.println(c1.getArea());
System.out.println(c1.getPerimeter());
System.out.println(c1.getColor());
System.out.println(c1.isFilled());
System.out.println(c1.getRadius());

Shape s2 = new Shape();

Shape s3 = new Rectangle(1.0, 2.0, "red", false); // Upcast
System.out.println(s3);
System.out.println(s3.getArea());
System.out.println(s3.getPerimeter());
System.out.println(s3.getColor());
System.out.println(s3.getLength());
```



```

Rectangle r1 = (Rectangle)s3;    // downcast
System.out.println(r1);
System.out.println(r1.getArea());
System.out.println(r1.getColor());
System.out.println(r1.getLength());

Shape s4 = new Square(6.6);      // Upcast
System.out.println(s4);
System.out.println(s4.getArea());
System.out.println(s4.getColor());
System.out.println(s4.getSide());

// Take note that we downcast Shape s4 to Rectangle,
// which is a superclass of Square, instead of Square
Rectangle r2 = (Rectangle)s4;
System.out.println(r2);
System.out.println(r2.getArea());
System.out.println(r2.getColor());
System.out.println(r2.getSide());
System.out.println(r2.getLength());

// Downcast Rectangle r2 to Square
Square sq1 = (Square)r2;
System.out.println(sq1);
System.out.println(sq1.getArea());
System.out.println(sq1.getColor());
System.out.println(sq1.getSide());
System.out.println(sq1.getLength());

```

Program

Shape.java

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

public abstract class Shape {
    // Data members
    private String color = "red";
    private boolean filled = true;

    // Constructors
    public Shape() {
    }

    public Shape(String color, boolean filled) {
        this.color = color;
        this.filled = filled;
    }

    // Getter and setter methods
    public String getColor() {
        return color;
    }

    public void setColor(String color) {
    }

```



```

        this.color = color;
    }

    public boolean isFilled() {
        return filled;
    }

    public void setFilled(boolean filled) {
        this.filled = filled;
    }

    // Abstract methods
    public abstract double getArea();

    public abstract double getPerimeter();

    // toString method
    @Override
    public String toString() {
        return "Shape[color=" + color + ",filled=" + filled + "];"
    }
}

```

Circle.java

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

// Circle class
public class Circle extends Shape {
    // Data member
    private double radius = 1.0;

    // Constructors
    public Circle() {
    }

    public Circle(double radius) {
        this.radius = radius;
    }

    public Circle(double radius, String color, boolean filled) {
        super(color, filled);
        this.radius = radius;
    }

    // Getter and setter methods for radius
    public double getRadius() {
        return radius;
    }

    public void setRadius(double radius) {
        this.radius = radius;
    }

    // Override abstract methods

```

```

@Override
public double getArea() {
    return Math.PI * radius * radius;
}

@Override
public double getPerimeter() {
    return 2 * Math.PI * radius;
}

// Override toString method
@Override
public String toString() {
    return "Circle[" + super.toString() + ",radius=" + radius +
        "]\n";
}
}

```

Rectangle.java

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

// Rectangle class
public class Rectangle extends Shape {
    // Data members
    private double width = 1.0;
    private double length = 1.0;

    // Constructors
    public Rectangle() {
    }

    public Rectangle(double width, double length) {
        this.width = width;
        this.length = length;
    }

    public Rectangle(double width, double length, String color, boolean
        filled) {
        super(color, filled);
        this.width = width;
        this.length = length;
    }

    // Getter and setter methods for width and length
    public double getWidth() {
        return width;
    }

    public void setWidth(double width) {
        this.width = width;
    }

    public double getLength() {
        return length;
    }
}

```

```

    }

    public void setLength(double length) {
        this.length = length;
    }

    // Override abstract methods
    @Override
    public double getArea() {
        return width * length;
    }

    @Override
    public double getPerimeter() {
        return 2 * (width + length);
    }

    // Override toString method
    @Override
    public String toString() {
        return "Rectangle[" + super.toString() + ",width=" + width + ",
            length=" + length + "]";
    }
}

```

Square.java

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

// Square class
public class Square extends Rectangle {
    // Constructors
    public Square() {
    }

    public Square(double side) {
        super(side, side);
    }

    public Square(double side, String color, boolean filled) {
        super(side, side, color, filled);
    }

    // Getter and setter methods for side
    public double getSide() {
        return getWidth(); // or getLength(), since they are the same
        for a square
    }

    public void setSide(double side) {
        setWidth(side);
        setLength(side);
    }

    // Override setWidth and setLength to ensure they are the same for

```

```

        a square
@Override
public void setWidth(double side) {
    super.setWidth(side);
    super.setLength(side);
}

@Override
public void setLength(double side) {
    super.setWidth(side);
    super.setLength(side);
}

// Override toString method
@Override
public String toString() {
    return "Square[" + super.toString() + "];"
}
}

```

TestShape.java

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

public class TestShape {
    public static void main(String[] args) {
        Shape s1 = new Circle(5.5, "red", false); // Upcast Circle to
        Shape
        System.out.println(s1); // which version?
        System.out.println("Area: " + s1.getArea()); // which
        version?
        System.out.println("Perimeter: " + s1.getPerimeter()); //
        which version?
        System.out.println("Color: " + s1.getColor());
        System.out.println("Filled: " + s1.isFilled());
        // System.out.println(s1.getRadius()); // Compilation error

        Circle c1 = (Circle) s1; // Downcast back to
        Circle
        System.out.println("\nDowncast to Circle:");
        System.out.println(c1);
        System.out.println("Area: " + c1.getArea());
        System.out.println("Perimeter: " + c1.getPerimeter());
        System.out.println("Color: " + c1.getColor());
        System.out.println("Filled: " + c1.isFilled());
        System.out.println("Radius: " + c1.getRadius());

        Shape s2 = new Shape(); //
        System.out.println("\nShape s2:");
        System.out.println(s2);
        // System.out.println(s2.getArea()); // Compilation error
        // System.out.println(s2.getPerimeter()); // Compilation error
        System.out.println("Color: " + s2.getColor());
        System.out.println("Filled: " + s2.isFilled());
        // System.out.println(s2.getWidth()); // Compilation error
    }
}

```

```

Shape s3 = new Rectangle(1.0, 2.0, "red", false);    // Upcast
System.out.println("\nUpcast to Rectangle:");
System.out.println(s3);
System.out.println("Area: " + s3.getArea());
System.out.println("Perimeter: " + s3.getPerimeter());
System.out.println("Color: " + s3.getColor());
// System.out.println(s3.getLength()); // Compilation error

Rectangle r1 = (Rectangle) s3;    // downcast
System.out.println("\nDowncast to Rectangle:");
System.out.println(r1);
System.out.println("Area: " + r1.getArea());
System.out.println("Color: " + r1.getColor());
System.out.println("Length: " + r1.getLength());

Shape s4 = new Square(6.6);        // Upcast
System.out.println("\nUpcast to Square:");
System.out.println(s4);
System.out.println("Area: " + s4.getArea());
System.out.println("Color: " + s4.getColor());
// System.out.println(s4.getSide()); // Compilation error

// Take note that we downcast Shape s4 to Rectangle,
// which is a superclass of Square, instead of Square
Rectangle r2 = (Rectangle) s4;
System.out.println("\nDowncast to Rectangle (from Square):");
System.out.println(r2);
System.out.println("Area: " + r2.getArea());
System.out.println("Color: " + r2.getColor());
System.out.println("Side: " + r2.getSide()); // Accessing
    method from Square
System.out.println("Length: " + r2.getLength());

// Downcast Rectangle r2 to Square
Square sq1 = (Square) r2;
System.out.println("\nDowncast to Square:");
System.out.println(sq1);
System.out.println("Area: " + sq1.getArea());
System.out.println("Color: " + sq1.getColor());
System.out.println("Side: " + sq1.getSide());
System.out.println("Length: " + sq1.getLength());
    }
}

```

Observations:

1. In the line

```
Shape s2 = new Shape();
```

you are trying to instantiate an object of the abstract class Shape, which is not allowed. Abstract classes cannot be directly instantiated.

2. In the line

```
System.out.println(s3.getLength());
```

you are trying to access the `getLength()` method directly on a Shape reference. However, the `getLength()` method is defined in the Rectangle class, not in the Shape class.

3. In the line Rectangle

```
r2 = (Rectangle)s4;
```

you are trying to downcast a Square object to a Rectangle. While this might compile, it is not semantically correct because a Square is a subclass of Rectangle, and downcasting it to a Rectangle might lead to unexpected behavior.

Modified Code:

```
/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */
public class TestShape2 {
    public static void main(String[] args) {
        Circle c1 = new Circle(5.5, "red", false);
        System.out.println(c1);
        System.out.println("Area: " + c1.getArea());
        System.out.println("Perimeter: " + c1.getPerimeter());
        System.out.println("Color: " + c1.getColor());
        System.out.println("Filled: " + c1.isFilled());
        System.out.println("Radius: " + c1.getRadius());

        Shape s1 = new Circle(5.5, "red", false); // Upcast Circle to
        Shape
        System.out.println(s1);
        System.out.println("Area: " + s1.getArea());
        System.out.println("Perimeter: " + s1.getPerimeter());
        System.out.println("Color: " + s1.getColor());
        System.out.println("Filled: " + s1.isFilled());
        // System.out.println("Radius: " + s1.getRadius()); //
        Compilation error

        Circle c2 = (Circle) s1; // Downcast back to Circle
        System.out.println(c2);
        System.out.println("Area: " + c2.getArea());
        System.out.println("Perimeter: " + c2.getPerimeter());
        System.out.println("Color: " + c2.getColor());
        System.out.println("Filled: " + c2.isFilled());
        System.out.println("Radius: " + c2.getRadius());

        // Shape s2 = new Shape(); // Abstract class cannot be
        instantiated

        Shape s3 = new Rectangle(1.0, 2.0, "red", false); // Upcast
        System.out.println(s3);
        System.out.println("Area: " + s3.getArea());
        System.out.println("Perimeter: " + s3.getPerimeter());
        System.out.println("Color: " + s3.getColor());
        // System.out.println("Length: " + s3.getLength()); //
        Compilation error

        Rectangle r1 = (Rectangle) s3; // Downcast
        System.out.println(r1);
        System.out.println("Area: " + r1.getArea());
        System.out.println("Color: " + r1.getColor());
        System.out.println("Length: " + r1.getLength());
    }
}
```

```

        Shape s4 = new Square(6.6); // Upcast
        System.out.println(s4);
        System.out.println("Area: " + s4.getArea());
        System.out.println("Color: " + s4.getColor());
        // System.out.println("Side: " + s4.getSide()); // Compilation
            error

        // Downcast Shape s4 to Square, which is a more appropriate
            downcast than to Rectangle
        Square sq1 = (Square) s4;
        System.out.println(sq1);
        System.out.println("Area: " + sq1.getArea());
        System.out.println("Color: " + sq1.getColor());
        System.out.println("Side: " + sq1.getSide());

        // Downcast Rectangle r1 to Square
        if (r1 instanceof Square sq2) {
            System.out.println(sq2);
            System.out.println("Area: " + sq2.getArea());
            System.out.println("Color: " + sq2.getColor());
            System.out.println("Side: " + sq2.getSide());
        } else {
            System.out.println("Cannot cast Rectangle to Square");
        }

    }
}

```

Result

```

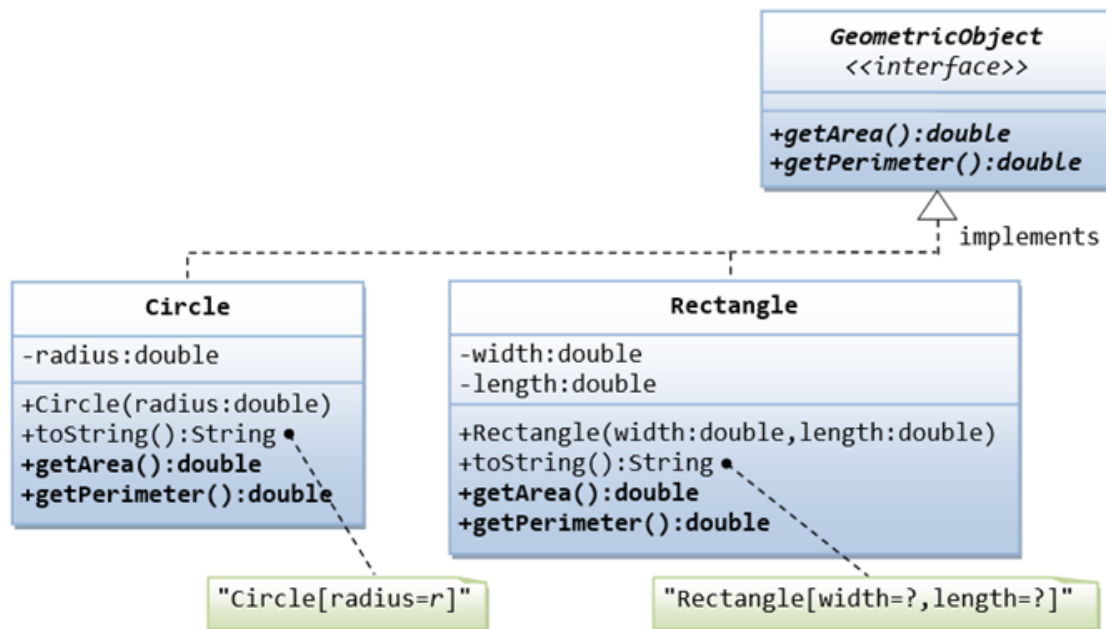
Circle[Shape[color=red,filled=false],radius=5.5]
Area: 95.03317777109123
Perimeter: 34.55751918948772
Color: red
Filled:
Radius: 5.5
Circle[Shape[color=red,filled=false],radius=5.5]
Area: 95.03317777109123
Perimeter: 34.55751918948772
Color: red
Filled: false
Circle[Shape[color=red,filled=false],radius=5.5]
Area: 95.03317777109123
Perimeter: 34.55751918948772
Color: red
Filled: false
Radius: 5.5
Rectangle[Shape[color=red,filled=false],width=1.0,length=2.0]
Area: 2.0
Perimeter: 6.0
Color: red
Rectangle[Shape[color=red,filled=false],width=1.0,length=2.0]
Area: 2.0
Color: red

```

Length: 2.0
 Square[Rectangle[Shape[color=red, filled=true], width=6.6, length=6.6]]
 Area: 43.559999999999995
 Color: red
 Square[Rectangle[Shape[color=red, filled=true], width=6.6, length=6.6]]
 Area: 43.559999999999995
 Color: red
 Side: 6.6
 Cannot cast Rectangle to Square

14.5.2 GeometricObject Interface and its Implementation Classes Circle and Rectangle

Aim: Write an interface called GeometricObject, which contains 2 abstract methods: getArea() and getPerimeter(), as shown in the class diagram. Also write an implementation class called Circle. Mark all the overridden methods with annotation @Override.



Program

GeometricObject.java

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

// GeometricObject interface
public interface GeometricObject {
    double getArea(); // Abstract method to calculate area
    double getPerimeter(); // Abstract method to calculate perimeter
}

```

Circle.java


```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

// Circle class implementing GeometricObject interface
public class Circle implements GeometricObject {
    private double radius; // instance variable

    // Constructor
    public Circle(double radius) {
        this.radius = radius;
    }

    // Getter and setter methods for radius
    public double getRadius() {
        return radius;
    }

    public void setRadius(double radius) {
        this.radius = radius;
    }

    // Implementing abstract method to calculate area
    @Override
    public double getArea() {
        return Math.PI * radius * radius;
    }

    // Implementing abstract method to calculate perimeter
    @Override
    public double getPerimeter() {
        return 2 * Math.PI * radius;
    }
}

```

Rectangle.java

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

// Rectangle class implementing GeometricObject interface
public class Rectangle implements GeometricObject {
    private double width;
    private double length;

    // Constructor
    public Rectangle(double width, double length) {
        this.width = width;
        this.length = length;
    }

    // Getter and setter methods for width and length
    public double getWidth() {
        return width;
    }

```

```

    }

    public void setWidth(double width) {
        this.width = width;
    }

    public double getLength() {
        return length;
    }

    public void setLength(double length) {
        this.length = length;
    }

    // Implementing abstract method to calculate area
    @Override
    public double getArea() {
        return width * length;
    }

    // Implementing abstract method to calculate perimeter
    @Override
    public double getPerimeter() {
        return 2 * (width + length);
    }
}

```

TestGeometricObject.java

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

public class TestGeometricObject {

    public static void main(String[] args) {
        // Test Circle
        GeometricObject circle = new Circle(5.0);
        System.out.println("Circle Area: " + circle.getArea());
        System.out.println("Circle Perimeter: " + circle.getPerimeter()
            );

        // Test Rectangle
        GeometricObject rectangle = new Rectangle(4.0, 6.0);
        System.out.println("Rectangle Area: " + rectangle.getArea());
        System.out.println("Rectangle Perimeter: " + rectangle.
            getPerimeter());

        // Test invalid case (cannot instantiate interface)
        // GeometricObject invalidObject = new GeometricObject(); //
        // This line will result in a compilation error

        // Test GeometricObject array
        GeometricObject[] geometricObjects = new GeometricObject[2];
        geometricObjects[0] = new Circle(3.0);
        geometricObjects[1] = new Rectangle(5.0, 8.0);
    }
}

```

```

        // Display information for each object in the array
        System.out.println("\nInformation for each object in the array\n");
        for (GeometricObject obj : geometricObjects) {
            //System.out.println(obj);
            System.out.println("Area: " + obj.getArea());
            System.out.println("Perimeter: " + obj.getPerimeter());
            System.out.println();
        }
    }
}

```

Result

Circle Area: 78.53981633974483

Circle Perimeter: 31.41592653589793

Rectangle Area: 24.0

Rectangle Perimeter: 20.0

Information for each object in the array

Area: 28.274333882308138

Perimeter: 18.84955592153876

Area: 40.0

Perimeter: 26.0

14.5.3 Movable Interface and its Implementation MovablePoint Class

Aim: Write an interface called Movable, which contains 4 abstract methods moveUp(), moveDown(), moveLeft() and moveRight(), as shown in the class diagram. Also write an implementation class called MovablePoint. Mark all the overridden methods with annotation @Override.

Program

Movable.java

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

// Movable interface
public interface Movable {
    void moveUp(); // Abstract method to move up
    void moveDown(); // Abstract method to move down
    void moveLeft(); // Abstract method to move left
    void moveRight(); // Abstract method to move right
}

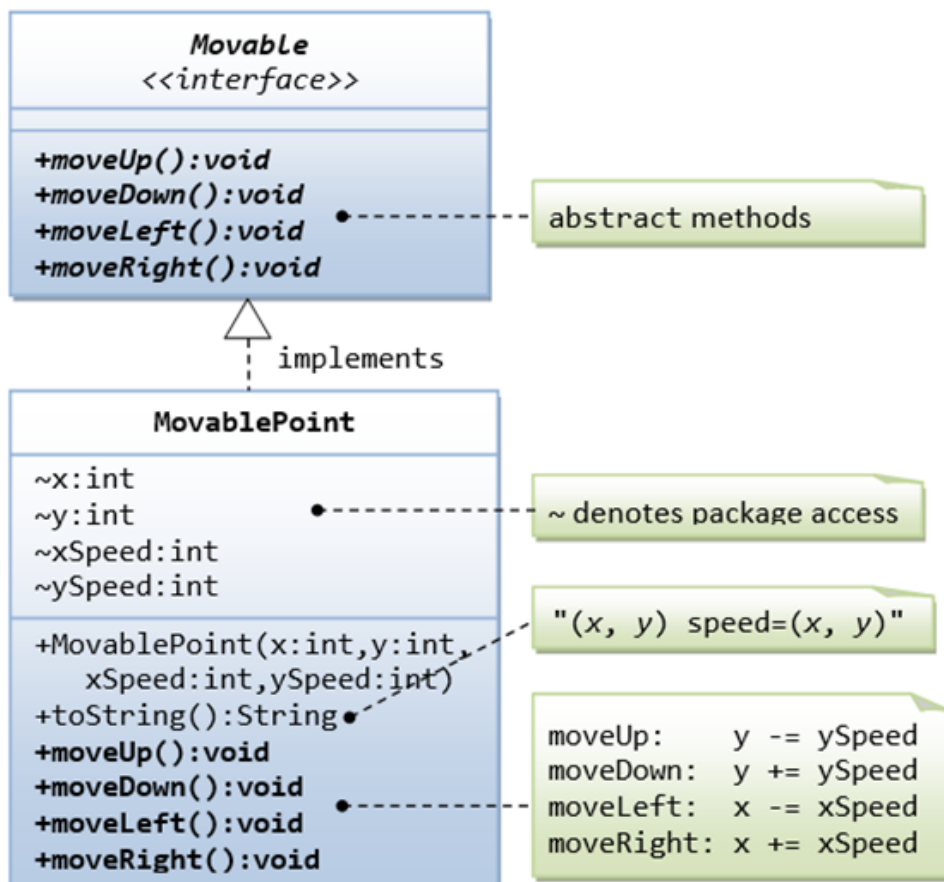
```

MovablePoint.java

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

```



```

// MovablePoint class implementing Movable interface
public class MovablePoint implements Movable {
    private int x;
    private int y;
    private int xSpeed;
    private int ySpeed;

    // Constructor
    public MovablePoint(int x, int y, int xSpeed, int ySpeed) {
        this.x = x;
        this.y = y;
        this.xSpeed = xSpeed;
        this.ySpeed = ySpeed;
    }

    // Implementing abstract method to move up
    @Override
    public void moveUp() {
        y -= ySpeed;
    }

    // Implementing abstract method to move down
    @Override
    public void moveDown() {
        y += ySpeed;
    }

    // Implementing abstract method to move left
    @Override
    public void moveLeft() {
        x -= xSpeed;
    }

    // Implementing abstract method to move right
    @Override
    public void moveRight() {
        x += xSpeed;
    }

    // Additional getter and setter methods
    public int getX() {
        return x;
    }

    public void setX(int x) {
        this.x = x;
    }

    public int getY() {
        return y;
    }

    public void setY(int y) {
        this.y = y;
    }

    public int getXSpeed() {
        return xSpeed;
    }

```

```

    }

    public void setXSpeed(int xSpeed) {
        this.xSpeed = xSpeed;
    }

    public int getYSpeed() {
        return ySpeed;
    }

    public void setYSpeed(int ySpeed) {
        this.ySpeed = ySpeed;
    }

    @Override
    public String toString() {
        return "(" + x + ", " + y + ")" + ", speed=(" + xSpeed + ", " +
            ySpeed + ")";
    }
}

```

TestMovablePoint.java

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

public class TestMovablePoint {
    public static void main(String[] args) {
        // Test MovablePoint
        Movable movablePoint = new MovablePoint(3, 4, 1, 1);
        System.out.println("Original MovablePoint: " + movablePoint);
        movablePoint.moveUp();
        movablePoint.moveLeft();
        System.out.println("After moving up and left: " + movablePoint);
    }
}

```

Result

Original MovablePoint: MovablePoint[x=3, y=4, xSpeed=1, ySpeed=1]

After moving up and left: MovablePoint[x=2, y=3, xSpeed=1, ySpeed=1]

After moving down and right: MovablePoint[x=3, y=4, xSpeed=1, ySpeed=1]

14.5.4 Movable Interface and Classes MovablePoint and MovableCircle

Aim: Write an interface called Movable, which contains 4 abstract methods moveUp(), moveDown(), moveLeft() and moveRight(), as shown in the class diagram. Also write the implementation classes called MovablePoint and MovableCircle. Mark all the overridden methods with annotation @Override.

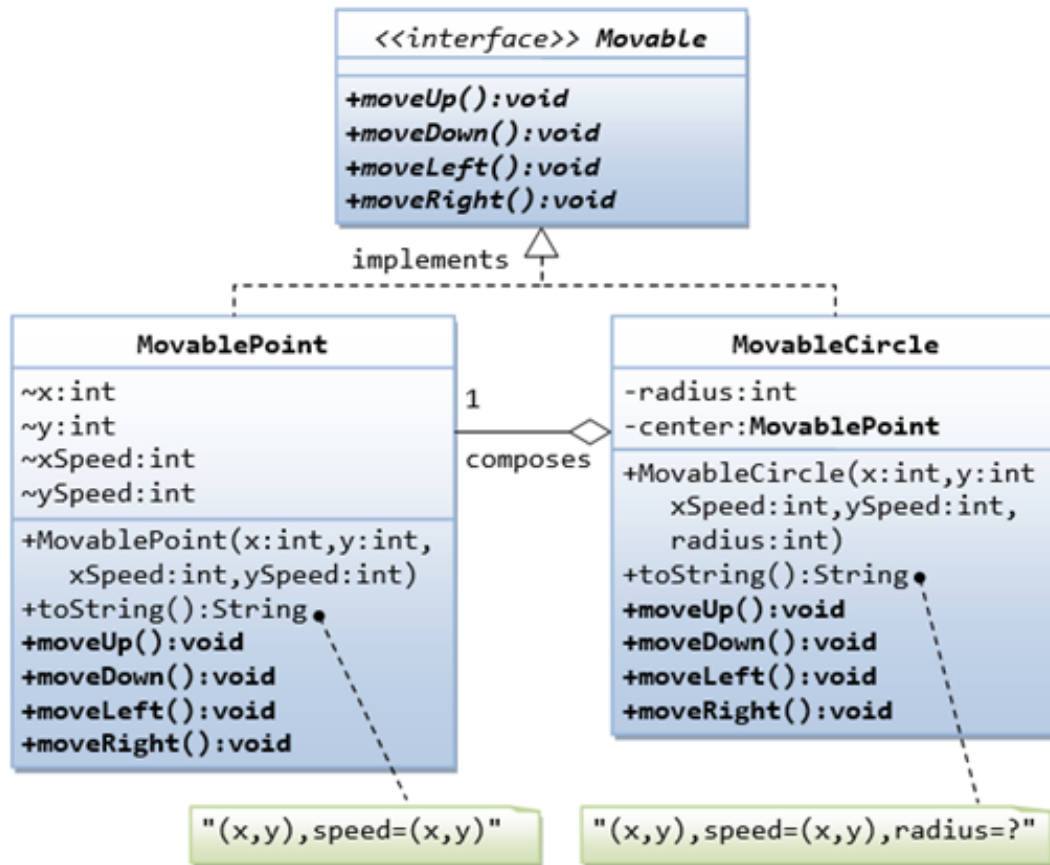
Program

Movable.java

```

/**
 * @author Dr.C V Rama Padmaja

```



```

* Associate Professor, Dept CSE
* IARE, Dundigal.
*/

```

```

// Movable interface
public interface Movable {
    void moveUp();    // Abstract method to move up
    void moveDown();  // Abstract method to move down
    void moveLeft();  // Abstract method to move left
    void moveRight(); // Abstract method to move right
}

```

MovablePoint.java

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

// MovablePoint class implementing Movable interface
public class MovablePoint implements Movable {
    private int x;
    private int y;
    private int xSpeed;
    private int ySpeed;

    // Constructor
    public MovablePoint(int x, int y, int xSpeed, int ySpeed) {

```

```

        this.x = x;
        this.y = y;
        this.xSpeed = xSpeed;
        this.ySpeed = ySpeed;
    }

    // Implementing abstract method to move up
    @Override
    public void moveUp() {
        y -= ySpeed;
    }

    // Implementing abstract method to move down
    @Override
    public void moveDown() {
        y += ySpeed;
    }

    // Implementing abstract method to move left
    @Override
    public void moveLeft() {
        x -= xSpeed;
    }

    // Implementing abstract method to move right
    @Override
    public void moveRight() {
        x += xSpeed;
    }

    // Additional getter and setter methods
    public int getX() {
        return x;
    }

    public void setX(int x) {
        this.x = x;
    }

    public int getY() {
        return y;
    }

    public void setY(int y) {
        this.y = y;
    }

    public int getXSpeed() {
        return xSpeed;
    }

    public void setXSpeed(int xSpeed) {
        this.xSpeed = xSpeed;
    }

    public int getYSpeed() {
        return ySpeed;
    }
}

```



```

    public void setYSpeed(int ySpeed) {
        this.ySpeed = ySpeed;
    }

    @Override
    public String toString() {
        return String.format("MovablePoint[x=%d, y=%d, xSpeed=%d,
            ySpeed=%d]", x, y, xSpeed, ySpeed);
    }
}

```

MovableCircle.java

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

// MovableCircle Implementation
class MovableCircle implements Movable {
    private int radius;
    private MovablePoint center;

    public MovableCircle(int x, int y, int xSpeed, int ySpeed, int
        radius) {
        this.center = new MovablePoint(x, y, xSpeed, ySpeed);
        this.radius = radius;
    }

    @Override
    public void moveUp() {
        center.moveUp();
    }

    @Override
    public void moveDown() {
        center.moveDown();
    }

    @Override
    public void moveLeft() {
        center.moveLeft();
    }

    @Override
    public void moveRight() {
        center.moveRight();
    }

    @Override
    public String toString() {
        return String.format("MovableCircle[center=%s, radius=%d]",
            center, radius);
    }
}

```

TestMovableCircle.java

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

public class TestMovableCircle {
    public static void main(String[] args) {
        Movable movablePoint = new MovablePoint(3, 4, 1, 1);
        Movable movableCircle = new MovableCircle(1, 2, 2, 1, 5);

        System.out.println("Original MovablePoint: " + movablePoint);
        movablePoint.moveUp();
        movablePoint.moveLeft();
        System.out.println("After moving up and left: " + movablePoint);
        ;

        System.out.println("\nOriginal MovableCircle: " + movableCircle);
        ;
        movableCircle.moveDown();
        movableCircle.moveRight();
        movableCircle.moveDown();
        movableCircle.moveRight();
        System.out.println("After moving down and right: " +
            movableCircle);
    }
}

```

Result

Original MovablePoint: MovablePoint[x=3, y=4, xSpeed=1, ySpeed=1]

After moving up and left: MovablePoint[x=2, y=3, xSpeed=1, ySpeed=1]

Original MovableCircle: MovableCircle[center=MovablePoint[x=1, y=2, xSpeed=2, ySpeed=1], radius=5]

After moving down and right: MovableCircle[center=MovablePoint[x=5, y=4, xSpeed=2, ySpeed=1], radius=5]

14.5.5 Interfaces Resizable and GeometricObject

Aim: Write the interface called GeometricObject, which declares two abstract methods: getParameter() and getArea(), as specified in the class diagram.

Hints:

```

public interface GeometricObject {
    public double getPerimeter();
    .....
}

```

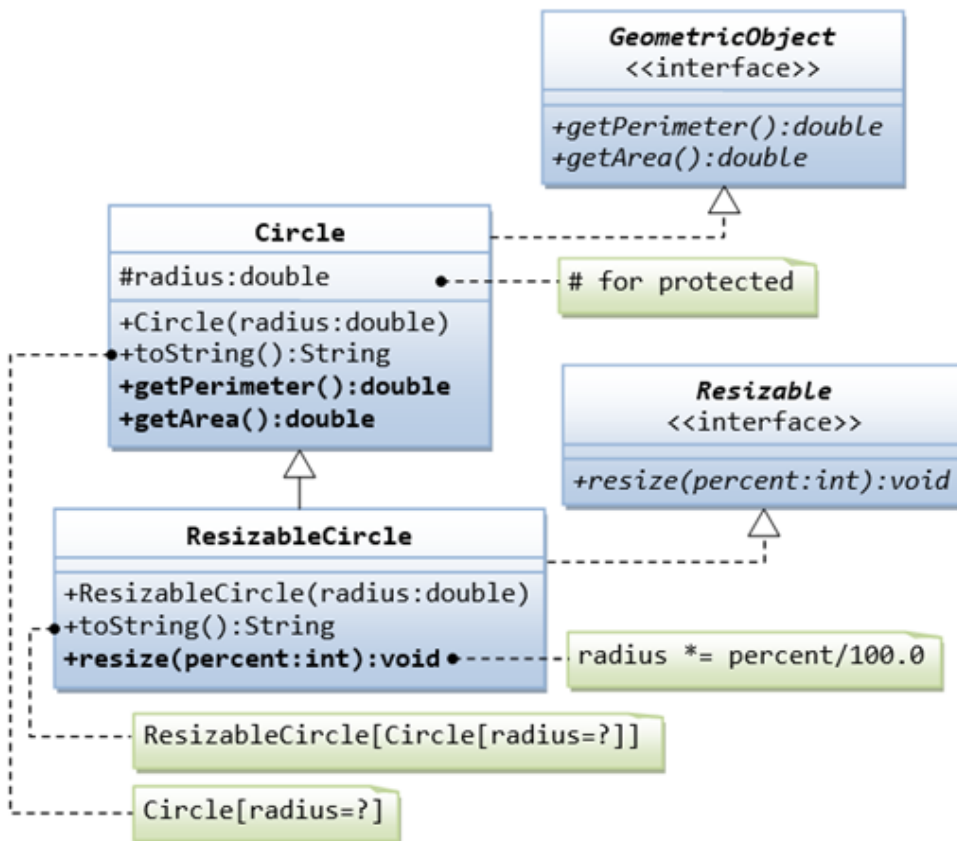
Write the implementation class Circle, with a protected variable radius, which implements the interface GeometricObject.

Hints:

```

public class Circle implements GeometricObject {
    // Private variable

```



```

.....

// Constructor
.....

// Implement methods defined in the interface GeometricObject
@Override
public double getPerimeter() { ..... }
.....
}

```

Write a test program called `TestCircle` to test the methods defined in `Circle`.

The class `ResizableCircle` is defined as a subclass of the class `Circle`, which also implements an interface called `Resizable`, as shown in class diagram. The interface `Resizable` declares an abstract method `resize()`, which modifies the dimension (such as radius) by the given percentage. Write the interface `Resizable` and the class `ResizableCircle`.

Hints:

```

public interface Resizable {
    public double resize(...);
}

```

```

public class ResizableCircle extends Circle implements Resizable {

    // Constructor
    public ResizableCircle(double radius) {

```

```

        super(...);
    }

    // Implement methods defined in the interface Resizable
    @Override
    public double resize(int percent) { ..... }
}

```

Program

GeometricObject.java

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

// GeometricObject interface
public interface GeometricObject {
    double getArea();          // Abstract method to calculate area
    double getPerimeter();     // Abstract method to calculate perimeter
}

```

Circle.java

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

// Circle class implementing GeometricObject interface
public class Circle implements GeometricObject {
    double radius; // instance variable

    // Constructor
    public Circle(double radius) {
        this.radius = radius;
    }

    // Getter and setter methods for radius
    public double getRadius() {
        return radius;
    }

    public void setRadius(double radius) {
        this.radius = radius;
    }

    // Implementing abstract method to calculate area
    @Override
    public double getArea() {
        return Math.PI * radius * radius;
    }

    // Implementing abstract method to calculate perimeter
    @Override
    public double getPerimeter() {
        return 2 * Math.PI * radius;
    }
}

```

```
}
```

Resizable.java

```
/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

// Resizable Interface
public interface Resizable {
    double resize(int percent);
}
```

ResizableCircle.java

```
/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

// ResizableCircle Implementation
public class ResizableCircle extends Circle implements Resizable {
    public ResizableCircle(double radius) {
        super(radius);
    }

    @Override
    public double resize(int percent) {
        double scaleFactor = 1.0 + percent / 100.0;
        radius *= scaleFactor;
        return radius;
    }
}
```

TestResizableCircle.java

```
/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

// TestResizableCircle
public class TestResizableCircle {
    public static void main(String[] args) {
        ResizableCircle resizableCircle = new ResizableCircle(5.0);

        System.out.println("Original Radius: " + resizableCircle.
            getRadius());
        System.out.println("Original Perimeter: " + resizableCircle.
            getPerimeter());
        System.out.println("Original Area: " + resizableCircle.getArea
            ());

        int resizePercentage = 50;
        resizableCircle.resize(resizePercentage);

        System.out.println("\nAfter Resizing by " + resizePercentage +
            "%");
    }
}
```

```

        System.out.println("New Radius: " + resizableCircle.radius);
        System.out.println("New Perimeter: " + resizableCircle.
            getPerimeter());
        System.out.println("New Area: " + resizableCircle.getArea());
    }
}

```

Result

Original Radius: 5.0

Original Perimeter: 31.41592653589793

Original Area: 78.53981633974483

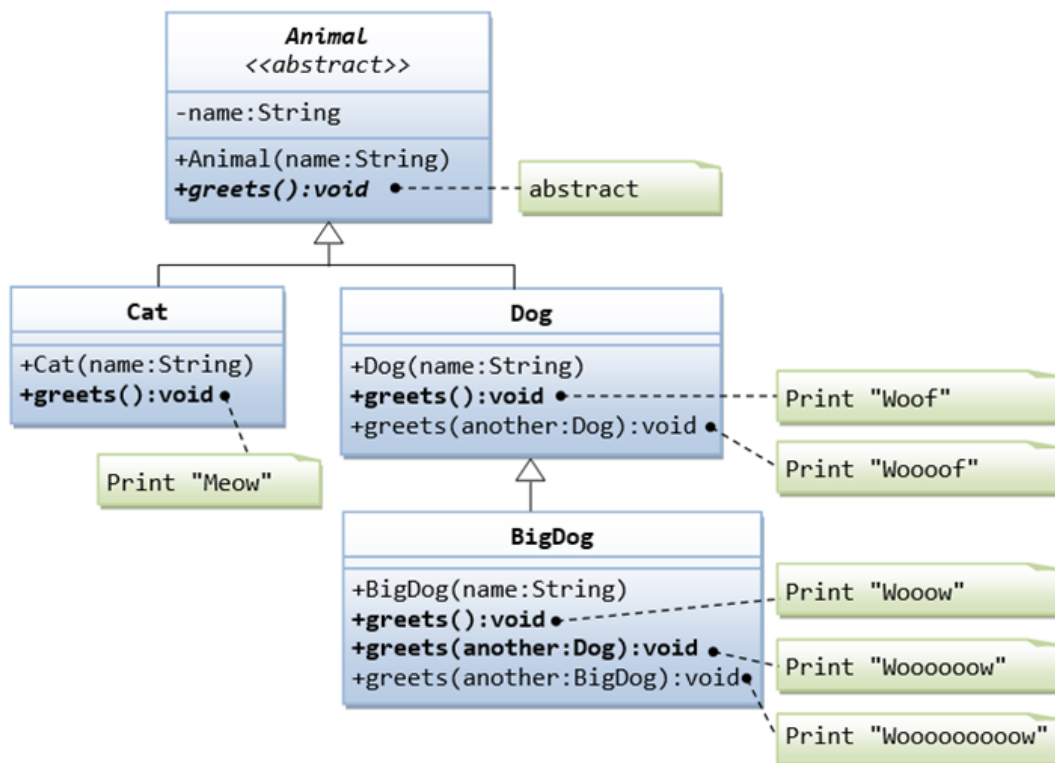
After Resizing by 50New Radius: 7.5

New Perimeter: 47.12388980384689

New Area: 176.71458676442586

14.5.6 Abstract Superclass Animal and its Implementation Subclasses

Aim: Write the codes for all the classes shown in the class diagram. Mark all the overridden methods with annotation @Override.



Program

Animal.java

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

```

```
// Abstract Animal class
abstract public class Animal {
    abstract public void greeting();
}
```

Cat.java

```
/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

// Cat class, subclass of Animal
public class Cat extends Animal {
    @Override
    public void greeting() {
        System.out.println("Meow!");
    }
}
```

Dog.java

```
/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

// Dog class, subclass of Animal
public class Dog extends Animal {
    @Override
    public void greeting() {
        System.out.println("Woof!");
    }

    // Additional method in Dog class
    public void greeting(Dog another) {
        System.out.println("Wooooooooooof!");
    }
}
```

BigDog.java

```
/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

// BigDog class, subclass of Dog
public class BigDog extends Dog {
    @Override
    public void greeting() {
        System.out.println("Woow!");
    }

    // Override method with a different signature
    @Override
    public void greeting(Dog another) {
```

```

        System.out.println("Woooooowwww!");
    }
}

```

TestAnimals.java

```

/**
 * @author Dr.C V Rama Padmaja
 * Associate Professor, Dept CSE
 * IARE, Dundigal.
 */

public class TestAnimals {
    public static void main(String[] args) {
        // Using the subclasses
        Cat cat1 = new Cat();
        cat1.greeting(); // Output: Meow!

        Dog dog1 = new Dog();
        dog1.greeting(); // Output: Woof!

        BigDog bigDog1 = new BigDog();
        bigDog1.greeting(); // Output: Woow!

        // Using Polymorphism
        Animal animal1 = new Cat();
        animal1.greeting(); // Output: Meow!

        Animal animal2 = new Dog();
        animal2.greeting(); // Output: Woof!

        Animal animal3 = new BigDog();
        animal3.greeting(); // Output: Woow!

        // Animal animal4 = new Animal();
        // Error: Animal is an abstract class and cannot be
        // instantiated directly.

        // Downcast
        Dog dog2 = (Dog) animal2;
        BigDog bigDog2 = (BigDog) animal3;
        Dog dog3 = (Dog) animal3;
        //Cat cat2 = (Cat) animal2; // Runtime error: Dog cannot be
        // cast to Cat

        dog2.greeting(dog3); // Output: Wooooooooooof!
        dog3.greeting(dog2); // Output: Woooooowwww!
        dog2.greeting(bigDog2); // Output: Wooooooooooof!
        bigDog2.greeting(dog2); // Output: Woooooowwww!
        bigDog2.greeting(bigDog1); // Output: Woooooowwww!
    }
}

```

Result

Meow!
 Woof!
 Woow!
 Meow!

Woof!
Woow!
Wooooooooooooof!
Wooooooooowwww!
Wooooooooooooof!
Wooooooooowwww!
Wooooooooowwww!

14.6 Sample Viva Voce Questions

1. Can you demonstrate how interfaces allow for code reuse and extensibility in Java?
2. How do interfaces facilitate loose coupling and flexibility in Java applications?
3. Can a class implement multiple interfaces in Java? If yes, how does it achieve this?
4. Provide a real-world scenario where polymorphism, abstract classes, and interfaces would be beneficial in Java application development.

14.7 Further Probing Experiments

1. Design a MyRectangle class which is composed of two MyPoint instances as its top-left and bottom-right corners. Draw the class diagrams, write the codes, and write the test drivers.
2. Develop classes that shows the difference between an interface and an abstract class